



ChatGPT

Секреты метапрограммирования Python, который пишет Python

Модуль 1 - Врата метамагии

Урок 1 - Что такое метапрограммирование

Шаг 01:

Миссия 1

Программист Артём услышал на митапе слово «метапрограммирование». Что это значит?

Подсказка: Подумай о коде, который может писать или изменять код.

Ответ:

- Программирование, где код может создавать или модифицировать другой код
- Методика написания комментариев к коду
- Процесс запуска тестов автоматически
- Синоним обычного объектно-ориентированного программирования

Шаг 02:

Миссия 2

Алиса пишет Python-скрипт, который генерирует другой Python-файл. К какому направлению это относится?

Подсказка: Это ключевой пример нашей темы.

Ответ:

- Метапрограммирование

- Машинное обучение
- Скриптовое программирование
- Функциональное программирование

🔮 Шаг 03:

Миссия 3

Борис спорит с коллегой: «Метапрограммирование помогает избавиться от повторяющегося кода». Прав ли он?

Подсказка: Вспомни, что можно генерировать шаблонные куски кода вместо ручного копирования.

Ответ:

- **Да, метапрограммирование может автоматизировать создание однотипных частей**
- Нет, оно используется только для визуализации данных

🔮 Шаг 04:

Миссия 4

Ваня пишет декоратор, который автоматически логирует вызовы функций. К какой технике Python это относится?

Подсказка: Декораторы — один из популярных инструментов метапрограммирования.

Ответ:

- **Метапрограммирование через декораторы**
- Асинхронное программирование
- Работа с сетями
- Программирование на уровне железа

🔮 Шаг 05:

Миссия 5

Саша использует функцию `type()`, чтобы динамически создавать новый класс. Что это демонстрирует?

Подсказка: В Python классы — тоже объекты, и их можно создавать на лету.

Ответ:

- **Создание классов во время выполнения (метапрограммирование)**
- Обычное наследование
- Работу с циклами
- Рекурсию

🔮 Шаг 06:

Миссия 6

Марина исследует модуль `inspect`, чтобы узнать аргументы функции в рантайме. К какой категории приёмов это относится?

Подсказка: Эта техника позволяет заглянуть в сам код и структуру объектов.

Ответ:

- **Рефлексия (часть метапрограммирования)**
- Инкапсуляция
- Тестирование

- Сериализация

🔮 Шаг 07:

Миссия 7

Игорь сгенерировал Python-код строкой и выполнил его через `exec`. Это...

Подсказка: Иногда опасно, но демонстрирует мощь динамики Python.

Ответ:

- **Метапрограммирование через генерацию и выполнение кода**
- Оптимизация производительности
- Работа с файлами

🔮 Шаг 08:

Миссия 8

Соотнеси ключевые понятия с их объяснениями:

Подсказка: Вспомни, что метакод — это код о коде.

Ответ:

- **Рефлексия** — Изучение структуры программы во время выполнения
- **Декоратор** — Функция, модифицирующая другую функцию
- **Генерация кода** — Создание исходного текста программы автоматически
- **`type()`** — Инструмент для динамического создания классов

🔮 Шаг 09:

Миссия 9

Настя спорит с другом: «`eval()` и `exec()` похожи». Соотнеси, что делает каждая функция:

Подсказка: Одна возвращает результат, другая — просто выполняет код.

Ответ:

- **`eval()`** — Вычисляет выражение и возвращает значение
- **`exec()`** — Выполняет произвольный код без возврата результата

🔮 Шаг 10:

Миссия 10

Коля использует декоратор для проверки прав доступа. Сопоставь объект и его роль:

Подсказка: Декоратор может добавить дополнительное поведение функции.

Ответ:

- **Декоратор** — Функция, принимающая и возвращающая другую функцию
- **Проверка прав** — Дополнительная логика, оборачиваемая вокруг вызова
- **Исходная функция** — Основное действие, которое выполняется
- **Результат** — Функция с расширенным поведением

🔮 Шаг 11:

Миссия 11

Программисты обсуждают магические методы. Сопоставь метод и его назначение:

Подсказка: Эти методы начинаются и заканчиваются двойным подчёркиванием.

Ответ:

- **`__init__`** — Конструктор объекта
- **`__str__`** — Строковое представление объекта
- **`__call__`** — Делает объект вызываемым как функцию
- **`__getattr__`** — Вызывается при обращении к несуществующему атрибуту

Шаг 12:

Миссия 12

Дима пишет фреймворк. Ему нужно автоматически регистрировать плагины. Сопоставь технику с описанием:

Подсказка: Вспомни про возможности Python работать с объектами во время выполнения.

Ответ:

- **Метакласс** — Класс, создающий другие классы
- **Декоратор регистрации** — Добавляет функцию в список доступных плагинов
- **`inspect`** — Позволяет анализировать параметры функций
- **`globals()`** — Возвращает словарь глобальных объектов

Шаг 13:

Миссия 13

📋 Упорядочь шаги создания простого декоратора:

Подсказка: Сначала определяем функцию, потом пишем внутреннюю, затем возвращаем её.

Ответ:

- Определить функцию-декоратор
- Внутри неё создать обёртку (wrapper)
- В обёртке вызвать исходную функцию
- Вернуть обёртку из декоратора

Шаг 14:

Миссия 14

📋 Упорядочь этапы работы eval:

Подсказка: Сначала строка попадает в Python-интерпретатор, потом анализируется, вычисляется и возвращается результат.

Ответ:

- Передача строки в eval
- Анализ выражения
- Вычисление выражения
- Возврат результата

Шаг 15:

Миссия 15

📋 Упорядочь шаги при создании класса с помощью `type()`:

Подсказка: Нужно дать имя, список родителей, словарь атрибутов, а затем получить новый класс.

Ответ:

- Указать имя нового класса
- Передать кортеж базовых классов
- Создать словарь с методами и атрибутами
- Вызвать `type()` с этими параметрами

 Шаг 16:

Миссия 16

📖 Упорядочь этапы написания простого метакласса:

Подсказка: Сначала создаётся класс-наследник `type`, затем переопределяются методы, и только после этого используется для генерации классов.

Ответ:

- Создать класс, наследующий от `type`
- Переопределить метод `__new__` или `__init__`
- Определить дополнительные проверки или модификации
- Использовать метакласс при объявлении нового класса

 Шаг 17:

Миссия 17

📖 Упорядочь шаги анализа функции с помощью `inspect`:

Подсказка: Сначала импортируем модуль, потом вызываем функцию анализа, а затем используем результат.

Ответ:

- Импортировать модуль `inspect`
- Вызвать `inspect.signature(func)`
- Получить объект `Signature`
- Проанализировать параметры функции

 Урок 2 - Магия introspection в Python

 Шаг 01:

Миссия 1

Разработчик Артём захотел узнать, какие методы доступны у объекта строки в Python. Какой встроенной функцией он воспользуется?

Подсказка: В Python есть функция, которая возвращает список атрибутов объекта.

Ответ:

- `dir()`
- `type()`
- `id()`
- `help()`

 Шаг 02:

Миссия 2

Программистка Оля проверяет, является ли объект списком. Какую функцию она применит?

Подсказка: Эта функция возвращает класс объекта.

Ответ:

- **type()**
- dir()
- id()
- vars()

🔮 **Шаг 03:**

Миссия 3

Алекс запускает отладку и хочет получить документацию прямо в консоли Python. Какой инструмент он вызовет?

Подсказка: Эта команда встроена и показывает docstring объекта.

Ответ:

- **help()**
- doc()
- print()
- show()

🔮 **Шаг 04:**

Миссия 4

Марина пишет плагин и хочет узнать все переменные текущего объекта. Какой функцией стоит воспользоваться?

Подсказка: Эта функция возвращает словарь ****dict**** объекта.

Ответ:

- **vars()**
- type()
- id()
- dir()

🔮 **Шаг 05:**

Миссия 5

Разработчик Иван хочет узнать, где именно в памяти хранится объект. Какую функцию он вызовет?

Подсказка: Эта функция возвращает уникальный идентификатор объекта, обычно адрес.

Ответ:

- **id()**
- dir()
- vars()
- hash()

🔮 **Шаг 06:**

Миссия 6

Сопоставьте функцию introspection с её назначением.

Подсказка: Подумайте, что именно возвращает каждая функция: тип, атрибуты, память или документацию.

Ответ:

- **type()** — Возвращает класс объекта
- **dir()** — Возвращает список атрибутов объекта
- **id()** — Возвращает уникальный идентификатор объекта
- **help()** — Выводит документацию объекта

🔮 Шаг 07:

Миссия 7

Сопоставьте разработчика и его цель с правильным инструментом introspection.

Подсказка: У каждого инструмента своя роль: docstring, атрибуты, класс, словарь.

Ответ:

- **Артём хочет узнать документацию функции** — help()
- **Марина проверяет все переменные объекта** — vars()
- **Иван ищет список методов строки** — dir()
- **Оля хочет убедиться, что переменная** — список — type()

🔮 Шаг 08:

Миссия 8

Сопоставьте специальные атрибуты объектов Python с их описанием.

Подсказка: Эти атрибуты начинаются и заканчиваются двойным подчёркиванием.

Ответ:

- ****dict**** — Словарь атрибутов объекта
- ****doc**** — Строка документации
- ****name**** — Имя объекта
- ****class**** — Ссылка на класс объекта

🔮 Шаг 09:

Миссия 9

Сопоставьте инструменты introspection с примером их использования.

Подсказка: Вспомните, что именно выводит каждая команда.

Ответ:

- **type([1,2,3])** —
- **dir('hi')** — ['**add**', '**class**', ...]
- **id(42)** — Числовой идентификатор
- **help(len)** — Документация функции len

🔮 Шаг 10:

Миссия 10

Сопоставьте инструмент introspection с ситуацией программиста.

Подсказка: Ориентируйтесь на цель: узнать документацию, атрибуты или класс.

Ответ:

- **dir()** — Программист ищет доступные методы объекта
- **help()** — Разработчику нужна документация функции
- **type()** — Нужно определить тип переменной

- **vars()** — Получить все атрибуты объекта в виде словаря

Шаг 11:

Миссия 11

Сопоставьте встроенные средства introspection с их результатами.

Подсказка: Сравните вывод команд в Python REPL.

Ответ:

- **help(str)** — Документация класса str
- **id('x')** — Числовой адрес объекта
- **vars(obj)** — Словарь атрибутов объекта
- **type(3.14)** —

Шаг 12:

Миссия 12

Упорядочите шаги, которые выполняет Даша, изучая новый модуль с помощью introspection.

Подсказка: Сначала документация, потом атрибуты, затем детали конкретных методов.

Ответ:

- Вызвать help(module)
- Посмотреть список функций через dir(module)
- Открыть документацию конкретной функции
- Протестировать функцию в коде

Шаг 13:

Миссия 13

Определи порядок действий Пети, когда он проверяет новый объект в Python.

Подсказка: Обычно сначала определяют тип, потом изучают методы, затем обращаются к документации.

Ответ:

- Вызвать type(obj)
- Вызвать dir(obj)
- Вызвать help(obj)
- Применить функцию из объекта

Шаг 14:

Миссия 14

Расположите шаги, которые выполняет разработчик, исследуя атрибуты объекта через ****dict****.

Подсказка: Вспомните, что сначала обращаются к объекту, потом смотрят его словарь.

Ответ:

- Создать объект класса
- Вызвать vars(obj)
- Посмотреть ключи в ****dict****
- Изучить значения атрибутов

Шаг 15:

Миссия 15

Упорядочите действия Саши, который хочет понять, какие методы наследуются классом.

Подсказка: Сначала определите класс, потом используйте `dir()`, затем ищите родительские методы.

Ответ:

- Определить класс
- Вызвать `dir(класс)`
- Сравнить методы с базовым классом
- Выделить новые методы

Шаг 16:

Миссия 16

Расставь порядок шагов, которые делает программист, используя introspection для функции.

Подсказка: Обычно сначала смотрят тип, затем атрибуты, потом документацию, наконец тестируют.

Ответ:

- `type(func)`
- `dir(func)`
- `help(func)`
- Вызвать `func()` в коде

Шаг 17:

Миссия 17

Программист Андрей исследует новый класс. Расставьте шаги в правильном порядке.

Подсказка: Сначала узнаем тип и документацию, потом список методов, затем тестируем.

Ответ:

- `type(MyClass)`
- `help(MyClass)`
- `dir(MyClass)`
- Создать экземпляр и протестировать метод

Урок 3 - eval и exec заклинания исполнения

Шаг 01:

Миссия 1

Какое главное отличие между `eval` и `exec` ?

Подсказка: Один из них что-то возвращает, другой — только выполняет.

Ответ:

- **eval возвращает значение, а exec — нет**
- `exec` быстрее, чем `eval`
- `eval` всегда работает только с числами
- `exec` используется только в C++

Шаг 02:

Миссия 2

Расположите шаги при использовании eval в правильном порядке.

Подсказка: Сначала формируем строку, потом проверяем, потом выполняем.

Ответ:

- Создать строковое выражение
- Проверить безопасность ввода
- Передать строку в eval
- Получить результат

Шаг 03:

Миссия 3

Расположите шаги при работе exes для создания переменной.

Подсказка: Сначала пишем код, потом запускаем, потом используем.

Ответ:

- Составить строку 'x=42'
- Передать её в exes
- Переменная x появляется в окружении
- Использовать x в коде

Шаг 04:

Миссия 4

Упорядочьте этапы безопасного использования eval.

Подсказка: Необходимо фильтровать ввод.

Ответ:

- Ограничить доступные функции
- Создать строку выражения
- Вызвать eval с ограниченным словарем
- Получить результат

Шаг 05:

Миссия 5

Расположите шаги для выполнения нескольких команд через exes.

Подсказка: exes принимает много строк сразу.

Ответ:

- Создать строку с несколькими командами
- Передать строку в exes
- Все команды выполняются
- Результат сохраняется в окружении

Шаг 06:

Миссия 7

Расположите шаги демонстрации на митапе о метапрограммировании.

Подсказка: Логично сначала объяснить eval, потом exec, потом показать пример их связки.

Ответ:

- Показать пример eval
- Объяснить exec
- Показать exec в действии
- Соединить eval и exec в одном примере

Шаг 07:

Миссия 8

Чем опасен код: `exec(input())` ?

Подсказка: Представьте, что кто-то введет команду на удаление файлов.

Ответ:

- **Можно выполнить вредоносный код**
- exec не принимает ввод
- exec всегда ломает программу
- exec не работает в Python 3

Шаг 08:

Миссия 9

Расставьте шаги для объяснения отличий eval и exec ученику.

Подсказка: Сначала про eval, потом про exec, потом про разницу.

Ответ:

- Показать пример с eval
- Показать пример с exec
- Сравнить результаты
- Объяснить, когда использовать каждый

Шаг 09:

Миссия 10

Расставьте шаги при использовании exec для запуска функции из строки.

Подсказка: Нужно сначала определить функцию, потом вызвать её.

Ответ:

- Создать строку с определением функции
- Передать строку в exec
- Функция появляется в окружении
- Вызвать функцию

Шаг 10:

Миссия 10

Аня пишет интерактивный скрипт и хочет выполнить строку кода, которую пользователь вводит сам. Она думает о eval . Что делает eval в Python?

Подсказка: Представьте себе калькулятор, который считает выражение в строке.

Ответ:

- **Вычисляет строку как выражение Python и возвращает результат**

- Превращает строку в список символов
- Сохраняет строку в файл и запускает его
- Выводит строку без изменений

🔮 Шаг 11:

Миссия 11

Игорь работает над чат-ботом. Пользователь прислал команду в виде кода, и Игорь применил `exec`. Что делает `exec`?

Подсказка: Это похоже на сценарий, который выполняется целиком, а не просто выражение.

Ответ:

- **Выполняет строку как Python-код (включая несколько строк)**
- Сохраняет строку в переменную без выполнения
- Вычисляет строку как математическое выражение
- Удаляет строку из памяти

🔮 Шаг 12:

Миссия 12

Сопоставьте правильный инструмент с его описанием.

Подсказка: `eval` работает с выражениями, `exec` с блоками кода.

Ответ:

- **`eval`** — Вычисляет выражение и возвращает результат
- **`exec`** — Выполняет произвольный код без возврата
- **`print`** — Выводит данные в консоль
- **`input`** — Получает строку от пользователя

🔮 Шаг 13:

Миссия 13

Расставьте шаги по безопасному использованию `eval` в порядке выполнения.

Подсказка: Сначала фильтрация, потом проверка, затем выполнение.

Ответ:

- Ограничить набор допустимых выражений
- Проверить ввод пользователя
- Выполнить через `eval`
- Использовать результат в программе

🔮 Шаг 14:

Миссия 14

Расставьте этапы работы `exec` при выполнении многострочного кода.

Подсказка: `exec` сначала компилирует, потом выполняет.

Ответ:

- Компиляция строки в байткод
- Выполнение инструкций
- Изменение локальных переменных
- Вывод результата (если есть)

Шаг 15:

Миссия 15

Расставьте действия программиста в порядке, если он хочет выполнить пользовательский код через `exes`.

Подсказка: Никогда не запускаем код сразу, сначала проверяем.

Ответ:

- Получить код от пользователя
- Проверить код на опасные конструкции
- Выполнить с помощью `exes`
- Проверить результат

Шаг 16:

Миссия 16

Определи порядок работы при комбинировании `eval` и `exes` в проекте.

Подсказка: `eval` возвращает данные, `exes` выполняет действия.

Ответ:

- Получить выражение и вычислить через `eval`
- Сохранить результат в переменную
- Передать переменную в `exes` для дальнейшего использования
- Вывести итог

Шаг 17:

Миссия 17

Расположите этапы в порядке, если разработчик хочет безопасно встроить `eval` в калькулятор.

Подсказка: Нужно проверять допустимые символы.

Ответ:

- Проверить строку на допустимые символы
- Удалить лишние пробелы
- Вычислить через `eval`
- Показать пользователю результат

Шаг 18:

Миссия 18

Расставьте шаги работы над проектом, где используется `exes` для генерации функций.

Подсказка: Сначала формируем код, потом выполняем.

Ответ:

- Сформировать строку с кодом функции
- Передать строку в `exes`
- Определить функцию в пространстве имён
- Вызвать функцию

Урок 4 - Практика функции, что создают функции

Шаг 01:

Миссия 1

Андрей написал функцию, которая возвращает другую функцию. Как в Python называется такой приём?

Подсказка: Вспомните, что функции в Python — объекты первого класса.

Ответ:

- **Функции высшего порядка**
- Замыкание классов
- Статическая типизация
- Сериализация объектов

Шаг 02:

Миссия 2

Лена создала функцию `make_multiplier(n)`, которая возвращает функцию умножения на `n`. Что произойдёт при вызове `make_multiplier(3)(5)` ?

Подсказка: Аргумент `n` хранится внутри возвращённой функции.

Ответ:

- **Вернётся 15**
- Вернётся 8
- Будет ошибка, так как нет аргументов
- Вернётся сама функция

Шаг 03:

Миссия 3

Игорь хочет создать функцию, которая возвращает разные приветствия в зависимости от имени. Как правильно использовать вложенные функции?

Подсказка: Вложенные функции могут обращаться к переменным внешней функции.

Ответ:

- **Определить функцию внутри другой и вернуть её**
- Использовать глобальные переменные
- Сначала объявить класс, затем метод
- Создать словарь вместо функции

Шаг 04:

Миссия 4

Маша использует функцию, которая возвращает другую функцию для вычисления площади фигур. Что это за концепция?

Подсказка: Это связано с динамическим созданием поведения через функции.

Ответ:

- **Замыкание**
- Полиморфизм
- Множественное наследование
- Итераторы

🔮 Шаг 05:

Миссия 5

Кирилл написал функцию, которая создаёт новую функцию возведения в степень. Что хранит внутренняя функция?

Подсказка: Она запоминает значения из внешнего контекста.

Ответ:

- **Значение аргументов внешней функции**
- Все глобальные переменные
- Только имя функции
- Только результат вызова

🔮 Шаг 06:

Миссия 6

Сопоставьте программистские ситуации с концепциями Python.

Подсказка: Подумайте, какие термины описывают возврат функций и хранение состояния.

Ответ:

- **Функция возвращает другую функцию** — Функция высшего порядка
- **Функция хранит аргументы из внешней области** — Замыкание
- **Создание функций внутри функций** — Вложенные функции
- **Передача функции как аргумента** — Функции как объекты первого класса

🔮 Шаг 07:

Миссия 7

Сопоставьте примеры кода с правильными описаниями.

Подсказка: Важно увидеть, где функция создаётся и что возвращается.

Ответ:

- **def outer(): def inner(): return 5; return inner** — Внешняя функция возвращает внутреннюю
- **def maker(x): return lambda y: x+y** — Создание функции через лямбду
- **def wrap(func): return func** — Функция принимает и возвращает функцию
- **def const(): return 42** — Функция возвращает значение, а не функцию

🔮 Шаг 08:

Миссия 8

Сопоставьте задачи разработчиков с приёмами метапрограммирования.

Подсказка: Разные цели достигаются разными конструкциями функций.

Ответ:

- **Создать функцию генератор приветствий** — Вложенные функции
- **Передать сортировку как параметр** — Функция как аргумент
- **Запомнить коэффициент скидки** — Замыкание
- **Вернуть функцию умножителя** — Функция высшего порядка

🔮 Шаг 09:

Миссия 9

Сопоставьте понятия с их практическим применением.

Подсказка: Вспомните, что делают замыкания, вложенные функции и функции высшего порядка.

Ответ:

- **Замыкание** — Хранит состояние между вызовами
- **Вложенные функции** — Служат для структурирования логики внутри функции
- **Функция высшего порядка** — Принимает или возвращает функции
- **Лямбда** — Создаёт краткие анонимные функции

🔗 Шаг 10:

Миссия 10

Сопоставьте описание с примером кода.

Подсказка: Важно увидеть, где возвращается функция, а где — результат.

Ответ:

- **Функция возвращает функцию** — `def outer(): def inner(): return 1; return inner`
- **Функция хранит значение из внешней области** — `def maker(n): return lambda x: x*n`
- **Функция используется как аргумент** — `map(str, [1,2,3])`
- **Функция возвращает константу** — `def const(): return 7`

🔗 Шаг 11:

Миссия 11

Расположите шаги создания функции-генератора коэффициентов в правильном порядке.

Подсказка: Сначала пишем внешнюю функцию, внутри объявляем внутреннюю, потом возвращаем её и используем.

Ответ:

- Определить внешнюю функцию с параметром
- Определить внутреннюю функцию с использованием параметра
- Вернуть внутреннюю функцию
- Вызвать внешнюю и затем внутреннюю функцию

🔗 Шаг 12:

Миссия 12

Расставьте шаги создания функции `make_power`, которая создаёт функции возведения в степень.

Подсказка: Сначала определяем внешнюю функцию, затем внутреннюю, а потом используем.

Ответ:

- Определить функцию `make_power` с параметром `n`
- Внутри объявить функцию `power(x)` возвращающую `x**n`
- Вернуть функцию `power`
- Вызвать `make_power(2)(5)`

🔗 Шаг 13:

Миссия 13

Расставьте шаги при создании функции, которая возвращает калькулятор с запомненной ставкой НДС.

Подсказка: Важно сначала сохранить параметр, затем построить вычисление.

Ответ:

- Написать функцию `outer(rate)`
- Создать функцию `inner(price)` возвращающую `price*rate`
- Вернуть `inner`
- Вызвать `outer(1.2)(100)`

Шаг 14:

Миссия 14

Упорядочьте действия для создания функции, которая возвращает другую функцию-валидатор пароля.

Подсказка: Сначала задаём условия, потом возвращаем проверку.

Ответ:

- Определить функцию `create_validator(min_len)`
- Внутри объявить функцию `validate(pw)`
- Функция `validate` возвращает `True` если длина $\geq$ `min_len`
- Вернуть `validate` и протестировать

Шаг 15:

Миссия 15

Упорядочьте шаги при написании функции, создающей разные приветствия в зависимости от языка.

Подсказка: Сначала описываем внешнюю функцию с языком, затем внутреннюю с текстом приветствия.

Ответ:

- Определить функцию `greeter(lang)`
- Создать внутреннюю `greet(name)`
- Внутри `greet` использовать `lang`
- Вернуть `greet` и вызвать её

Шаг 16:

Миссия 16

Расставьте шаги написания функции, создающей функции фильтрации списка по условию.

Подсказка: Внешняя функция хранит условие, внутренняя применяет его.

Ответ:

- Написать функцию `make_filter(cond)`
- Внутри определить функцию `filter_list(lst)`
- `filter_list` применяет `cond` к каждому элементу
- Вернуть `filter_list` и протестировать

Шаг 17:

Миссия 17

Расположите шаги для реализации функции, которая создаёт функции-декораторы.

Подсказка: Вспомните, что декоратор сам возвращает функцию.

Ответ:

- Написать функцию `make_decorator()`
- Внутри определить функцию `decorator(func)`
- `decorator` возвращает функцию `wrapper`
- Вернуть `decorator` и использовать

Урок 5 - Лабиринт динамические атрибуты классов

Шаг 01:

Миссия 1

Программист Костя добавил новый атрибут в класс динамически: `obj.new_attr = 42` . Что произойдёт?

Подсказка: В Python объекты могут хранить новые поля прямо во время исполнения.

Ответ:

- **У объекта появится новое свойство `new_attr` со значением 42**
- Будет ошибка компиляции
- Будет ошибка времени выполнения
- Python проигнорирует эту строку

Шаг 02:

Миссия 2

Маша экспериментирует с классами: `setattr(obj, 'x', 99)` . Чем это отличается от `obj.x = 99` ?

Подсказка: Оба варианта работают одинаково, но один из них удобен при программировании с переменными именами.

Ответ:

- **Это то же самое, только в функции можно передать имя атрибута как строку**
- `setattr` работает только с методами
- `obj.x = 99` быстрее в 100 раз
- `setattr` добавляет переменную в глобальную область

Шаг 03:

Миссия 3

Артём создаёт класс для игры. Он хочет, чтобы при обращении к атрибуту, которого нет, программа не падала. Что ему нужно переопределить?

Подсказка: Один из магических методов позволяет перехватывать доступ к отсутствующим атрибутам.

Ответ:

- **Метод `**getattr**`**
- Метод `**setattr**`
- Метод `**delattr**`
- Метод `**dir**`

Шаг 04:

Миссия 4

Настя пишет систему логирования. Она хочет, чтобы при каждом изменении атрибута объекта автоматически печаталось сообщение. Какой метод нужно переопределить?

Подсказка: Этот метод срабатывает каждый раз при установке значения.

Ответ:

- **`**setattr**`**
- **`**getattr**`**
- **`**dict**`**
- **`**repr**`**

Шаг 05:

Миссия 5

Программист Илья решил динамически удалить атрибут у объекта. Какой функцией он воспользуется?

Подсказка: Это противоположность `setattr`.

Ответ:

- **`delattr`**
- `removeattr`
- `dropattr`
- `forgetattr`

Шаг 06:

Миссия 6

 Сопоставьте функции работы с атрибутами с их назначением.

Подсказка: Подумайте: какие действия связаны с чтением, записью и удалением?

Ответ:

- **`getattr`** — Получение значения атрибута
- **`setattr`** — Установка нового значения атрибута
- **`delattr`** — Удаление атрибута объекта
- **`hasattr`** — Проверка наличия атрибута

Шаг 07:

Миссия 7

Сопоставьте магические методы с ситуациями, когда они вызываются.

Подсказка: Представьте: вы читаете, записываете или удаляете свойства объекта.

Ответ:

- **`**getattr**`** — Вызывается при обращении к несуществующему атрибуту
- **`**setattr**`** — Вызывается при присвоении значения атрибуту
- **`**delattr**`** — Вызывается при удалении атрибута
- **`**dir**`** — Вызывается при вызове `dir(obj)`

Шаг 08:

Миссия 8

Сопоставьте код с его результатом.

Подсказка: Вспомните, как работает `**getattr**` и `**setattr**`.

Ответ:

- **`obj.x = 10`** — Вызовется `**setattr**`

- **print(obj.y) # y не существует** — Вызовется ****getattr****
- **del obj.z** — Вызовется ****delattr****
- **dir(obj)** — Вызовется ****dir****

Шаг 09:

Миссия 9

Свяжите атрибуты объекта с ожидаемым результатом.

Подсказка: Попробуйте мысленно выполнить код и посмотреть, что останется в ****dict****.

Ответ:

- **setattr(obj, 'a', 1)** — У obj появится атрибут a = 1
- **getattr(obj, 'a')** — Вернёт 1
- **delattr(obj, 'a')** — Атрибут a удалится
- **hasattr(obj, 'a')** — Проверка существования атрибута

Шаг 10:

Миссия 10

 Определите, какой механизм лучше всего подходит под описание.

Подсказка: Одни методы позволяют перехватывать обращение, другие управляют отображением атрибутов.

Ответ:

- ****getattr**** — Подходит для задания значения по умолчанию
- ****dir**** — Определяет список видимых атрибутов
- ****setattr**** — Можно добавить проверку перед присвоением
- ****delattr**** — Позволяет контролировать удаление

Шаг 11:

Миссия 11

 Сопоставьте действия программиста с методами, которые для этого подойдут.

Подсказка: Вспомните: динамика = контроль атрибутов в реальном времени.

Ответ:

- **Добавить динамический атрибут через строковое имя** — setattr
- **Проверить наличие свойства у объекта** — hasattr
- **Удалить поле объекта при очистке** — delattr
- **Вернуть список всех атрибутов для отладки** — dir / ****dir****

Шаг 12:

Миссия 12

Расположите шаги динамического добавления и использования атрибута.

Подсказка: Сначала создаём, потом используем.

Ответ:

- Создать объект класса
- Добавить атрибут через setattr
- Использовать новый атрибут в коде
- Вывести его значение на экран

🧙 Шаг 13:

Миссия 13

Определите порядок действий программиста при работе с `**getattr**`.

Подсказка: Если атрибут есть — берём его, если нет — срабатывает магия.

Ответ:

- Запросить значение существующего атрибута
- Запросить несуществующий атрибут
- Сработает `**getattr**`
- Вернётся значение по умолчанию

🧙 Шаг 14:

Миссия 14

Упорядочите шаги логирования при изменении атрибутов.

Подсказка: Всё начинается с вызова присвоения.

Ответ:

- Вызов `obj.x = 5`
- Срабатывает `**setattr**`
- Записывается лог в консоль
- Атрибут сохраняется в `**dict**`

🧙 Шаг 15:

Миссия 15

Расставьте этапы удаления атрибута с контролем.

Подсказка: При `del obj.attr` — не сразу удаляется, а сначала вызывается метод.

Ответ:

- Вызов `del obj.attr`
- Срабатывает `**delattr**`
- Логирование операции
- Фактическое удаление из `**dict**`

🧙 Шаг 16:

Миссия 16

Определите правильный порядок работы с `hasattr` и `getattr`.

Подсказка: Проверка перед чтением избавит от ошибок.

Ответ:

- Вызвать `hasattr(obj, 'field')`
- **Если True** — вызвать `getattr(obj, 'field')`
- Сохранить значение в переменную
- Использовать в программе

🧙 Шаг 17:

Миссия 17

Расположите шаги использования `**dir**` для отладки.

Подсказка: Представьте, что вы ищете скрытые атрибуты.

Ответ:

- Вызвать `dir(obj)`
- Сработает `**dir**`
- Вернётся список атрибутов
- Программист анализирует результат

Модуль 2 - Древние свитки AST

Урок 1 - Введение в абстрактное синтаксическое дерево

Шаг 01:

Миссия 1

Программист Саша открыл для себя модуль `ast` и задался вопросом: что именно хранит абстрактное синтаксическое дерево (AST)?

Подсказка: AST — это не исходный код и не байт-код, а что-то между ними.

Ответ:

- **Структурированное представление программы в виде дерева узлов**
- Скомпилированный байт-код Python
- Только список функций и переменных
- Комментарии и документацию к коду

Шаг 02:

Миссия 2

Маша решила распарсить строку Python-кода в AST. Какой функцией из модуля `ast` она воспользуется?

Подсказка: Эта функция принимает строку кода и возвращает дерево.

Ответ:

- **`ast.parse()`**
- `ast.dump()`
- `compile()`
- `eval()`

Шаг 03:

Миссия 3

Петя хочет изучить структуру AST и вывести её в текстовом виде. Какой функцией это можно сделать?

Подсказка: Эта функция превращает дерево в строку.

Ответ:

- **`ast.dump()`**
- `ast.walk()`
- `repr()`
- `print()`

Шаг 04:

Миссия 4

Андрей заметил, что при парсинге кода в AST комментарии исчезают. Почему?

Подсказка: AST хранит только структуру выполнения программы.

Ответ:

- **Потому что комментарии не участвуют в выполнении кода**
- Потому что Python считает их ошибкой
- Комментарии заменяются специальными узлами AST
- Они сохраняются, но скрыты по умолчанию

Шаг 05:

Миссия 5

Ирина исследует AST для выражения $2 + 3$. Какие узлы она там обнаружит?

Подсказка: В AST всё разбивается на операцию и её операнды.

Ответ:

- **BinOp, Constant, Constant**
- Expression, Int, Int
- BinaryTree, Node, Leaf
- OpNode, NumNode, NumNode

Шаг 06:

Миссия 6

Вася хочет модифицировать AST и потом снова получить исполняемый код. Какой функцией он воспользуется?

Подсказка: Эта функция превращает AST обратно в объект кода Python.

Ответ:

- **compile()**
- ast.rebuild()
- exec()
- parse()

Шаг 07:

Миссия 7

Программист Лена использовала `ast.walk()`. Что делает эта функция?

Подсказка: Она обходит дерево, как будто гуляет по всем узлам.

Ответ:

- **Проходит по всем узлам дерева**
- Удаляет все узлы
- Превращает дерево в строку
- Заменяет узлы на новые

Шаг 08:

Миссия 8

Сопоставьте ключевые функции модуля ast с их назначением.

Подсказка: Вспомните: одна функция парсит код, другая печатает дерево, а третья обходит его.

Ответ:

- **ast.parse()** — Создаёт AST из строки кода
- **ast.dump()** — Возвращает строковое представление AST
- **ast.walk()** — Обходит все узлы дерева

🔮 Шаг 09:

Миссия 9

Кирилл изучает узлы AST. Сопоставьте названия узлов с их описанием.

Подсказка: Одни узлы отвечают за числа, другие за операции.

Ответ:

- **Constant** — Хранит значение числа или строки
- **BinOp** — Представляет бинарную операцию (например, сложение)
- **Name** — Ссылка на переменную по имени

🔮 Шаг 10:

Миссия 10

Сопоставьте понятие с тем, что оно обозначает в мире AST.

Подсказка: Вспомните, что дерево — это структура, а узлы бывают разных типов.

Ответ:

- **AST** — Абстрактное синтаксическое дерево
- **Node** — Отдельный элемент дерева
- **Field** — Поле узла, хранящее данные

🔮 Шаг 11:

Миссия 11

Сопоставьте действия программиста с правильной функцией из модуля ast .

Подсказка: Если нужно напечатать дерево — используем dump, если пройти по всем элементам — walk.

Ответ:

- **Хотим создать AST из текста кода** — ast.parse()
- **Хотим вывести структуру дерева** — ast.dump()
- **Хотим пройти по всем узлам** — ast.walk()

🔮 Шаг 12:

Миссия 12

Расположите шаги работы с AST в правильном порядке.

Подсказка: Сначала мы создаём дерево, потом исследуем его, и только потом преобразуем.

Ответ:

- Написать исходный код
- Вызвать ast.parse()
- Использовать ast.dump() для просмотра структуры
- Модифицировать AST и скомпилировать обратно

Шаг 13:

Миссия 13

Расставьте шаги от кода до выполнения через AST.

Подсказка: Помните: текст → дерево → объект кода → запуск.

Ответ:

- Текст программы
- AST дерево
- Компиляция в объект кода
- Выполнение `exec()`

Шаг 14:

Миссия 14

Программист решил изменить Python-код через AST. Расположите шаги действий.

Подсказка: Сначала дерево надо получить, потом изменить и только после — скомпилировать.

Ответ:

- Вызвать `ast.parse()`
- Изменить узлы AST
- Скомпилировать дерево с помощью `compile()`
- Выполнить код через `exec()`

Шаг 15:

Миссия 15

Расставьте этапы работы команды разработчиков, которые используют AST для автоматической оптимизации кода.

Подсказка: Сначала код парсится, потом дерево анализируется и модифицируется, а в конце — компилируется и тестируется.

Ответ:

- Парсинг кода в AST
- Анализ узлов и поиск оптимизаций
- Модификация дерева
- Компиляция и тестирование

Урок 2 - `ast.parse` превращение кода в дерево

Шаг 01:

Миссия 1

Вы пишете инструмент для анализа кода. Что делает функция `ast.parse()` ?

Подсказка: Представьте, что Python читает ваш код и превращает его в дерево объектов.

Ответ:

- **Преобразует строку Python-кода в абстрактное синтаксическое дерево (AST)**
- Выполняет Python-код сразу
- Преобразует Python-код в байткод
- Преобразует Python-код в JSON-объект

Шаг 02:

Миссия 2

Что произойдёт, если в `ast.parse()` передать строку с синтаксической ошибкой?

Подсказка: Вспомните, что Python всегда проверяет корректность кода перед анализом.

Ответ:

- Будет выброшено исключение **`SyntaxError`**
- Будет возвращён пустой AST
- Код будет исправлен автоматически
- Возвратится `None`

Шаг 03:

Миссия 3

Какой параметр можно передать в `ast.parse()` , чтобы указать режим парсинга (например, `'eval'`, `'exec'`)?

Подсказка: Этот параметр управляет тем, ожидается ли выражение или целый блок кода.

Ответ:

- **`mode`**
- `context`
- `syntax`
- `tree`

Шаг 04:

Миссия 4

Когда используется режим `'eval'` в `ast.parse()` ?

Подсказка: Подумайте, что может быть вычислено сразу как выражение.

Ответ:

- **Когда парсится одно выражение (например, `2+2`)**
- Когда нужно распарсить функцию
- Когда код содержит цикл
- Всегда по умолчанию

Шаг 05:

Миссия 5

Сопоставьте элементы AST с их описанием.

Подсказка: Вспомните: `Module` — это весь файл, `Assign` — присваивание, `Constant` — число или строка.

Ответ:

- **`Module`** — Корневой узел AST, представляющий весь файл
- **`Assign`** — Узел, отвечающий за операцию присваивания
- **`Constant`** — Узел, содержащий литерал (число, строку и т.д.)
- **`BinOp`** — Узел, представляющий бинарную операцию

Шаг 06:

Миссия 6

Сопоставьте код с тем, какой тип узла он создаст.

Подсказка: Смотрите на структуру: где присваивание, где вызов функции, где число.

Ответ:

- **x = 5** — Assign
- **42** — Constant
- **print('hi')** — Call
- **a + b** — BinOp

🔗 Шаг 07:

Миссия 7

Сопоставьте имена классов из модуля ast с их ролью.

Подсказка: Обратите внимание: Expr — это выражение, Name — переменная, FunctionDef — функция.

Ответ:

- **Expr** — Обычное выражение внутри тела модуля
- **Name** — Узел для переменной
- **FunctionDef** — Определение функции
- **Return** — Оператор возврата

🔗 Шаг 08:

Миссия 8

Сопоставьте код с корневым типом узла AST, который будет получен через ast.parse().

Подсказка: Всегда сначала есть Module, но внутри нас интересует тип первого узла.

Ответ:

- **def foo(): pass** — FunctionDef
- **class A: pass** — ClassDef
- **x = 10** — Assign
- **y** — Name

🔗 Шаг 09:

Миссия 9

Сопоставьте функции модуля ast с их назначением.

Подсказка: parse превращает строку в AST, dump показывает дерево, walk проходит по узлам.

Ответ:

- **ast.parse** — Преобразует строку кода в AST
- **ast.dump** — Возвращает строковое представление дерева
- **ast.walk** — Проходит по всем узлам дерева
- **ast.fix_missing_locations** — Заполняет отсутствующие позиции в AST

🔗 Шаг 10:

Миссия 10

Расположите шаги превращения строки кода в AST и его анализа в правильном порядке.

Подсказка: Сначала вы получаете строку, затем вызываете parse, потом можете посмотреть дерево через dump, и наконец пройти по узлам.

Ответ:

- Написать строку кода
- Вызвать `ast.parse()`
- Посмотреть дерево через `ast.dump()`
- Пройтись по дереву через `ast.walk()`

🔮 **Шаг 11:**

Миссия 11

Расположите шаги работы с AST при анализе функции.

Подсказка: Сначала распарсить код, потом найти `FunctionDef`, затем внутри `Return`, и наконец — выражение.

Ответ:

- `ast.parse()`
- `FunctionDef`
- `Return`
- `BinOp`

🔮 **Шаг 12:**

Миссия 12

Что делает функция `ast.parse()` в Python?

Подсказка: Подумайте, что является первым шагом при работе с AST.

Ответ:

- **Преобразует код Python в дерево AST**
- Компилирует код в байт-код
- Выполняет код Python
- Сохраняет код в файл

🔮 **Шаг 13:**

Миссия 13

Какой узел AST отвечает за присваивание переменной значения?

Подсказка: Узел хранит действие `'x = 42'`.

Ответ:

- **Assign**
- `Name`
- `Module`
- `Expr`

🔮 **Шаг 14:**

Миссия 14

Какой узел AST используется для хранения константного значения, например числа 42?

Подсказка: Узел хранит значение, а не имя переменной.

Ответ:

- **Constant**
- `Name`

- Assign
- Call

🔮 Шаг 15:

Миссия 15

Что возвращает `ast.dump(node)`?

Подсказка: Подумайте о текстовом представлении структуры AST.

Ответ:

- **Строковое представление AST**
- Исполняемый код Python
- Список переменных
- Список функций

🔮 Шаг 16:

Миссия 16

Какой модуль поможет преобразовать AST обратно в исполняемый код?

Подсказка: Существует стандартный и сторонний вариант.

Ответ:

- **astor или ast.unparse**
- json
- pickle
- sys

🔮 Шаг 17:

Миссия 17

Сопоставьте узлы AST с их ролью.

Подсказка: Узлы имеют разные функции: хранение имени, значения или присваивания.

Ответ:

- **Module** — Корневой узел AST
- **Assign** — Присваивание переменной
- **Name** — Имя переменной
- **Constant** — Хранение значения

🔮 Шаг 18:

Миссия 18

Сопоставьте узлы AST с примерами кода.

Подсказка: Подумайте, какая конструкция соответствует какому узлу.

Ответ:

- **FunctionDef** — `def greet(): pass`
- **Call** — `print('Hello')`
- **BinOp** — `x + y`
- **If** — `if x > 0: pass`

🔗 Шаг 19:

Миссия 19

Сопоставьте методы AST с их действием.

Подсказка: Подумайте о функции парсинга, изменения и генерации кода.

Ответ:

- **ast.parse()** — Создает AST из строки кода
- **ast.dump()** — Возвращает строковое представление дерева
- **astor.to_source()** — Преобразует AST обратно в код
- **node.value = Constant(42)** — Изменяет значение узла

📖 Урок 3 - Обход дерева поиск узлов функций

🔗 Шаг 01:

Миссия 1

Программист Саша изучает дочерние узлы. Сопоставьте тип узла с его примером.

Подсказка: Подумайте, как выражения в коде выглядят в AST.

Ответ:

- **Assign** — Присваивание переменной
- **Expr** — Выражение, например вызов функции
- **BinOp** — Бинарная операция a+b
- **If** — Условная конструкция

🔗 Шаг 02:

Миссия 2

Пётр хочет проверить свойства дерева. Сопоставьте свойство с тем, что оно описывает.

Подсказка: У каждого узла есть важные атрибуты — тело, имя, аргументы.

Ответ:

- **body** — Список дочерних узлов
- **name** — Имя функции или переменной
- **args** — Параметры функции
- **lineno** — Номер строки в исходном коде

🔗 Шаг 03:

Миссия 3

Сопоставьте узлы AST с их назначением в коде.

Подсказка: Представьте, что вы рисуете дерево программы: каждая часть кода соответствует узлу.

Ответ:

- **Return** — Возврат значения из функции
- **While** — Цикл с условием
- **Import** — Подключение модуля
- **ClassDef** — Определение класса

Шаг 04:

Миссия 4

Расставьте шаги анализа AST в правильном порядке.

Подсказка: Вспомните: сначала нужно получить код, затем распарсить, потом исследовать дерево.

Ответ:

- Получение строки кода
- Вызов `ast.parse`
- Обход дерева
- Извлечение информации об узлах

Шаг 05:

Миссия 5

Программист пишет анализатор. Расположите этапы разработки:

Подсказка: Нужно сначала понять задачу, потом получить дерево, затем построить обработчик.

Ответ:

- Формулировка цели анализа
- Парсинг кода с `ast.parse`
- Обход дерева
- Реализация логики проверки

Шаг 06:

Миссия 6

Никита автоматизирует проверку кода. Расставьте шаги:

Подсказка: Сначала нужен код, потом дерево, затем фильтрация и вывод.

Ответ:

- Написание тестового кода
- Применение `ast.parse`
- Проход по дереву
- Вывод найденных элементов

Шаг 07:

Миссия 7

Сергей хочет построить инструмент для подсчета функций. Расставьте шаги.

Подсказка: Вспомните логику: получить дерево → найти нужные узлы → посчитать.

Ответ:

- Вызов `ast.parse`
- Обход дерева
- Фильтрация `FunctionDef`
- Подсчет количества

Шаг 08:

Миссия 8

Александр отлаживает AST-анализатор. Каковы шаги?

Подсказка: Сначала код, затем дерево, проверка структуры, потом исправления.

Ответ:

- Написание кода
- Парсинг AST
- Проверка узлов
- Исправление ошибок

Шаг 09:

Миссия 9

Финальный квест: расположите шаги полного цикла работы с AST.

Подсказка: Подумайте как инженер: от кода до анализа и результата.

Ответ:

- Написание Python-кода
- Парсинг через `ast.parse`
- Анализ дерева
- Вывод результатов

Шаг 10:

Миссия 10

Что делает `ast.walk(node)` в Python?

Подсказка: Подумайте о способе прохода по всему дереву AST.

Ответ:

- **Итеративно проходит по всем узлам AST**
- Создает AST из текста кода
- Выполняет найденные функции
- Удаляет узлы AST

Шаг 11:

Миссия 11

Какой узел AST соответствует определению функции?

Подсказка: Подумайте о ключевом слове `def` в коде.

Ответ:

- **FunctionDef**
- Call
- Expr
- Assign

Шаг 12:

Миссия 12

Какой метод удобен для поиска всех функций в дереве AST?

Подсказка: Подумайте о итерации по всем узлам и фильтрации по типу узла.

Ответ:

- **`[n for n in ast.walk(tree) if isinstance(n, ast.FunctionDef)]`**
- `tree.findall('FunctionDef')`

- `ast.dump(tree)`
- `eval(tree)`

🔗 Шаг 13:

Миссия 13

Что хранит узел `FunctionDef` помимо имени функции?

Подсказка: Вспомните стандартные атрибуты узла функции в AST.

Ответ:

- **Аргументы, тело функции, декораторы**
- Только имя функции
- Список всех вызовов функций
- Содержимое файлов модуля

🔗 Шаг 14:

Миссия 14

Сопоставьте методы обхода AST с их действием.

Подсказка: Подумайте, какой метод возвращает все узлы, а какой только один уровень.

Ответ:

- **`ast.walk(node)`** — Итеративно проходит по всем узлам дерева
- **`ast.iter_child_nodes(node)`** — Возвращает только прямых потомков узла
- **`ast.dump(node)`** — Возвращает строковое представление узла
- **`isinstance(n, ast.FunctionDef)`** — Проверяет, является ли узел определением функции

🔗 Шаг 15:

Миссия 15

Сопоставьте узлы AST с их кодовыми примерами.

Подсказка: Определение функции, вызов функции, тело функции — разные узлы.

Ответ:

- **`FunctionDef`** — `def greet(): pass`
- **`Call`** — `greet()`
- **`arguments`** — `def func(x, y): pass`
- **`Expr`** — `print('Hello')`

🔗 Шаг 16:

Миссия 16

Сопоставьте атрибуты `FunctionDef` с их описанием.

Подсказка: Имя функции, аргументы, тело и декораторы — все это атрибуты.

Ответ:

- **`name`** — Имя функции
- **`args`** — Аргументы функции
- **`body`** — Список инструкций внутри функции
- **`decorator_list`** — Список декораторов функции

Шаг 17:

Миссия 17

Расположите шаги поиска всех функций в AST в правильном порядке:

1. Создать AST с помощью `ast.parse()`
2. Итеративно пройти по всем узлам с помощью `ast.walk()`
3. Проверить каждый узел на `FunctionDef`
4. Собрать найденные функции в список

Подсказка: Сначала создаем дерево, потом обходим, проверяем и собираем.

Ответ:

- Создать AST с помощью `ast.parse()`
- Итеративно пройти по всем узлам с помощью `ast.walk()`
- Проверить каждый узел на `FunctionDef`
- Собрать найденные функции в список

Урок 4 - Модификация AST и генерация нового кода

Шаг 01:

Миссия 1

Анна хочет изменить код на лету с помощью AST. Какой модуль в Python отвечает за генерацию AST из строки кода?

Подсказка: Это стандартный модуль для анализа синтаксического дерева.

Ответ:

- **ast**
- json
- sys
- re

Шаг 02:

Миссия 2

Вася хочет изменить присваивание `x = 1` на `x = 42` через AST. Какой узел ему нужно изменить?

Подсказка: Подумайте, какой узел содержит значение переменной.

Ответ:

- **Constant**
- Name
- Assign
- Module

Шаг 03:

Миссия 3

Мария использует `ast.dump()` для узла дерева. Что она получит?

Подсказка: Подумайте: это не сам код, а его текстовое представление структуры дерева.

Ответ:

- **Строковое представление AST**
- Исполняемый код Python

- Список всех переменных
- Список функций

🔗 Шаг 04:

Миссия 4

Дима хочет преобразовать AST обратно в код. Какой модуль ему поможет?

Подсказка: Это модуль, который позволяет преобразовывать дерево обратно в исполняемый код.

Ответ:

- **astor** или **ast.unparse**
- json
- pickle
- re

🔗 Шаг 05:

Миссия 5

Андрей хочет добавить вызов функции `print('Hello')` в AST. Какой узел нужно создать для самого вызова?

Подсказка: Вспомните, что вызов функции — это отдельный тип узла.

Ответ:

- **Call**
- Expr
- FunctionDef
- Module

🔗 Шаг 06:

Миссия 6

Сопоставьте узлы AST с их назначением при модификации кода.

Подсказка: Каждый узел выполняет свою роль: хранит значение, операцию или конструкцию.

Ответ:

- **Module** — Корень дерева AST
- **Assign** — Присваивание значения переменной
- **Name** — Имя переменной
- **Constant** — Константное значение

🔗 Шаг 07:

Миссия 7

Сопоставьте узлы AST с примерами кода.

Подсказка: Вспомните: каждый узел отражает синтаксическую конструкцию Python.

Ответ:

- **FunctionDef** — `def greet(): pass`
- **Call** — `print('Hello')`
- **BinOp** — `x + y`
- **If** — `if x > 0: pass`

🔗 Шаг 08:

Миссия 8

Сопоставьте методы работы с AST и их результат.

Подсказка: Думайте о преобразовании дерева и его изменении.

Ответ:

- **ast.parse()** — Создает дерево из строки кода
- **ast.dump()** — Возвращает текстовое представление дерева
- **astor.to_source()** — Преобразует дерево обратно в код
- **node.value = Constant(42)** — Изменяет значение в узле

🔗 Шаг 09:

Миссия 9

Сопоставьте узлы AST с их использованием в генерации нового кода.

Подсказка: Подумайте, где меняются значения и где добавляются новые выражения.

Ответ:

- **Expr** — Обертка для выражения, которое будет вычислено
- **Call** — Вызов функции
- **Assign** — Присваивание переменной
- **Constant** — Значение, которое можно изменить

🔗 Шаг 10:

Миссия 10

Сопоставьте элементы AST с их ролью при генерации кода.

Подсказка: Узлы определяют структуру и поведение кода.

Ответ:

- **Module** — Корневой узел модуля
- **FunctionDef** — Определение функции
- **Return** — Возврат значения функции
- **For** — Цикл по элементам

🔗 Шаг 11:

Миссия 11

Что делает функция ast.parse() в Python?

Подсказка: Подумайте, как превращается текст программы в структуру для анализа.

Ответ:

- **Превращает код Python в дерево AST**
- Выполняет Python код
- Создает новый модуль
- Сохраняет код в файл

🔗 Шаг 12:

Миссия 12

Как можно добавить новую функцию в существующее AST дерево?

Подсказка: Думайте о добавлении нового узла типа FunctionDef в тело модуля.

Ответ:

- **Создать ast.FunctionDef и вставить его в модуль.body**
- Использовать eval для вставки функции
- Просто добавить строку кода в текст программы
- Присвоить функцию переменной tree

🔮 **Шаг 13:**

Миссия 13

Что делает ast.dump(node) в Python?

Подсказка: Подумайте о текстовом представлении AST.

Ответ:

- **Возвращает строковое представление узла AST**
- Выполняет код узла
- Удаляет узел из дерева
- Создает новый узел FunctionDef

🔮 **Шаг 14:**

Миссия 14

Какой атрибут FunctionDef хранит тело функции?

Подсказка: Подумайте, где хранится список инструкций функции.

Ответ:

- **body**
- name
- args
- decorator_list

🔮 **Шаг 15:**

Миссия 15

Сопоставьте методы AST с их функцией.

Подсказка: Думайте, какой метод создает дерево, а какой показывает его структуру.

Ответ:

- **ast.parse()** — Создает AST из текста кода
- **ast.dump()** — Возвращает текстовое представление узла
- **ast.fix_missing_locations()** — Добавляет недостающие позиции строк для компиляции
- **compile()** — Компилирует AST в исполняемый объект

🔮 **Шаг 16:**

Миссия 16

Сопоставьте атрибуты FunctionDef с их описанием.

Подсказка: Имя, аргументы, тело и декораторы — всё разные атрибуты.

Ответ:

- **name** — Имя функции
- **args** — Список аргументов функции

- **body** — Список инструкций внутри функции
- **decorator_list** — Список декораторов функции

🔗 Шаг 17:

Миссия 17

Сопоставьте типы узлов AST с их примером кода.

Подсказка: Определение функции, вызов функции, выражение — разные узлы.

Ответ:

- **FunctionDef** — `def greet(): pass`
- **Call** — `greet()`
- **Expr** — `print('Hello')`
- **Assign** — `x = 42`

🔗 Шаг 18:

Миссия 18

Расположите шаги добавления новой функции в AST в правильном порядке:

1. Создать `ast.FunctionDef` с нужным именем и телом
2. Добавить узел в `module.body`
3. Вызвать `ast.fix_missing_locations()` для корректировки позиций
4. Скомпилировать AST с помощью `compile()`

Подсказка: Сначала создаем узел, потом добавляем, исправляем позиции и компилируем.

Ответ:

- Создать `ast.FunctionDef` с нужным именем и телом
- Добавить узел в `module.body`
- Вызвать `ast.fix_missing_locations()` для корректировки позиций
- Скомпилировать AST с помощью `compile()`

📖 Урок 5 - Практика трансформация Python-кода

🔗 Шаг 01:

Миссия 1

Что нужно сделать, чтобы заменить все вызовы функции `foo` на `bar` в коде Python через AST?

Подсказка: Подумайте о обходе дерева и замене узлов `Call`.

Ответ:

- **Обойти AST, найти узлы `Call` с `func=foo` и заменить на `bar`**
- Просто заменить текст в исходном коде
- Удалить все узлы и написать код заново
- Использовать `eval` на старом коде

🔗 Шаг 02:

Миссия 2

Как можно вставить новую инструкцию `print('Hello')` в начало функции через AST?

Подсказка: Подумайте, как модифицировать список `body` у `FunctionDef`.

Ответ:

- Создать узел **Expr/Call** и вставить в функцию **body** на индекс **0**
- Просто написать `print` в тексте функции
- Использовать `exec` на строке `'print("Hello")'`
- Удалить все остальные узлы функции

🔗 Шаг 03:

Миссия 3

Что делает `ast.fix_missing_locations(node)`?

Подсказка: Подумайте о позиции строк и столбцов в узлах AST перед компиляцией.

Ответ:

- Заполняет отсутствующие атрибуты **lineno** и **col_offset**
- Удаляет лишние узлы
- Исполняет код узла
- Преобразует узлы в текст

🔗 Шаг 04:

Миссия 4

Как можно сгенерировать новый исполняемый код из AST?

Подсказка: Думайте о `compile()` и `exec()`.

Ответ:

- Использовать **`compile(tree, filename, 'exec')`** и **`exec()`**
- Просто вызвать `print(tree)`
- Вставить AST в текстовый файл без `compile`
- `eval(tree)`

🔗 Шаг 05:

Миссия 5

Сопоставьте методы AST с их назначением.

Подсказка: Думайте, что создает дерево, что исправляет позиции, а что компилирует.

Ответ:

- **`ast.parse()`** — Создает AST из текста кода
- **`ast.fix_missing_locations()`** — Добавляет пропущенные `lineno` и `col_offset`
- **`compile()`** — Компилирует AST в исполняемый объект
- **`exec()`** — Выполняет скомпилированный объект

🔗 Шаг 06:

Миссия 6

Сопоставьте атрибуты `FunctionDef` с их описанием.

Подсказка: Имя, аргументы, тело и декораторы — разные части функции.

Ответ:

- **`name`** — Имя функции
- **`args`** — Аргументы функции
- **`body`** — Список инструкций внутри функции

- **decorator_list** — Список декораторов

🔮 Шаг 07:

Миссия 7

Сопоставьте типы узлов AST с примером кода.

Подсказка: Разные узлы соответствуют разным конструкциям.

Ответ:

- **Assign** — `x = 10`
- **Expr** — `print(x)`
- **Call** — `func()`
- **FunctionDef** — `def foo(): pass`

🔮 Шаг 08:

Миссия 8

Расположите шаги трансформации Python-кода через AST:

1. Разбор исходного кода с помощью `ast.parse()`
2. Модификация узлов AST (например, `FunctionDef`, `Call`)
3. Исправление пропущенных атрибутов с `ast.fix_missing_locations()`
4. Компиляция с помощью `compile()` и выполнение `exec()`

Подсказка: Сначала создается дерево, потом модифицируется, исправляются позиции и выполняется.

Ответ:

- Разбор исходного кода с помощью `ast.parse()`
- Модификация узлов AST (например, `FunctionDef`, `Call`)
- Исправление пропущенных атрибутов с `ast.fix_missing_locations()`
- Компиляция с помощью `compile()` и выполнение `exec()`

🔮 Шаг 09:

Миссия 9

Сопоставьте типы выражений AST с их примером использования.

Подсказка: Разные выражения создают разные узлы.

Ответ:

- **BinOp** — `x + y`
- **UnaryOp** — `-x`
- **BoolOp** — `a and b`
- **Compare** — `x > 10`

🔮 Шаг 10:

Миссия 10

Сопоставьте методы `NodeTransformer` с их функцией.

Подсказка: Подумайте, что изменяет дерево, а что просто посещает узлы.

Ответ:

- **visit()** — Обходит дерево и возвращает узел без изменений
- **generic_visit()** — Обходит все дочерние узлы узла

- **NodeTransformer.visit()** — Может заменить узел на новый

🔗 Шаг 11:

Миссия 11

Сопоставьте типы литералов с примерами в Python.

Подсказка: Числа, строки и логические значения имеют свои узлы.

Ответ:

- **Num** — 42
- **Str** — 'hello'
- **Constant** — True
- **Bytes** — b'abc'

🔗 Шаг 12:

Миссия 12

Расположите шаги замены всех print() на логирование через AST:

1. Разбор кода с ast.parse()
2. Создание NodeTransformer для замены Call с func=print на logging.info
3. Вызов visit() на дереве
4. Исправление пропущенных lineno и col_offset с ast.fix_missing_locations()
5. Компиляция и выполнение с помощью compile() и exec()

Подсказка: Сначала создаем дерево, потом трансформируем, исправляем позиции, компилируем и выполняем.

Ответ:

- Разбор кода с ast.parse()
- Создание NodeTransformer для замены Call с func=print на logging.info
- Вызов visit() на дереве
- Исправление пропущенных lineno и col_offset с ast.fix_missing_locations()
- Компиляция и выполнение с помощью compile() и exec()

🔗 Шаг 13:

Миссия 13

Расположите этапы добавления нового аргумента в функцию через AST:

1. Разбор кода с ast.parse()
2. Найти FunctionDef узлы
3. Изменить args.args, добавив новый аргумент
4. Использовать ast.fix_missing_locations()
5. Компиляция и выполнение с compile() и exec()

Подсказка: Изменяем структуру функции перед выполнением.

Ответ:

- Разбор кода с ast.parse()
- Найти FunctionDef узлы
- Изменить args.args, добавив новый аргумент
- Использовать ast.fix_missing_locations()
- Компиляция и выполнение с compile() и exec()

Шаг 14:

Миссия 14

Расположите шаги для удаления всех pass в теле функций через AST:

1. Разбор кода с `ast.parse()`
2. Создать `NodeTransformer` для удаления `Pass`
3. Вызвать `visit()` на дереве
4. Исправить позиции с `ast.fix_missing_locations()`
5. Компиляция и выполнение с `compile()` и `exec()`

Подсказка: Удаляем ненужные инструкции перед компиляцией.

Ответ:

- Разбор кода с `ast.parse()`
- Создать `NodeTransformer` для удаления `Pass`
- Вызвать `visit()` на дереве
- Исправить позиции с `ast.fix_missing_locations()`
- Компиляция и выполнение с `compile()` и `exec()`

Шаг 15:

Миссия 15

Расположите этапы добавления логирования аргументов функций через AST:

1. Разбор кода с `ast.parse()`
2. Найти `FunctionDef` узлы
3. Создать `Expr/Call` для логирования аргументов
4. Вставить новые узлы в `body` функции
5. Исправить `lineno` и `col_offset` с `ast.fix_missing_locations()`
6. Компиляция и выполнение с `compile()` и `exec()`

Подсказка: Логирование добавляется перед основной логикой функции.

Ответ:

- Разбор кода с `ast.parse()`
- Найти `FunctionDef` узлы
- Создать `Expr/Call` для логирования аргументов
- Вставить новые узлы в `body` функции
- Исправить `lineno` и `col_offset` с `ast.fix_missing_locations()`
- Компиляция и выполнение с `compile()` и `exec()`

Шаг 16:

Миссия 16

Расположите шаги полной трансформации кода: добавление функции, изменение вызовов, логирование через AST:

1. Разбор исходного кода с `ast.parse()`
2. Добавление нового `FunctionDef` узла
3. Замена всех вызовов старой функции на новую через `NodeTransformer`
4. Добавление логирования аргументов
5. Исправление пропущенных `lineno` и `col_offset` с `ast.fix_missing_locations()`
6. Компиляция и выполнение с `compile()` и `exec()`

Подсказка: Сначала строим дерево, потом трансформируем, добавляем логирование и выполняем.

Ответ:

- Разбор исходного кода с `ast.parse()`
- Добавление нового `FunctionDef` узла
- Замена всех вызовов старой функции на новую через `NodeTransformer`
- Добавление логирования аргументов
- Исправление пропущенных `lineno` и `col_offset` с `ast.fix_missing_locations()`
- Компиляция и выполнение с `compile()` и `exec()`

Модуль 3 - Алхимия code generation

Урок 1 - compile превращение AST в байткод

Шаг 01:

Миссия 1

Что делает функция `compile()` в Python?

Подсказка: Она превращает код в объект, который может быть выполнен интерпретатором.

Ответ:

- **Преобразует строку кода в объект байткода**
- Запускает код напрямую
- Создаёт новый файл `.py`
- Компилирует Python в C

Шаг 02:

Миссия 2

Какой режим компиляции используется для выполнения кода как скрипта?

Подсказка: Подумайте, что чаще всего запускается напрямую, а не как модуль.

Ответ:

- `'exec'`
- `'eval'`
- `'single'`

Шаг 03:

Миссия 3

Что необходимо передать функции `compile()` вместе с исходным кодом?

Подсказка: Подумайте, что Python должен знать, чтобы правильно выполнять код.

Ответ:

- **Имя файла и режим компиляции**
- Только имя функции
- Версию Python
- Список всех переменных

Шаг 04:

Миссия 4

Какой объект возвращает `compile()`?

Подсказка: Подумайте, что может быть выполнено с помощью функции `exec()`.

Ответ:

- **Объект кода (code object)**
- Строку с байткодом
- Модуль Python
- Список инструкций Python

🔮 Шаг 05:

Миссия 5

Что будет, если передать некорректный синтаксис в `compile()`?

Подсказка: Подумайте, как Python реагирует на синтаксические ошибки.

Ответ:

- Python вызовет **SyntaxError**
- Функция вернёт `None`
- Код выполнится частично

🔮 Шаг 06:

Миссия 6

Сопоставьте объекты и их назначение при работе с `compile()`

Подсказка: Подумайте, что можно делать с объектами кода и как они используются.

Ответ:

- **'exec'** — Выполняет блок кода целиком
- **'eval'** — Выполняет выражение и возвращает результат
- **code object** — Представление скомпилированного кода
- **filename** — Используется для сообщений об ошибках

🔮 Шаг 07:

Миссия 7

Сопоставьте функции и их роль в метапрограммировании с `compile()`

Подсказка: Подумайте, какие функции позволяют выполнять скомпилированный код и как они влияют на выполнение.

Ответ:

- **exec()** — Выполняет code object без возврата результата
- **eval()** — Выполняет code object выражения и возвращает результат
- **globals()** — Позволяет определить глобальные переменные при выполнении

🔮 Шаг 08:

Миссия 8

Сопоставьте режимы `compile()` с их назначением

Подсказка: Каждый режим используется для определённого типа кода: выражение, модуль, или единичная инструкция.

Ответ:

- **'exec'** — Для выполнения целого блока кода

- **'eval'** — Для вычисления одного выражения
- **'single'** — Для выполнения одной инструкции

Шаг 09:

Миссия 9

Сопоставьте ошибки Python с ситуациями при compile()

Подсказка: Разные типы ошибок возникают при неправильном коде.

Ответ:

- **SyntaxError** — Когда код написан с нарушением синтаксиса
- **NameError** — Когда переменная или функция не определена
- **TypeError** — Когда тип данных не соответствует операции

Шаг 10:

Миссия 10

Сопоставьте атрибуты code object с их смыслом

Подсказка: Подумайте, какие данные хранятся в скомпилированном объекте.

Ответ:

- **co_code** — Список инструкций байткода
- **co_names** — Имена переменных и функций, используемые в коде
- **co_consts** — Константы, встречающиеся в коде
- **co_filename** — Имя файла исходного кода

Шаг 11:

Миссия 11

Расположите шаги выполнения скомпилированного кода:

1. Скомпилировать исходный код в code object
2. Определить глобальные и локальные контексты
3. Выполнить code object с помощью exec или eval
4. Обработать возможные ошибки выполнения

Подсказка: Подумайте, что сначала нужно подготовить, потом выполнить и только потом проверять результат.

Ответ:

- Скомпилировать исходный код в code object
- Определить глобальные и локальные контексты
- Выполнить code object с помощью exec или eval
- Обработать возможные ошибки выполнения

Шаг 12:

Миссия 12

Расположите шаги при динамической генерации кода:

1. Создать строку с кодом Python
2. Скомпилировать строку с compile()
3. Выполнить скомпилированный code object
4. Проверить результат выполнения

Подсказка: Сначала создаём код, затем превращаем в объект, потом выполняем и проверяем.

Ответ:

- Создать строку с кодом Python
- Скомпилировать строку с `compile()`
- Выполнить скомпилированный code object
- Проверить результат выполнения

 **Шаг 13:**

Миссия 13

Расположите шаги при отладке кода с `compile()`:

1. Обнаружить место ошибки в исходном коде
2. Исправить синтаксис или логические ошибки
3. Перекомпилировать код с `compile()`
4. Выполнить и проверить корректность

Подсказка: Ошибки исправляются до выполнения, после — проверяем результат.

Ответ:

- Обнаружить место ошибки в исходном коде
- Исправить синтаксис или логические ошибки
- Перекомпилировать код с `compile()`
- Выполнить и проверить корректность

 **Шаг 14:**

Миссия 14

Расположите действия для безопасного выполнения динамического кода:

1. Ограничить доступные переменные и функции
2. Скомпилировать код с `compile()`
3. Выполнить code object в изолированном пространстве
4. Обработать исключения и ошибки

Подсказка: Безопасность важна: сначала ограничиваем контекст, потом выполняем и проверяем.

Ответ:

- Ограничить доступные переменные и функции
- Скомпилировать код с `compile()`
- Выполнить code object в изолированном пространстве
- Обработать исключения и ошибки

 **Шаг 15:**

Миссия 15

Расположите шаги создания интерактивного кода через `compile()`:

1. Получить пользовательский ввод
2. Создать строку кода на основе ввода
3. Скомпилировать строку с `compile()`
4. Выполнить скомпилированный код

Подсказка: Ввод пользователя используется для генерации кода, потом выполняем его.

Ответ:

- Получить пользовательский ввод
- Создать строку кода на основе ввода
- Скомпилировать строку с `compile()`
- Выполнить скомпилированный код

 **Шаг 16:**

Миссия 16

Расположите шаги при создании модульного метапрограммирования с `compile()`:

1. Создать функции-генераторы кода
2. Скомпилировать каждый блок через `compile()`
3. Выполнить блоки кода в нужном порядке
4. Объединить результаты в основной модуль

Подсказка: Модули собираются из блоков: сначала генерация, потом компиляция, выполнение и объединение.

Ответ:

- Создать функции-генераторы кода
- Скомпилировать каждый блок через `compile()`
- Выполнить блоки кода в нужном порядке
- Объединить результаты в основной модуль

Урок 2 - Генерация функций на лету

 **Шаг 01:**

Миссия 1

Что позволяет сделать функция `type()` в Python при метапрограммировании?

Подсказка: Она может создавать новые классы на лету.

Ответ:

- **Создавать новые классы динамически**
- Компилировать байткод
- Запускать асинхронные функции
- Сериализовать объекты

 **Шаг 02:**

Миссия 2

Как можно создавать функции на лету в Python?

Подсказка: Подумайте, какие конструкции позволяют формировать код динамически.

Ответ:

- **Использовать `def` внутри функции или `exec` с `compile`**
- Создавать переменные глобально
- Писать статические классы

🔮 Шаг 03:

Миссия 3

Что возвращает `exec()` при выполнении сгенерированного кода функции?

Подсказка: Эта функция выполняет код, а не возвращает результат.

Ответ:

- **None, но функция создается в заданном пространстве имен**
- Результат выполнения функции
- Строку с исходным кодом

🔮 Шаг 04:

Миссия 4

Как безопасно выполнить динамически сгенерированный код?

Подсказка: Контекст выполнения можно ограничивать словарями `globals` и `locals`.

Ответ:

- **Выполнить с ограниченным пространством имен**
- Выполнять код напрямую без контроля
- Перезаписывать встроенные функции
- Сохранять код в файл `.exe`

🔮 Шаг 05:

Миссия 5

Что нужно сделать перед выполнением строки кода через `exec`?

Подсказка: Python должен понимать, что это код, а не обычная строка.

Ответ:

- **Скомпилировать строку в code object через `compile()`**
- Просто вызвать `print` на строке
- Присвоить строку переменной

🔮 Шаг 06:

Миссия 6

Сопоставьте методы генерации функций с их особенностями

Подсказка: Каждая техника имеет свои преимущества и ограничения.

Ответ:

- **`def` внутри функции** — Создает вложенные функции с локальной областью видимости
- **`exec` с `compile`** — Позволяет генерировать код из строк динамически
- **`lambda`** — Создает маленькие анонимные функции на лету
- **`type` с функцией** — Используется для создания методов класса динамически

🔮 Шаг 07:

Миссия 7

Сопоставьте объекты с их ролью при генерации функций

Подсказка: В динамическом коде важны пространство имен и код-объекты.

Ответ:

- **globals** — Словарь глобальных переменных для выполнения кода
- **locals** — Словарь локальных переменных при ехес
- **code object** — Скомпилированный объект, который можно выполнить
- **функция** — Реальный callable объект, созданный динамически

🔗 **Шаг 08:**

Миссия 8

Сопоставьте ошибки Python с ситуациями при генерации функций

Подсказка: Ошибки часто возникают при неправильном синтаксисе или области видимости.

Ответ:

- **SyntaxError** — Когда строка с кодом не соответствует синтаксису Python
- **NameError** — Когда используемая переменная не определена
- **TypeError** — Когда объект не является вызываемым

🔗 **Шаг 09:**

Миссия 9

Сопоставьте типы функций с примерами использования

Подсказка: Некоторые функции лучше подходят для однократного вызова, другие для многократного.

Ответ:

- **lambda** — Короткие функции для использования на месте
- **вложенная функция** — Используется для замыкания и локальных переменных
- **динамически с ехес** — Создается из строки кода на лету
- **метод класса через type** — Добавляется к классу при генерации

🔗 **Шаг 10:**

Миссия 10

Сопоставьте инструменты metaprogramming с их задачами

Подсказка: Каждое средство имеет уникальное предназначение.

Ответ:

- **compile** — Преобразует строку в code object
- **exec** — Выполняет code object или строку кода
- **type** — Создает класс или метод динамически
- **globals/locals** — Определяет контекст выполнения

🔗 **Шаг 11:**

Миссия 11

Расположите шаги динамического создания функции через ехес:

1. Создать строку с кодом функции
2. Скомпилировать строку с compile()
3. Выполнить код через exec с нужными globals и locals
4. Использовать созданную функцию в программе

Подсказка: Сначала создаем код, потом компилируем, затем выполняем и только после этого используем.

Ответ:

- Создать строку с кодом функции
- Скомпилировать строку с `compile()`
- Выполнить код через `exec` с нужными `globals` и `locals`
- Использовать созданную функцию в программе

 Шаг 12:

Миссия 12

Расположите шаги при генерации нескольких функций на лету:

1. Составить код для каждой функции
2. Скомпилировать каждую строку в `code object`
3. Выполнить каждый `code object` через `exec`
4. Добавить функции в словарь для последующего вызова

Подсказка: Динамическое создание нескольких функций похоже на создание одного объекта, только повторяем процесс для каждой.

Ответ:

- Составить код для каждой функции
- Скомпилировать каждую строку в `code object`
- Выполнить каждый `code object` через `exec`
- Добавить функции в словарь для последующего вызова

 Шаг 13:

Миссия 13

Расположите шаги при создании функции с замыканием через динамическое создание:

1. Создать строку кода с вложенной функцией
2. Скомпилировать строку через `compile()`
3. Выполнить `code object` через `exec`
4. Вызвать внешнюю функцию, получить внутреннюю

Подсказка: Замыкание требует сначала определения функции, потом выполнения, потом извлечения внутренней функции.

Ответ:

- Создать строку кода с вложенной функцией
- Скомпилировать строку через `compile()`
- Выполнить `code object` через `exec`
- Вызвать внешнюю функцию, получить внутреннюю

 Шаг 14:

Миссия 14

Расположите шаги безопасного выполнения пользовательских функций:

1. Собрать код функции из пользовательского ввода
2. Скомпилировать через `compile()`
3. Выполнить `code object` в ограниченном словаре `globals`

4. Вызвать функцию с проверенными аргументами

Подсказка: Безопасность важнее скорости: сначала ограничиваем контекст, потом выполняем.

Ответ:

- Собрать код функции из пользовательского ввода
- Скомпилировать через `compile()`
- Выполнить code object в ограниченном словаре `globals`
- Вызвать функцию с проверенными аргументами

Шаг 15:

Миссия 15

Расположите шаги при создании фабрики функций:

1. Создать функцию-генератор с параметрами
2. Сгенерировать строку кода с нужной логикой
3. Скомпилировать строку через `compile()`
4. Выполнить code object через `exec` и вернуть функцию

Подсказка: Фабрика создает функцию: сначала задаем параметры, потом генерируем и выполняем код.

Ответ:

- Создать функцию-генератор с параметрами
- Сгенерировать строку кода с нужной логикой
- Скомпилировать строку через `compile()`
- Выполнить code object через `exec` и вернуть функцию

Урок 3 - Построение классов динамически

Шаг 01:

Миссия 1

Что делает функция `type()` в контексте метапрограммирования?

Подсказка: Она может создавать объекты классов, не определяя их через `class`.

Ответ:

- **Создает класс динамически**
- Выполняет функцию
- Создает модуль
- Компилирует байткод

Шаг 02:

Миссия 2

Что необходимо указать при создании класса через `type()`?

Подсказка: Подумайте о том, что обычно пишут в `class Name(...)`:

Ответ:

- **Имя класса, кортеж базовых классов, словарь атрибутов**
- Имя функции, аргументы, тело функции
- Имя модуля, функции, переменные

Шаг 03:

Миссия 3

Зачем может пригодиться динамическое создание классов?

Подсказка: Иногда количество классов заранее неизвестно.

Ответ:

- **Для генерации классов в зависимости от конфигурации или данных**
- Для ускорения выполнения уже существующего кода
- Для вызова функций без аргументов
- Для создания глобальных переменных

Шаг 04:

Миссия 4

Что передается в словарь атрибутов при `type()`?

Подсказка: Словарь описывает свойства и методы класса.

Ответ:

- **Имя методов и переменные класса с их значениями**
- Список базовых классов
- Имя класса и его документацию

Шаг 05:

Миссия 5

Что позволяет метакласс в Python?

Подсказка: Это "класс классов" — подумайте, кто создает классы.

Ответ:

- **Контролировать создание и модификацию классов**
- Выполнять функции асинхронно
- Создавать объекты без конструктора
- Компилировать байткод

Шаг 06:

Миссия 6

Сопоставьте элементы `type()` с их ролью

Подсказка: Каждый элемент указывает на часть класса.

Ответ:

- **Имя класса** — Определяет идентификатор нового класса
- **Базовые классы** — Определяют наследование и иерархию
- **Словарь атрибутов** — Содержит методы и переменные класса

Шаг 07:

Миссия 7

Сопоставьте сценарий с правильным типом метапрограммирования

Подсказка: Подумайте, где нужна динамика.

Ответ:

- **Создать класс по данным конфигурации** — Динамическое создание классов через `type`
- **Добавить методы ко всем классам сразу** — Использование метакласса
- **Изменить поведение класса при наследовании** — Переопределение `__init_subclass__`

Шаг 08:

Миссия 8

Сопоставьте методы метаклассов с их функцией

Подсказка: Метаклассы управляют созданием класса и его атрибутами.

Ответ:

- **`__new__`** — Создает новый класс
- **`__init__`** — Инициализирует класс после создания
- **`__call__`** — Позволяет экземпляру класса вести себя как функция

Шаг 09:

Миссия 9

Сопоставьте ошибки при динамическом создании классов с причиной

Подсказка: Ошибки появляются при неправильных аргументах или синтаксисе.

Ответ:

- **`TypeError`** — Неправильное количество аргументов в `type` или метаклассе
- **`NameError`** — Попытка использовать неопределенное имя в атрибутах класса
- **`AttributeError`** — Попытка вызвать метод, которого нет

Шаг 10:

Миссия 10

Сопоставьте типы атрибутов с их назначением при динамическом создании классов

Подсказка: Подумайте о методах и переменных, которые будут у экземпляров.

Ответ:

- **Метод** — Функция, доступная через экземпляр класса
- **Свойство** — Переменная, специфичная для каждого экземпляра
- **Класс-атрибут** — Общий для всех экземпляров класса

Шаг 11:

Миссия 11

Сопоставьте ошибки при создании классов динамически с их причиной

Подсказка: Ошибки обычно связаны с неправильной структурой словаря атрибутов.

Ответ:

- **`TypeError`** — Неверный тип значения в словаре атрибутов
- **`ValueError`** — Попытка создать класс с недопустимым именем
- **`AttributeError`** — Использование несуществующего метода или свойства

Шаг 12:

Миссия 12

Сопоставьте сценарий использования динамических классов с примером

Подсказка: Думайте о конфигурациях и внешних данных.

Ответ:

- **Создать классы для всех типов API-эндпоинтов** — Динамическое создание класса для каждого эндпоинта
- **Добавить логирование ко всем методам** — Использование метакласса
- **Изменить поведение наследуемых классов** — Переопределение `__init_subclass__`

Шаг 13:

Миссия 13

Расположите шаги динамического создания класса через `type()` в правильном порядке:

Подсказка: Подумайте, как создается обычный класс, но с использованием `type`.

1. Создание словаря атрибутов класса
2. Определение базовых классов
3. Вызов `type()` с именем, базовыми классами и словарем атрибутов
4. Использование нового класса для создания экземпляров

Ответ:

- Создание словаря атрибутов класса
- Определение базовых классов
- Вызов `type()` с именем, базовыми классами и словарем атрибутов
- Использование нового класса для создания экземпляров

Шаг 14:

Миссия 14

Расположите действия при использовании метаклассов:

Подсказка: Подумайте о жизненном цикле класса и его экземпляров.

1. Определение метакласса
2. Переопределение `__new__` для изменения класса перед созданием
3. Создание класса с указанием метакласса
4. Создание экземпляра класса

Ответ:

- Определение метакласса
- Переопределение `__new__` для изменения класса перед созданием
- Создание класса с указанием метакласса
- Создание экземпляра класса

Шаг 15:

Миссия 15

Расположите этапы расширения классов с использованием метаклассов:

Подсказка: Сначала создается базовый класс, потом добавляются изменения метакласса.

1. Создание базового класса
2. Определение метакласса с дополнительными методами
3. Создание класса с использованием метакласса

4. Создание экземпляра и проверка добавленных методов

Ответ:

- Создание базового класса
- Определение метакласса с дополнительными методами
- Создание класса с использованием метакласса
- Создание экземпляра и проверка добавленных методов

Шаг 16:

Миссия 16

Расположите шаги тестирования динамически созданного класса:

Подсказка: Сначала проверяем существование класса, потом функциональность методов.

1. Создание экземпляра класса
2. Проверка атрибутов экземпляра
3. Вызов методов экземпляра
4. Проверка корректности поведения всех методов

Ответ:

- Создание экземпляра класса
- Проверка атрибутов экземпляра
- Вызов методов экземпляра
- Проверка корректности поведения всех методов

Урок 4 - Практика генератор декораторов

Шаг 01:

Миссия 1

Что является основной целью декоратора в Python?

Подсказка: Декоратор изменяет поведение функции без изменения её кода.

Ответ:

- **Изменение или расширение поведения функции**
- Создание нового класса
- Изменение глобальных переменных
- Перезапись стандартной библиотеки

Шаг 02:

Миссия 2

Какой синтаксис используется для применения декоратора к функции?

Подсказка: Сначала укажите @, а потом имя функции-декоратора.

Ответ:

- **@имя_декоратора**
- #имя_декоратора
- def имя_декоратора():
- \$имя_декоратора

Шаг 03:

Миссия 3

Что необходимо вернуть из функции-декоратора?

Подсказка: Декоратор заменяет исходную функцию другой функцией.

Ответ:

- **Функцию-обертку, которая будет вызываться вместо исходной**
- Строку с именем функции
- Класс функции
- Ничего не возвращать

Шаг 04:

Миссия 4

Какая задача может быть автоматизирована с помощью генератора декораторов?

Подсказка: Подумайте о повторяющихся действиях перед и после вызова функции.

Ответ:

- **Логирование вызовов разных функций с одинаковым шаблоном**
- Создание новых классов
- Редактирование кода функции вручную
- Компиляция модулей Python

Шаг 05:

Миссия 5

Сопоставьте понятия генераторов декораторов с их описанием

Подсказка: Обратите внимание на то, какие функции они возвращают.

Ответ:

- **Функция-обертка** — Функция, которая выполняет дополнительный код и вызывает исходную функцию
- **Генератор декораторов** — Функция, возвращающая декоратор с параметрами
- **Декоратор без параметров** — Функция, принимающая только исходную функцию

Шаг 06:

Миссия 6

Сопоставьте типы декораторов с типичными примерами использования

Подсказка: Подумайте, где удобно применять функции-обертки.

Ответ:

- **Декоратор логирования** — Отслеживание вызовов функции и параметров
- **Декоратор кэширования** — Сохранение результатов функции для ускорения повторных вызовов
- **Декоратор авторизации** — Проверка прав пользователя перед выполнением функции

Шаг 07:

Миссия 7

Сопоставьте элементы генератора декораторов с их назначением

Подсказка: Подумайте о том, что создается и возвращается на каждом уровне вложенности функций.

Ответ:

- **Внешняя функция генератора** — Принимает параметры декоратора
- **Декоратор** — Принимает функцию и возвращает функцию-обертку
- **Функция-обертка** — Вызывается вместо исходной функции и выполняет дополнительный код

Шаг 08:

Миссия 8

Сопоставьте ошибки при использовании генераторов декораторов с их причиной

Подсказка: Часто ошибки связаны с несоответствием параметров или возвращаемого значения.

Ответ:

- **TypeError** — Функция-обертка возвращает None вместо функции
- **SyntaxError** — Неправильный синтаксис @ перед функцией
- **NameError** — Использование переменной до её определения

Шаг 09:

Миссия 9

Расположите шаги генератора декораторов с параметрами в правильном порядке:

Подсказка: Сначала создается генератор, потом декоратор, потом функция-обертка.

1. Вызов генератора с параметрами
2. Создание функции-декоратора
3. Создание функции-обертки внутри декоратора
4. Вызов исходной функции через функцию-обертку

Ответ:

- Вызов генератора с параметрами
- Создание функции-декоратора
- Создание функции-обертки внутри декоратора
- Вызов исходной функции через функцию-обертку

Шаг 10:

Миссия 10

Расположите этапы применения нескольких декораторов к одной функции:

Подсказка: Декораторы применяются сверху вниз, а вызываются снизу вверх.

1. Объявление функции с несколькими @декораторами
2. Применение самого верхнего декоратора
3. Применение нижнего декоратора
4. Вызов функции, обернутой всеми декораторами

Ответ:

- Объявление функции с несколькими @декораторами
- Применение самого верхнего декоратора
- Применение нижнего декоратора
- Вызов функции, обернутой всеми декораторами

Шаг 11:

Миссия 11

Расположите шаги создания универсального генератора декораторов:

Подсказка: Сначала определяем внешний интерфейс, потом внутренние функции.

1. Определение функции генератора с параметрами
2. Создание функции-декоратора внутри генератора
3. Создание функции-обертки внутри декоратора
4. Возврат функции-декоратора из генератора
5. Вызов функции-обертки вместо исходной функции

Ответ:

- Определение функции генератора с параметрами
- Создание функции-декоратора внутри генератора
- Создание функции-обертки внутри декоратора
- Возврат функции-декоратора из генератора
- Вызов функции-обертки вместо исходной функции

Шаг 12:

Миссия 12

Расположите шаги тестирования декораторов с параметрами:

Подсказка: Сначала проверяем генератор, потом декоратор, потом функцию-обертку.

1. Вызов генератора с параметрами
2. Проверка корректности возвращаемого декоратора
3. Применение декоратора к тестовой функции
4. Вызов функции и проверка результатов
5. Логирование и анализ вызовов

Ответ:

- Вызов генератора с параметрами
- Проверка корректности возвращаемого декоратора
- Применение декоратора к тестовой функции
- Вызов функции и проверка результатов
- Логирование и анализ вызовов

Шаг 13:

Миссия 13

Расположите этапы создания декоратора для логирования времени выполнения:

Подсказка: Сначала определяем обертку, потом измеряем время, затем вызываем функцию.

1. Определение функции-обертки
2. Замер времени начала выполнения
3. Вызов исходной функции
4. Замер времени окончания и вывод результата
5. Возврат результата исходной функции

Ответ:

- Определение функции-обертки

- Замер времени начала выполнения
- Вызов исходной функции
- Замер времени окончания и вывод результата
- Возврат результата исходной функции

🔮 Шаг 14:

Миссия 14

Расположите шаги создания декоратора с параметром "максимальное время выполнения":

Подсказка: Сначала обрабатываем параметр, потом создаем функцию-обертку и проверяем время.

1. Определение функции генератора с параметром `max_time`
2. Создание функции-декоратора внутри генератора
3. Создание функции-обертки, которая измеряет время выполнения
4. Проверка, что время не превышает `max_time`
5. Возврат функции-обертки из декоратора

Ответ:

- Определение функции генератора с параметром `max_time`
- Создание функции-декоратора внутри генератора
- Создание функции-обертки, которая измеряет время выполнения
- Проверка, что время не превышает `max_time`
- Возврат функции-обертки из декоратора

🔮 Шаг 15:

Миссия 15

Расположите этапы создания декоратора кэширования с параметрами:

Подсказка: Сначала определяем генератор, потом декоратор, потом функцию-обертку, затем кэшируем результаты.

1. Определение генератора декоратора с параметрами
2. Создание функции-декоратора внутри генератора
3. Создание функции-обертки
4. Проверка кэша перед вызовом функции
5. Сохранение результата в кэш после вызова функции

Ответ:

- Определение генератора декоратора с параметрами
- Создание функции-декоратора внутри генератора
- Создание функции-обертки
- Проверка кэша перед вызовом функции
- Сохранение результата в кэш после вызова функции

🔮 Шаг 16:

Миссия 16

Расположите шаги создания декоратора проверки прав пользователя:

Подсказка: Сначала получаем права, потом проверяем, потом вызываем функцию или блокируем.

1. Определение функции-обертки
2. Получение текущих прав пользователя
3. Проверка прав на выполнение функции
4. Вызов исходной функции при разрешении
5. Возврат результата функции или ошибка доступа

Ответ:

- Определение функции-обертки
- Получение текущих прав пользователя
- Проверка прав на выполнение функции
- Вызов исходной функции при разрешении
- Возврат результата функции или ошибка доступа

 **Шаг 17:**

Миссия 17

Расположите полные шаги использования генератора декораторов от создания до вызова:

Подсказка: Сначала генератор, потом декоратор, потом функция-обертка, затем тестирование и вызов функции.

1. Создание генератора декоратора с параметрами
2. Создание декоратора внутри генератора
3. Создание функции-обертки внутри декоратора
4. Применение декоратора к исходной функции
5. Вызов функции-обертки
6. Проверка корректности работы и логирования

Ответ:

- Создание генератора декоратора с параметрами
- Создание декоратора внутри генератора
- Создание функции-обертки внутри декоратора
- Применение декоратора к исходной функции
- Вызов функции-обертки
- Проверка корректности работы и логирования

Урок 5 - Лабиринт автоматический генератор API

 **Шаг 01:**

Миссия 1

Что нужно определить в первую очередь при автоматическом создании API?

Подсказка: Начните с базовой структуры и целей API.

Ответ:

- **Определить конечные точки (endpoints) и их назначения**
- Сразу писать документацию
- Создавать тесты перед кодом
- Реализовать авторизацию без структуры

Шаг 02:

Миссия 2

Что важно учитывать при генерации моделей данных для API?

Подсказка: Подумайте о типах данных и связи между объектами.

Ответ:

- **Типы полей и их взаимосвязи**
- Только названия классов
- Создавать SQL-запросы вручную
- Игнорировать валидацию данных

Шаг 03:

Миссия 3

Какой подход эффективен для автоматического создания маршрутов API?

Подсказка: Подумайте о повторяющихся структурах и шаблонах.

Ответ:

- **Использование шаблонов и генераторов маршрутов**
- Создание каждой функции вручную
- Игнорирование маршрутов
- Сразу писать тесты без структуры

Шаг 04:

Миссия 4

 Сопоставьте элементы API с их описанием:

Подсказка: Подумайте, какая часть API за что отвечает.

Ответ:

- Endpoint
 - Serializer
 - View
 - Permission
1. Определяет путь и метод запроса
 2. Преобразует данные между форматом JSON и объектами Python
 3. Содержит логику обработки запроса и возврата ответа
 4. Проверяет права доступа пользователя
- **Endpoint** — Определяет путь и метод запроса
 - **Serializer** — Преобразует данные между форматом JSON и объектами Python
 - **View** — Содержит логику обработки запроса и возврата ответа
 - **Permission** — Проверяет права доступа пользователя

Шаг 05:

Миссия 5

 Сопоставьте типы HTTP-запросов с их назначением:

Подсказка: Подумайте, какой метод изменяет данные, а какой только получает.

Ответ:

- GET
- POST
- PUT
- DELETE

1. Получение данных без их изменения
2. Создание нового ресурса
3. Полная замена существующего ресурса
4. Удаление ресурса

- **GET** — Получение данных без их изменения
- **POST** — Создание нового ресурса
- **PUT** — Полная замена существующего ресурса
- **DELETE** — Удаление ресурса

 Шаг 06:

Миссия 6

 Сопоставьте компоненты автоматического API с их задачей:

Подсказка: Каждый компонент выполняет уникальную функцию.

Ответ:

- Router
- Model
- Service
- Validator

1. Маршрутизирует запросы к нужной функции
2. Определяет структуру данных и поля
3. Содержит бизнес-логику приложения
4. Проверяет корректность входных данных

- **Router** — Маршрутизирует запросы к нужной функции
- **Model** — Определяет структуру данных и поля
- **Service** — Содержит бизнес-логику приложения
- **Validator** — Проверяет корректность входных данных

 Шаг 07:

Миссия 7

Расположите шаги генерации API в правильном порядке:

Подсказка: Начните с моделей, затем маршруты и сервисы, в конце тестирование.

1. Создание моделей данных
2. Создание сериализаторов
3. Создание сервисов с бизнес-логикой
4. Настройка маршрутов (Router)
5. Тестирование API и проверка корректности

Ответ:

- Создание моделей данных

- Создание сериализаторов
- Создание сервисов с бизнес-логикой
- Настройка маршрутов (Router)
- Тестирование API и проверка корректности

Шаг 08:

Миссия 8

Расположите этапы генерации документации API в правильном порядке:

Подсказка: Сначала описание моделей, потом endpoints, затем примеры запросов и ответов.

1. Определение моделей и схем данных
2. Описание маршрутов и методов
3. Примеры запросов и ответов
4. Генерация итоговой документации

Ответ:

- Определение моделей и схем данных
- Описание маршрутов и методов
- Примеры запросов и ответов
- Генерация итоговой документации

Шаг 09:

Миссия 9

Что помогает избежать дублирования кода при генерации API?

Подсказка: Подумайте о повторно используемых шаблонах.

Ответ:

- **Использование универсальных генераторов и шаблонов**
- Копирование функций между модулями
- Создание одинаковых функций для каждого endpoint
- Игнорирование повторяющихся частей

Шаг 10:

Миссия 10

Что следует сделать для автоматической валидации входных данных API?

Подсказка: Подумайте о проверке типов, обязательных полей и формата.

Ответ:

- **Использовать валидаторы и схемы данных**
- Игнорировать формат данных
- Сразу записывать данные в базу без проверки
- Создавать только ручные проверки в коде каждого метода

Шаг 11:

Миссия 11

 Сопоставьте типы API-тестов с их назначением:

Подсказка: Каждый тест проверяет разные аспекты работы API.

Ответ:

- Unit Test
- Integration Test
- End-to-End Test
- Load Test

1. Проверка отдельных функций или методов
2. Проверка взаимодействия между компонентами
3. Проверка полной работы системы от начала до конца
4. Проверка устойчивости API под нагрузкой

- **Unit Test** — Проверка отдельных функций или методов
- **Integration Test** — Проверка взаимодействия между компонентами
- **End-to-End Test** — Проверка полной работы системы от начала до конца
- **Load Test** — Проверка устойчивости API под нагрузкой

 Шаг 12:

Миссия 12

 Сопоставьте этапы деплоя API с их задачей:

Подсказка: Думайте о порядке от разработки к производству.

Ответ:

- Build
- Test
- Deploy
- Monitor

1. Сборка кода и зависимостей
2. Запуск тестов и проверка работы
3. Размещение API на сервере
4. Отслеживание состояния и производительности API

- **Build** — Сборка кода и зависимостей
- **Test** — Запуск тестов и проверка работы
- **Deploy** — Размещение API на сервере
- **Monitor** — Отслеживание состояния и производительности API

 Шаг 13:

Миссия 13

Расположите этапы генерации REST API с использованием генератора в правильном порядке:

Подсказка: Начните с моделей, затем сериализаторы, views, маршруты и тестирование.

1. Создание моделей данных
2. Создание сериализаторов
3. Создание views с логикой
4. Настройка маршрутов
5. Тестирование работы API

Ответ:

- Создание моделей данных

- Создание сериализаторов
- Создание views с логикой
- Настройка маршрутов
- Тестирование работы API

🔮 Шаг 14:

Миссия 14

Расположите этапы автоматической генерации CRUD API:

Подсказка: Сначала создаём структуры, потом методы и маршруты, затем проверку.

1. Определение моделей
2. Создание CRUD методов (create, read, update, delete)
3. Настройка маршрутов для каждого метода
4. Тестирование и валидация API

Ответ:

- Определение моделей
- Создание CRUD методов (create, read, update, delete)
- Настройка маршрутов для каждого метода
- Тестирование и валидация API

🔮 Шаг 15:

Миссия 15

Какой подход ускоряет разработку API при генерации кода?

Подсказка: Используйте повторно применяемые шаблоны и генераторы.

Ответ:

- **Использование шаблонов кода и генераторов функций**
- Каждый endpoint писать вручную
- Игнорировать повторяющиеся структуры
- Сразу писать документацию без кода

🔮 Шаг 16:

Миссия 16

Что важно проверить после генерации API перед деплоем?

Подсказка: Подумайте о корректности работы, безопасности и соответствии спецификации.

Ответ:

- **Тесты, валидация данных и права доступа**
- Игнорирование ошибок при генерации
- Пропуск проверки маршрутов
- Сразу публиковать без тестирования

Модуль 4 - Метамагия runtime

Урок 1 - Monkey patching тёмное искусство

Шаг 01:

Миссия 1

Что такое monkey patching в Python?

Подсказка: Это способ изменить поведение объекта во время выполнения.

Ответ:

- **Изменение или добавление методов/атрибутов объекта во время выполнения**
- Создание нового класса без изменений существующих
- Переписывание стандартной библиотеки на C
- Использование только функций без классов

Шаг 02:

Миссия 2

Когда безопаснее использовать monkey patching?

Подсказка: Подумайте о случаях, когда нет возможности изменить исходный код.

Ответ:

- **Для временных исправлений внешней библиотеки без изменения её исходного кода**
- Для переписывания своего проекта с нуля
- Для полного отказа от тестов
- Для изменения стандартной библиотеки на постоянной основе без контроля

Шаг 03:

Миссия 3

Какой модуль помогает временно заменить функцию для тестов?

Подсказка: Он используется для подмены функций без постоянного изменения.

Ответ:

- **unittest.mock**
- time
- random
- functools

Шаг 04:

Миссия 4

 Сопоставьте типы monkey patch с примером:

Подсказка: Внимательно читайте, где изменяется поведение объекта.

Ответ:

- Патч класса
- Патч метода экземпляра
- Патч модуля

- Патч функции

1. Замена метода класса на кастомный
2. Изменение метода только одного объекта
3. Подмена функции внутри модуля
4. Подмена глобальной функции в коде

- **Патч класса** — Замена метода класса на кастомный
- **Патч метода экземпляра** — Изменение метода только одного объекта
- **Патч модуля** — Подмена функции внутри модуля
- **Патч функции** — Подмена глобальной функции в коде

Шаг 05:

Миссия 5

 Сопоставьте опасность monkey patch с описанием:

Подсказка: Обратите внимание на долгосрочные последствия.

Ответ:

- Нарушение инкапсуляции
 - Неожиданные ошибки
 - Сложность отладки
 - Конфликты с другими библиотеками
1. Доступ к внутренним данным объектов напрямую
 2. Методы ведут себя не так, как ожидалось
 3. Трудно понять, что изменено
 4. Разные патчи могут перезаписывать друг друга
- **Нарушение инкапсуляции** — Доступ к внутренним данным объектов напрямую
 - **Неожиданные ошибки** — Методы ведут себя не так, как ожидалось
 - **Сложность отладки** — Трудно понять, что изменено
 - **Конфликты с другими библиотеками** — Разные патчи могут перезаписывать друг друга

Шаг 06:

Миссия 6

Как снизить риск при использовании monkey patch?

Подсказка: Подумайте о документации и ограничении времени жизни патча.

Ответ:

- **Использовать контекстные менеджеры и подробно документировать изменения**
- Менять всё глобально без ограничений
- Игнорировать тесты
- Применять патчи на постоянной основе без контроля

Шаг 07:

Миссия 7

Что использовать для временной подмены функции только в тесте?

Подсказка: Подумайте о библиотеке для моков.

Ответ:

- **mock.patch**
- time.sleep
- importlib.reload
- globals()

Шаг 08:

Миссия 8

Расположите шаги применения monkey patch для исправления метода класса:

Подсказка: Сначала определяем новый метод, потом временно подменяем старый, тестируем и откатываем.

1. Определение новой функции
2. Замена метода класса на новую функцию
3. Проверка работы в тестах
4. Возврат исходного метода после теста

Ответ:

- Определение новой функции
- Замена метода класса на новую функцию
- Проверка работы в тестах
- Возврат исходного метода после теста

Шаг 09:

Миссия 9

Расположите шаги применения monkey patch для исправления метода класса:

Подсказка: Сначала определяем новый метод, потом временно подменяем старый, тестируем и откатываем.

1. Определение новой функции
2. Замена метода класса на новую функцию
3. Проверка работы в тестах
4. Возврат исходного метода после теста

Ответ:

- Определение новой функции
- Замена метода класса на новую функцию
- Проверка работы в тестах
- Возврат исходного метода после теста

Шаг 10:

Миссия 10

Почему не рекомендуется использовать monkey patching на продакшене?

Подсказка: Подумайте о стабильности и поддержке кода.

Ответ:

- **Может вызвать непредсказуемое поведение и сломать другие части системы**
- Программа будет работать быстрее
- Уменьшает количество тестов

- Всегда делает код проще

Шаг 11:

Миссия 11

 Сопоставьте термин monkey patch с его примером:

Подсказка: Смотрите, что именно меняется в коде во время выполнения.

Ответ:

- Патч метода экземпляра
 - Патч атрибута класса
 - Патч функции модуля
 - Временная подмена метода
1. Изменяем метод только одного объекта
 2. Изменяем атрибут, общий для всех объектов класса
 3. Подменяем функцию, импортированную из модуля
 4. Меняем метод на время выполнения теста
- **Патч метода экземпляра** — Изменяем метод только одного объекта
 - **Патч атрибута класса** — Изменяем атрибут, общий для всех объектов класса
 - **Патч функции модуля** — Подменяем функцию, импортированную из модуля
 - **Временная подмена метода** — Меняем метод на время выполнения теста

Шаг 12:

Миссия 12

Как можно минимизировать побочные эффекты monkey patch?

Подсказка: Подумайте о локальности изменений и использовании контекста.

Ответ:

- **Использовать контекстный менеджер и откатывать изменения после использования**
- Менять код глобально без отката
- Игнорировать документацию и тесты
- Оставлять изменения навсегда

Шаг 13:

Миссия 13

 Сопоставьте пример monkey patch с результатом:

Подсказка: Обратите внимание, что изменяется именно поведение функции.

Ответ:

- Подмена метода print
 - Добавление метода к классу
 - Изменение метода экземпляра
 - Подмена функции в модуле datetime
1. Все вызовы print выводят дополнительный текст
 2. Новый метод доступен всем объектам класса
 3. Только один объект изменил своё поведение
 4. Функция datetime.now() возвращает фиксированное время

- **Подмена метода `print`** — Все вызовы `print` выводят дополнительный текст
- **Добавление метода к классу** — Новый метод доступен всем объектам класса
- **Изменение метода экземпляра** — Только один объект изменил своё поведение
- **Подмена функции в модуле `datetime`** — Функция `datetime.now()` возвращает фиксированное время

🔧 Шаг 14:

Миссия 14

Что является преимуществом `monkey patch` для тестирования?

Подсказка: Подумайте, как временные изменения могут помочь проверить код.

Ответ:

- **Позволяет подменять функции или методы для изоляции тестов**
- Сразу исправляет все ошибки в продакшене
- Ускоряет выполнение всего приложения
- Убирает необходимость писать тесты

🔧 Шаг 15:

Миссия 15

Расположите действия по безопасному применению `monkey patch` в правильном порядке:

Подсказка: Сначала планируем, потом патчим, тестируем и откатываем.

1. Определяем метод, который нужно заменить
2. Создаем новую функцию с желаемым поведением
3. Подменяем метод на время теста
4. Проверяем результаты в тестах
5. Возвращаем исходный метод после теста

Ответ:

- Определяем метод, который нужно заменить
- Создаем новую функцию с желаемым поведением
- Подменяем метод на время теста
- Проверяем результаты в тестах
- Возвращаем исходный метод после теста

🔧 Шаг 16:

Миссия 16

📄 Сопоставьте метод `monkey patch` с его типичным применением:

Подсказка: Подумайте, где нужна временная модификация поведения.

Ответ:

- Патч метода класса
 - Патч функции модуля
 - Временная подмена для теста
 - Изменение метода экземпляра
1. Все объекты класса используют новую логику
 2. Изменяем поведение функции, импортированной в код
 3. Подменяем только на время выполнения теста

4. Меняется поведение только одного объекта

- **Патч метода класса** — Все объекты класса используют новую логику
- **Патч функции модуля** — Изменяем поведение функции, импортированной в код
- **Временная подмена для теста** — Подменяем только на время выполнения теста
- **Изменение метода экземпляра** — Меняется поведение только одного объекта

🔮 Шаг 17:

Миссия 17

Почему monkey patching может быть полезен для сторонних библиотек?

Подсказка: Подумайте о невозможности изменить исходный код.

Ответ:

- **Позволяет временно исправлять ошибки или добавлять функционал, не меняя исходный код**
- Автоматически делает код быстрее
- Удаляет необходимость тестов
- Всегда безопасно для продакшена

📖 Урок 2 - Магия декораторов и метаклассов

🔮 Шаг 01:

Миссия 1

Для чего в Python используют декораторы?

Подсказка: Подумайте, как можно изменить поведение функции без изменения её кода.

Ответ:

- **Для обёртывания функций и изменения их поведения**
- Для объявления глобальных переменных
- Для оптимизации циклов
- Для замены классов на функции

🔮 Шаг 02:

Миссия 2

Что делает метакласс в Python?

Подсказка: Подумайте о том, как создаются классы.

Ответ:

- **Определяет поведение создания классов**
- Создаёт объекты без конструктора
- Запускает код асинхронно
- Автоматически наследует методы

🔮 Шаг 03:

Миссия 3

📄 Сопоставьте тип декоратора с его примером:

Подсказка: Внимательно читайте, что именно изменяет декоратор.

Ответ:

- Декоратор функции
- Декоратор метода класса
- Декоратор класса
- Декоратор с параметрами

1. Оборачивает обычную функцию для дополнительного поведения
2. Изменяет поведение метода класса
3. Оборачивает весь класс, добавляя новые методы или атрибуты
4. Принимает аргументы и возвращает декоратор

- **Декоратор функции** — Оборачивает обычную функцию для дополнительного поведения
- **Декоратор метода класса** — Изменяет поведение метода класса
- **Декоратор класса** — Оборачивает весь класс, добавляя новые методы или атрибуты
- **Декоратор с параметрами** — Принимает аргументы и возвращает декоратор

 **Шаг 04:**

Миссия 4

Какой из вариантов наиболее безопасен при использовании декораторов в продакшене?

Подсказка: Подумайте о тестируемости и читаемости кода.

Ответ:

- **Использовать декоратор с сохранением метаданных исходной функции**
- Перезаписывать функции без обёртки
- Использовать глобальные переменные внутри декоратора
- Игнорировать тесты

 **Шаг 05:**

Миссия 5

 Сопоставьте термин метакласса с его функцией:

Подсказка: Подумайте, что именно создаёт метакласс.

Ответ:

- type
 - __new__
 - __init__
 - __call__
1. Создаёт новый класс динамически
 2. Выделяет память и формирует объект класса
 3. Инициализирует уже созданный класс
 4. Создаёт экземпляр класса при вызове
- **type** — Создаёт новый класс динамически
 - **__new__** — Выделяет память и формирует объект класса
 - **__init__** — Инициализирует уже созданный класс
 - **__call__** — Создаёт экземпляр класса при вызове

 **Шаг 06:**

Миссия 6

Почему важно сохранять метаданные функции при декорировании?

Подсказка: Подумайте о документации и тестах.

Ответ:

- **Чтобы документация, сигнатура и тесты функции оставались корректными**
- Чтобы ускорить выполнение функции
- Чтобы функция не требовала аргументов
- Чтобы функция автоматически стала методом класса

Шаг 07:

Миссия 7

 Сопоставьте декоратор с его эффектом:

Подсказка: Внимательно читайте описание поведения.

Ответ:

- `@staticmethod`
 - `@classmethod`
 - `@property`
 - `@functools.wraps`
1. Метод не зависит от экземпляра
 2. Метод получает класс как первый аргумент
 3. Метод вызывается как атрибут без скобок
 4. Сохраняет метаданные исходной функции
- **`@staticmethod`** — Метод не зависит от экземпляра
 - **`@classmethod`** — Метод получает класс как первый аргумент
 - **`@property`** — Метод вызывается как атрибут без скобок
 - **`@functools.wraps`** — Сохраняет метаданные исходной функции

Шаг 08:

Миссия 8

Что делает метод `__init_subclass__`?

Подсказка: Подумайте, когда вызывается этот метод.

Ответ:

- **Вызывается при создании подкласса, чтобы модифицировать его поведение**
- Создаёт экземпляр класса
- Оборачивает функцию декоратором
- Удаляет методы класса

Шаг 09:

Миссия 9

Что делает декоратор `@abstractmethod`?

Подсказка: Подумайте, какой метод обязательно должен быть реализован в подклассах.

Ответ:

- **Помечает метод как абстрактный, обязательный для реализации в подклассах**
- Автоматически реализует метод для всех подклассов
- Удаляет метод из класса

- Делает метод приватным

🔮 Шаг 10:

Миссия 10

📖 Сопоставьте методы класса с их ролью в метаклассе:

Подсказка: Внимательно читайте описание.

Ответ:

- `__prepare__`
 - `__new__`
 - `__init__`
 - `__call__`
1. Возвращает словарь для определения атрибутов класса
 2. Создаёт объект класса
 3. Инициализирует созданный класс
 4. Создаёт экземпляр класса при вызове
- **`__prepare__`** — Возвращает словарь для определения атрибутов класса
 - **`__new__`** — Создаёт объект класса
 - **`__init__`** — Инициализирует созданный класс
 - **`__call__`** — Создаёт экземпляр класса при вызове

🔮 Шаг 11:

Миссия 11

Почему стоит использовать `functools.wraps` при написании декоратора?

Подсказка: Подумайте о документации и отладке.

Ответ:

- **Чтобы сохранять имя, документацию и сигнатуру исходной функции**
- Чтобы ускорить выполнение функции
- Чтобы сделать функцию статической
- Чтобы автоматически регистрировать декораторы

🔮 Шаг 12:

Миссия 12

📖 Сопоставьте термин с его примером в контексте метаклассов:

Подсказка: Внимательно читайте, что именно создаёт класс.

Ответ:

- Метакласс
 - Экземпляр класса
 - Декоратор класса
 - Абстрактный метод
1. Создаёт новые классы динамически
 2. Создаёт объект на основе класса
 3. Оборачивает класс для изменения поведения
 4. Метод, который должен быть реализован в подклассах
- **Метакласс** — Создаёт новые классы динамически

- **Экземпляр класса** — Создаёт объект на основе класса
- **Декоратор класса** — Оборачивает класс для изменения поведения
- **Абстрактный метод** — Метод, который должен быть реализован в подклассах

🔮 Шаг 13:

Миссия 13

Какой декоратор позволяет методам работать без создания экземпляра класса?

Подсказка: Подумайте о методах, которые не используют `self`.

Ответ:

- **@staticmethod**
- **@classmethod**
- **@property**
- **@abstractmethod**

🔮 Шаг 14:

Миссия 14

📖 Сопоставьте термин декоратора с результатом его применения:

Подсказка: Внимательно читайте функционал.

Ответ:

- **@abstractmethod**
 - **@property**
 - **@classmethod**
 - **@staticmethod**
1. Метод обязателен для реализации в подклассах
 2. Метод доступен как атрибут без вызова
 3. Метод получает класс как первый аргумент
 4. Метод не зависит от экземпляра
- **@abstractmethod** — Метод обязателен для реализации в подклассах
 - **@property** — Метод доступен как атрибут без вызова
 - **@classmethod** — Метод получает класс как первый аргумент
 - **@staticmethod** — Метод не зависит от экземпляра

🔮 Шаг 15:

Миссия 15

Зачем использовать метаклассы вместо обычных классов?

Подсказка: Подумайте о контроле создания классов.

Ответ:

- **Чтобы контролировать создание и модификацию классов**
- Чтобы ускорить исполнение функций
- Чтобы заменить декораторы
- Чтобы создавать глобальные переменные автоматически

🔮 Шаг 16:

Миссия 16

Расположите действия при применении декоратора к функции в правильном порядке:

1. Создаётся функция-декоратор
2. Обертывается исходная функция
3. Возвращается новая функция
4. Вызывается обёрнутая функция

Подсказка: Сначала создаётся декоратор, потом оборачиваем функцию, затем новая функция возвращается, и только при вызове выполняется обёртка.

Ответ:

- Создаётся функция-декоратор
- Обертывается исходная функция
- Возвращается новая функция
- Вызывается обёрнутая функция

Урок 3 - Динамические прокси и адаптеры

Шаг 01:

Миссия 1

Что такое динамический прокси в Python?

Подсказка: Подумайте о объекте, который контролирует доступ к другому объекту.

Ответ:

- **Объект, который перехватывает обращения к другому объекту**
- Объект, который сохраняет состояние программы
- Объект, который автоматически выполняет тесты
- Объект, который сериализует данные

Шаг 02:

Миссия 2

Зачем использовать адаптер в Python?

Подсказка: Подумайте о совместимости интерфейсов разных классов.

Ответ:

- **Чтобы один объект мог работать с интерфейсом другого объекта**
- Чтобы ускорить выполнение методов
- Чтобы автоматически создавать классы
- Чтобы хранить конфигурацию приложения

Шаг 03:

Миссия 3

Что делает метод `__getattr__` в прокси-классе?

Подсказка: Подумайте о доступе к атрибутам объекта, которых нет в прокси.

Ответ:

- **Перехватывает обращения к отсутствующим атрибутам и перенаправляет их**
- Создаёт новые методы класса
- Удаляет атрибуты объекта
- Автоматически вызывает конструктор

Шаг 04:

Миссия 4

 Сопоставьте термин с его описанием:

Подсказка: Внимательно читайте роли объектов.

Ответ:

- Динамический прокси
 - Адаптер
 - Композиция
 - Метакласс
1. Объект, который контролирует доступ к другому объекту
 2. Объект, который позволяет использовать несовместимый интерфейс
 3. Объект, объединяющий несколько объектов для работы
 4. Класс, который создаёт другие классы
- **Динамический прокси** — Объект, который контролирует доступ к другому объекту
 - **Адаптер** — Объект, который позволяет использовать несовместимый интерфейс
 - **Композиция** — Объект, объединяющий несколько объектов для работы
 - **Метакласс** — Класс, который создаёт другие классы

Шаг 05:

Миссия 5

Какой метод следует переопределить, чтобы прокси перехватывал все вызовы методов объекта?

Подсказка: Подумайте о вызове функций через `__call__` или `__getattr__`.

Ответ:

- `__getattr__`
- `__init__`
- `__str__`
- `__eq__`

Шаг 06:

Миссия 6

 Сопоставьте паттерн с примером:

Подсказка: Подумайте о том, что именно меняет объект или интерфейс.

Ответ:

- Proxy
 - Adapter
 - Decorator
 - Singleton
1. Класс, который оборачивает объект для добавления логики
 2. Объект, который управляет доступом к другому объекту
 3. Объект, который позволяет использовать несовместимый интерфейс
 4. Класс, имеющий только один экземпляр
- **Proxy** — Объект, который управляет доступом к другому объекту

- **Adapter** — Объект, который позволяет использовать несовместимый интерфейс
- **Decorator** — Класс, который оборачивает объект для добавления логики
- **Singleton** — Класс, имеющий только один экземпляр

🔗 Шаг 07:

Миссия 7

Что обеспечивает прозрачность прокси для пользователя?

Подсказка: Подумайте о том, чтобы вызовы выглядели как обращения к настоящему объекту.

Ответ:

- **Переопределение `__getattr__` и `__setattr__` для передачи всех обращений к реальному объекту**
- Добавление глобальных переменных
- Сохранение кэша объектов
- Автоматическая сериализация объекта

🔗 Шаг 08:

Миссия 8

Как адаптер помогает интегрировать старый код с новым API?

Подсказка: Подумайте, как изменить интерфейс без изменения внутренней логики.

Ответ:

- **Он преобразует интерфейс старого объекта под требования нового API**
- Он переписывает внутреннюю логику старого объекта
- Он ускоряет выполнение методов старого объекта
- Он создаёт новый экземпляр класса автоматически

🔗 Шаг 09:

Миссия 9

📄 Сопоставьте метод прокси с его ролью:

Подсказка: Каждый метод отвечает за свой аспект контроля доступа.

Ответ:

- `__getattr__`
 - `__setattr__`
 - `__call__`
 - `__delattr__`
1. Перехватывает обращения к несуществующим атрибутам
 2. Перехватывает попытки установки атрибутов
 3. Перехватывает вызов объекта как функции
 4. Перехватывает удаление атрибутов объекта
- **`__getattr__`** — Перехватывает обращения к несуществующим атрибутам
 - **`__setattr__`** — Перехватывает попытки установки атрибутов
 - **`__call__`** — Перехватывает вызов объекта как функции
 - **`__delattr__`** — Перехватывает удаление атрибутов объекта

🔗 Шаг 10:

Миссия 10

Что такое виртуальный адаптер?

Подсказка: Подумайте о прокси, который меняет интерфейс без реального создания объекта.

Ответ:

- **Адаптер, который изменяет интерфейс объекта на лету, без создания нового экземпляра**
- Адаптер, который дублирует объект полностью
- Метод для вызова всех функций объекта
- Объект, который автоматически тестирует старый код

🔗 Шаг 11:

Миссия 11

📄 Сопоставьте паттерн с эффектом:

Подсказка: Смотрите, что именно объект изменяет.

Ответ:

- Dynamic Proxy
 - Adapter
 - Decorator
 - Facade
1. Перехватывает все обращения к объекту
 2. Изменяет интерфейс для совместимости
 3. Добавляет новые возможности объекту без изменения его кода
 4. Упрощает доступ к сложной системе
- **Dynamic Proxy** — Перехватывает все обращения к объекту
 - **Adapter** — Изменяет интерфейс для совместимости
 - **Decorator** — Добавляет новые возможности объекту без изменения его кода
 - **Facade** — Упрощает доступ к сложной системе

🔗 Шаг 12:

Миссия 12

Почему прокси часто используется для логирования и кеширования?

Подсказка: Подумайте о месте, где можно перехватить все обращения.

Ответ:

- **Потому что прокси перехватывает все вызовы к объекту и может добавлять дополнительную логику**
- Потому что прокси ускоряет выполнение методов
- Потому что прокси создаёт копии объекта
- Потому что прокси заменяет интерфейс полностью

🔗 Шаг 13:

Миссия 13

Расположите шаги создания динамического прокси в правильном порядке:

1. Определение класса прокси

2. Переопределение `__getattr__` для перехвата обращений
3. Добавление логики (кеширование, проверка прав, логирование)
4. Передача вызова реальному объекту

Подсказка: Сначала создаём класс, затем перехватываем, добавляем логику, и только потом передаём вызовы.

Ответ:

- Определение класса прокси
- Переопределение `__getattr__` для перехвата обращений
- Добавление логики (кеширование, проверка прав, логирование)
- Передача вызова реальному объекту

Шаг 14:

Миссия 14

Что такое динамический прокси в Python?

Подсказка: Подумайте о том, как можно управлять доступом к объектам во время выполнения.

Ответ:

- **Объект, который управляет доступом к другому объекту во время выполнения**
- Класс, который заменяет все функции стандартной библиотеки
- Метод для ускорения работы программы
- Способ статической проверки типов

Шаг 15:

Миссия 15

Для чего чаще всего используют адаптеры в Python?

Подсказка: Подумайте о совмещении несовместимых интерфейсов.

Ответ:

- **Для того, чтобы интерфейсы несовместимых классов могли работать вместе**
- Для увеличения производительности кода
- Для автоматической генерации документации
- Для подключения к базе данных

Шаг 16:

Миссия 16

 Сопоставьте тип паттерна с его описанием.

Подсказка: Внимательно читайте определения — они различаются по цели использования.

Ответ:

- Прокси
 - Адаптер
 - Декоратор
 - Фасад
1. Обеспечивает контроль доступа к объекту
 2. Приводит интерфейс одного класса к интерфейсу другого
 3. Добавляет новую функциональность существующим объектам
 4. Упрощает работу с комплексной подсистемой

- **Прокси** — Обеспечивает контроль доступа к объекту
- **Адаптер** — Приводит интерфейс одного класса к интерфейсу другого
- **Декоратор** — Добавляет новую функциональность существующим объектам
- **Фасад** — Упрощает работу с комплексной подсистемой

Урок 4 - Практика runtime встраивание логов

Шаг 01:

Миссия 1

Почему программисты вставляют логирование во время выполнения (runtime)?

Подсказка: Подумайте, как можно следить за состоянием программы без остановки её работы.

Ответ:

- **Чтобы отслеживать поведение программы в реальном времени**
- Чтобы ускорить выполнение кода
- Чтобы автоматически исправлять ошибки
- Чтобы скрыть ошибки от пользователей

Шаг 02:

Миссия 2

Какой подход позволяет внедрять логи без изменения исходного кода функций?

Подсказка: Подумайте о динамических способах модификации поведения функций.

Ответ:

- **Использование декораторов или прокси для обёртывания функций**
- Переписывание каждой функции вручную
- Создание глобальной переменной для логов
- Использование условного оператора внутри каждой функции

Шаг 03:

Миссия 3

Что делает декоратор для runtime логирования?

Подсказка: Подумайте о добавлении функционала к функции без её изменения.

Ответ:

- **Оборачивает функцию и добавляет логирование при вызове**
- Удаляет ошибки из функции автоматически
- Создаёт новый класс вместо функции
- Изменяет порядок вызова функций в программе

Шаг 04:

Миссия 4

 Сопоставьте элементы runtime логирования с их назначением:

Подсказка: Каждый элемент выполняет уникальную роль.

Ответ:

- Декоратор
- Прокси

- Логгер
- Уровень логирования

1. Оборачивает функцию для добавления логики
2. Перехватывает вызовы объектов для внедрения логики
3. Отвечает за запись сообщений в журнал
4. Определяет важность и фильтрует сообщения

- **Декоратор** — Оборачивает функцию для добавления логики
- **Прокси** — Перехватывает вызовы объектов для внедрения логики
- **Логгер** — Отвечает за запись сообщений в журнал
- **Уровень логирования** — Определяет важность и фильтрует сообщения

Шаг 05:

Миссия 5

 Сопоставьте сценарии использования логирования с подходом:

Подсказка: Подумайте, где проще применить декоратор, а где прокси.

Ответ:

- Добавление логов в отдельные функции
 - Логирование всех методов объекта без изменения классов
 - Фильтрация сообщений по важности
 - Объединение логов из разных модулей
1. Декоратор
 2. Прокси
 3. Логгер с уровнями
 4. Сборщик логов
- **Добавление логов в отдельные функции** — Декоратор
 - **Логирование всех методов объекта без изменения классов** — Прокси
 - **Фильтрация сообщений по важности** — Логгер с уровнями
 - **Объединение логов из разных модулей** — Сборщик логов

Шаг 06:

Миссия 6

Что важно учитывать при runtime логировании в продакшн-системах?

Подсказка: Подумайте о производительности и объёме данных.

Ответ:

- **Не перегружать систему лишними логами и фильтровать уровни сообщений**
- Записывать все сообщения без фильтрации
- Прерывать выполнение программы для логирования
- Сохранять логи только в памяти без файла

Шаг 07:

Миссия 7

Какой модуль Python чаще всего используют для runtime логирования?

Подсказка: Подумайте о стандартной библиотеке, где есть Logger и уровни сообщений.

Ответ:

- **logging**
- os
- sys
- functools

Шаг 08:

Миссия 8

 Сопоставьте тип сообщения в логах с примером:

Подсказка: Внимательно смотрите на серьёзность события.

Ответ:

- DEBUG
 - INFO
 - WARNING
 - ERROR
1. Сообщение для разработки и отладки
 2. Сообщение о нормальной работе программы
 3. Предупреждение о потенциальной проблеме
 4. Ошибка, вызывающая сбой функции
- **DEBUG** — Сообщение для разработки и отладки
 - **INFO** — Сообщение о нормальной работе программы
 - **WARNING** — Предупреждение о потенциальной проблеме
 - **ERROR** — Ошибка, вызывающая сбой функции

Шаг 09:

Миссия 9

Расположите шаги внедрения логирования через декоратор:

1. Создать функцию-обёртку с логикой записи сообщений
2. Определить декоратор, принимающий оригинальную функцию
3. Возвращать обёрнутую функцию вместо оригинальной
4. Применить декоратор к функциям, которые нужно логировать

Подсказка: Подумайте о правильной последовательности: обёртка, декоратор, возврат, применение.

Ответ:

- Создать функцию-обёртку с логикой записи сообщений
- Определить декоратор, принимающий оригинальную функцию
- Возвращать обёрнутую функцию вместо оригинальной
- Применить декоратор к функциям, которые нужно логировать

Шаг 10:

Миссия 10

 Сопоставьте тип сообщения в логах с примером:

Подсказка: Внимательно смотрите на серьёзность события.

Ответ:

- DEBUG
- INFO
- WARNING
- ERROR

1. Сообщение для разработки и отладки
2. Сообщение о нормальной работе программы
3. Предупреждение о потенциальной проблеме
4. Ошибка, вызывающая сбой функции

- **DEBUG** — Сообщение для разработки и отладки
- **INFO** — Сообщение о нормальной работе программы
- **WARNING** — Предупреждение о потенциальной проблеме
- **ERROR** — Ошибка, вызывающая сбой функции

 Шаг 11:

Миссия 11

Расположите шаги внедрения логирования через декоратор:

1. Создать функцию-обёртку с логикой записи сообщений
2. Определить декоратор, принимающий оригинальную функцию
3. Возвращать обёрнутую функцию вместо оригинальной
4. Применить декоратор к функциям, которые нужно логировать

Подсказка: Подумайте о правильной последовательности: обёртка, декоратор, возврат, применение.

Ответ:

- Создать функцию-обёртку с логикой записи сообщений
- Определить декоратор, принимающий оригинальную функцию
- Возвращать обёрнутую функцию вместо оригинальной
- Применить декоратор к функциям, которые нужно логировать

 Шаг 12:

Миссия 12

Расположите этапы внедрения прокси для логирования методов объекта:

1. Создать класс-прокси, принимающий исходный объект
2. Переопределить методы для добавления логики
3. Возвращать результаты вызова оригинальных методов
4. Использовать прокси вместо оригинального объекта в коде

Подсказка: Прокси оборачивает объект, добавляет функционал и сохраняет поведение оригинала.

Ответ:

- Создать класс-прокси, принимающий исходный объект
- Переопределить методы для добавления логики
- Возвращать результаты вызова оригинальных методов
- Использовать прокси вместо оригинального объекта в коде

Шаг 13:

Миссия 13

Почему логирование через прокси может быть удобнее декоратора для классов?

Подсказка: Подумайте о множестве методов и необходимости менять код класса.

Ответ:

- **Можно логировать все методы без изменения исходного класса**
- Прокси ускоряет выполнение методов
- Прокси автоматически исправляет ошибки кода
- Прокси делает код короче и понятнее

Шаг 14:

Миссия 14

 Сопоставьте проблему логирования с подходом её решения:

Подсказка: Подумайте, какой инструмент подходит для каждой задачи.

Ответ:

- Логи нужны во всех методах объекта
 - Фильтровать только критические ошибки
 - Добавить логи для отдельной функции
 - Собирать логи из разных модулей в единый файл
1. Прокси
 2. Логгер с уровнями
 3. Декоратор
 4. Сборщик логов
- **Логи нужны во всех методах объекта** — Прокси
 - **Фильтровать только критические ошибки** — Логгер с уровнями
 - **Добавить логи для отдельной функции** — Декоратор
 - **Собирать логи из разных модулей в единый файл** — Сборщик логов

Шаг 15:

Миссия 15

Расположите шаги организации runtime логирования в продакшн:

1. Выбрать уровни логов для разных событий
2. Определить способ внедрения логики (декоратор/прокси)
3. Настроить сбор и хранение сообщений
4. Применить логирование к ключевым точкам кода и протестировать

Подсказка: Сначала планируем что и как логировать, затем внедряем и проверяем.

Ответ:

- Выбрать уровни логов для разных событий
- Определить способ внедрения логики (декоратор/прокси)
- Настроить сбор и хранение сообщений
- Применить логирование к ключевым точкам кода и протестировать

Урок 5 - Лабиринт гибридные метаклассы

Шаг 01:

Миссия 1

Что такое метакласс в Python?

Подсказка: Подумайте о классе, который создаёт другие классы.

Ответ:

- **Класс, который создаёт и контролирует другие классы**
- Объект, который управляет данными программы
- Функция для оптимизации кода на лету
- Интерфейс для работы с базой данных

Шаг 02:

Миссия 2

Для чего используют гибридные метаклассы?

Подсказка: Подумайте о классах с несколькими уровнями контроля.

Ответ:

- **Для объединения функциональности нескольких метаклассов в одном классе**
- Для ускорения выполнения обычных функций
- Для работы только с базами данных
- Для автоматического тестирования кода

Шаг 03:

Миссия 3

 Сопоставьте метакласс с его описанием.

Подсказка: Обратите внимание, какой класс создаёт другие классы, а какой изменяет их поведение.

Ответ:

- type
 - ABCMeta
 - SingletonMeta
 - LoggerMeta
1. Стандартный метакласс для создания классов
 2. Метакласс для реализации паттерна «одиночка»
 3. Метакласс для создания абстрактных базовых классов
 4. Метакласс для автоматической вставки логирования
- **type** — Стандартный метакласс для создания классов
 - **ABCMeta** — Метакласс для создания абстрактных базовых классов
 - **SingletonMeta** — Метакласс для реализации паттерна «одиночка»
 - **LoggerMeta** — Метакласс для автоматической вставки логирования

Шаг 04:

Миссия 4

 Сопоставьте ситуацию с подходящим использованием гибридного метакласса.

Подсказка: Думайте о комбинации разных требований в одном классе.

Ответ:

- Объект должен быть логируемым и уникальным
 - Нужен класс с проверкой типов и абстрактными методами
 - Создание классов с дополнительными атрибутами для всех экземпляров
 - Управление доступом к ресурсам и автоматическое логирование
1. SingletonLoggerMeta
 2. CheckedABCMeta
 3. AttributeInjectorMeta
 4. AccessLoggerMeta
- **Объект должен быть логируемым и уникальным** — SingletonLoggerMeta
 - **Нужен класс с проверкой типов и абстрактными методами** — CheckedABCMeta
 - **Создание классов с дополнительными атрибутами для всех экземпляров** — AttributeInjectorMeta
 - **Управление доступом к ресурсам и автоматическое логирование** — AccessLoggerMeta

Шаг 05:

Миссия 5

Что происходит, когда создаётся класс с гибридным метаклассом?

Подсказка: Подумайте о последовательности применения разных метаклассов.

Ответ:

- **Метаклассы применяются последовательно, комбинируя свои эффекты при создании класса**
- Все метаклассы игнорируются, создаётся обычный класс
- Создаётся только первый указанный метакласс
- Метакласс создаёт новый процесс в системе

Шаг 06:

Миссия 6

Почему важно тестировать классы с гибридными метаклассами?

Подсказка: Подумайте, какие необычные ошибки могут возникнуть при комбинировании метаклассов.

Ответ:

- **Чтобы убедиться, что все эффекты разных метаклассов правильно сочетаются**
- Чтобы ускорить работу кода
- Чтобы автоматически документировать класс
- Чтобы обойти проверку типов

Шаг 07:

Миссия 7

 Сопоставьте ошибки при использовании гибридных метаклассов с их причиной.

Подсказка: Обратите внимание на порядок наследования и конфликты методов.

Ответ:

- TypeError: metaclass conflict

- Неправильное логирование событий
- Неправильная проверка абстрактных методов
- Дублирующиеся атрибуты у экземпляров

1. Разные метаклассы имеют несовместимый базовый метакласс
2. Методы логирования перекрываются
3. Метакласс не применён к ABC
4. Метакласс добавляет одинаковые атрибуты несколько раз

- **TypeError: metaclass conflict** — Разные метаклассы имеют несовместимый базовый метакласс
- **Неправильное логирование событий** — Методы логирования перекрываются
- **Неправильная проверка абстрактных методов** — Метакласс не применён к ABC
- **Дублирующиеся атрибуты у экземпляров** — Метакласс добавляет одинаковые атрибуты несколько раз

🔮 Шаг 08:

Миссия 8

Расположите шаги создания класса с гибридным метаклассом в правильном порядке:

1. Определение базовых метаклассов
2. Создание гибридного метакласса через множественное наследование
3. Создание нового класса с указанием гибридного метакласса
4. Тестирование поведения класса и комбинации метаклассов

Подсказка: Сначала создаём метаклассы, потом комбинируем, затем используем и тестируем.

Ответ:

- Определение базовых метаклассов
- Создание гибридного метакласса через множественное наследование
- Создание нового класса с указанием гибридного метакласса
- Тестирование поведения класса и комбинации метаклассов

🔮 Шаг 09:

Миссия 9

Расположите шаги создания класса с гибридным метаклассом в правильном порядке:

1. Определение базовых метаклассов
2. Создание гибридного метакласса через множественное наследование
3. Создание нового класса с указанием гибридного метакласса
4. Тестирование поведения класса и комбинации метаклассов

Подсказка: Сначала создаём метаклассы, потом комбинируем, затем используем и тестируем.

Ответ:

- Определение базовых метаклассов
- Создание гибридного метакласса через множественное наследование
- Создание нового класса с указанием гибридного метакласса
- Тестирование поведения класса и комбинации метаклассов

🔮 Шаг 10:

Миссия 10

Как можно динамически изменять методы класса через метакласс?

Подсказка: Подумайте о добавлении или замене методов во время создания класса.

Ответ:

- Путём изменения словаря атрибутов класса в `__new__` или `__init__` метакласса
- Только через декораторы функций
- С помощью глобальных переменных
- Только после создания экземпляра класса

Шаг 11:

Миссия 11

 Сопоставьте метод метакласса с его назначением.

Подсказка: Подумайте, какой метод отвечает за создание класса, а какой — за инициализацию.

Ответ:

- `__new__`
 - `__init__`
 - `__call__`
 - `__prepare__`
1. Создаёт класс перед его инициализацией
 2. Инициализирует класс после создания
 3. Вызывает класс как функцию для создания экземпляров
 4. Определяет словарь атрибутов перед созданием класса
- `__new__` — Создаёт класс перед его инициализацией
 - `__init__` — Инициализирует класс после создания
 - `__call__` — Вызывает класс как функцию для создания экземпляров
 - `__prepare__` — Определяет словарь атрибутов перед созданием класса

Шаг 12:

Миссия 12

Что такое динамический прокси-класс?

Подсказка: Подумайте о классе, который управляет доступом к другому объекту.

Ответ:

- Класс, который перехватывает обращения к объекту и добавляет дополнительное поведение
- Простой декоратор функции
- Новый тип исключений
- Класс без методов и атрибутов

Шаг 13:

Миссия 13

 Сопоставьте тип гибридного метакласса с его примером применения.

Подсказка: Подумайте, какой метакласс комбинирует свойства нескольких базовых метаклассов.

Ответ:

- Singleton + Logger

- ABC + Checked
- AttributeInjector + Proxy
- Access + Logger

1. Создание уникального логируемого объекта
2. Абстрактные методы с проверкой типов
3. Добавление атрибутов и управление доступом через прокси
4. Контроль доступа с логированием событий

- **Singleton + Logger** — Создание уникального логируемого объекта
- **ABC + Checked** — Абстрактные методы с проверкой типов
- **AttributeInjector + Proxy** — Добавление атрибутов и управление доступом через прокси
- **Access + Logger** — Контроль доступа с логированием событий

🔗 Шаг 14:

Миссия 14

Почему метаклассы полезны при построении фреймворков?

Подсказка: Подумайте о создании однотипных классов с контролем и проверками.

Ответ:

- **Они позволяют автоматически добавлять поведение, проверку и атрибуты ко всем классам фреймворка**
- Чтобы ускорить работу функций на 100%
- Чтобы заменить обычные классы полностью
- Чтобы создавать GUI интерфейсы

🔗 Шаг 15:

Миссия 15

📄 Сопоставьте метакласс с типом поведения, которое он внедряет.

Подсказка: Обратите внимание на паттерны проектирования и добавляемое поведение.

Ответ:

- SingletonMeta
 - LoggerMeta
 - AttributeInjectorMeta
 - CheckedABCMeta
1. Ограничение числа экземпляров одного класса
 2. Автоматическое логирование вызовов методов
 3. Добавление атрибутов ко всем экземплярам класса
 4. Проверка типов и абстрактных методов
- **SingletonMeta** — Ограничение числа экземпляров одного класса
 - **LoggerMeta** — Автоматическое логирование вызовов методов
 - **AttributeInjectorMeta** — Добавление атрибутов ко всем экземплярам класса
 - **CheckedABCMeta** — Проверка типов и абстрактных методов

🔗 Шаг 16:

Миссия 16

Расположите шаги от идеи до полноценного гибридного метакласса:

1. Определение потребностей класса
2. Выбор базовых метаклассов для комбинирования
3. Создание гибридного метакласса
4. Применение к реальному классу и тестирование

Подсказка: Сначала анализируем, потом комбинируем, создаём и проверяем.

Ответ:

- Определение потребностей класса
- Выбор базовых метаклассов для комбинирования
- Создание гибридного метакласса
- Применение к реальному классу и тестирование

Шаг 17:

Миссия 17

Что нужно учитывать при проектировании гибридных метаклассов?

Подсказка: Подумайте о совместимости и порядке применения метаклассов.

Ответ:

- **Совместимость базовых метаклассов и порядок их применения**
- Скорость компиляции Python
- Возможность менять глобальные переменные
- Внешний вид IDE

Шаг 18:

Миссия 18

 Сопоставьте пример кода с объяснением, что делает метакласс.

Подсказка: Обратите внимание на добавляемые методы и проверку атрибутов.

Ответ:

- `class MyMeta(type): ...`
 - `class LoggedSingletonMeta(type): ...`
 - `class ABCCheckedMeta(type): ...`
 - `class AttrProxyMeta(type): ...`
1. Создаёт обычный класс с кастомными методами
 2. Ограничивает экземпляры и добавляет логирование
 3. Абстрактные методы с проверкой типов
 4. Добавляет атрибуты и проксирует доступ
- **`class MyMeta(type): ...`** — Создаёт обычный класс с кастомными методами
 - **`class LoggedSingletonMeta(type): ...`** — Ограничивает экземпляры и добавляет логирование
 - **`class ABCCheckedMeta(type): ...`** — Абстрактные методы с проверкой типов
 - **`class AttrProxyMeta(type): ...`** — Добавляет атрибуты и проксирует доступ

Модуль 5 - Артефакты самоизменяющегося кода ✨

Урок 1 - Генерация Python-файлов автоматически

Шаг 01:

Миссия 1

Вы решили автоматически сгенерировать Python-файл с функцией приветствия. Какой подход будет самым простым для старта?

Подсказка: Начните с базового способа создания файла и записи текста.

Ответ:

- **Открыть файл в режиме 'w' и записать строку с кодом функции**
- Запустить интерпретатор Python и ждать генерации автоматически
- Создать бинарный файл без расширения
- Использовать стороннюю программу для генерации без Python

Шаг 02:

Миссия 2

Вы хотите добавить комментарий в автоматически сгенерированный Python-код. Как правильно это сделать?

Подсказка: Комментарии помогают другим понимать ваш код.

Ответ:

- **Добавить строку с # перед кодом комментария**
- Использовать спецсимволы \$, чтобы создать комментарий
- Закомментировать весь файл с помощью /* */
- Комментарии писать в отдельный текстовый файл

Шаг 03:

Миссия 3

Вы хотите автоматически генерировать несколько функций в одном файле. Какой метод наиболее удобен?

Подсказка: Подумайте о цикле или шаблоне для повторяющихся частей.

Ответ:

- **Использовать цикл для записи строк с функциями в файл**
- Каждую функцию писать вручную
- Генерировать бинарные функции напрямую
- Сначала сохранить функции в Excel, потом копировать

Шаг 04:

Миссия 4

 Сопоставьте метод Python с его назначением при генерации файлов

Методы:

Ответ:

- open()
- write()
- close()

- with

Назначение:

1. Закрывает файл после работы с ним
2. Открывает файл для записи или чтения
3. Автоматически управляет ресурсами файла
4. Записывает строку в файл

Подсказка:

Сначала файл открывается, потом с ним работают, и в конце закрывают. Использование with упрощает процесс.

- **open()** — Открывает файл для записи или чтения
- **write()** — Записывает строку в файл
- **close()** — Закрывает файл после работы с ним
- **with** — Автоматически управляет ресурсами файла

Шаг 05:

Миссия 5

 Сопоставьте концепцию генерации кода с её примером

Концепции:

Ответ:

- Шаблонизация
- Метапрограммирование
- Автоматическая документация
- Динамическая генерация функций

Примеры:

1. Создание функций на основе конфигурации
2. Использование строк и шаблонов для кода
3. Генерация README и docstring автоматически
4. Код, который пишет код во время выполнения

Подсказка:

Обратите внимание на различие между шаблонами кода, генерацией документации и метакодом.

- **Шаблонизация** — Использование строк и шаблонов для кода
- **Метапрограммирование** — Код, который пишет код во время выполнения
- **Автоматическая документация** — Генерация README и docstring автоматически
- **Динамическая генерация функций** — Создание функций на основе конфигурации

Шаг 06:

Миссия 6

 Сопоставьте тип ошибки при генерации кода с её причиной

Типы ошибок:

Ответ:

- SyntaxError
- IndentationError

- `NameError`
- `TypeError`

Причины:

1. Неправильный отступ в коде
2. Использование переменной до её определения
3. Неверный тип данных в операции
4. Ошибка в структуре Python-кода

Подсказка:

Сначала подумайте, какие ошибки возникают при генерации текста кода, а какие — при исполнении.

- **`SyntaxError`** — Ошибка в структуре Python-кода
- **`IndentationError`** — Неправильный отступ в коде
- **`NameError`** — Использование переменной до её определения
- **`TypeError`** — Неверный тип данных в операции

Шаг 07:

Миссия 7

 Сопоставьте инструмент Python с его ролью при генерации кода

Инструменты:

Ответ:

- `ast`
- `exec()`
- `jinja2`
- `inspect`

Роль:

1. Позволяет создавать код из шаблонов
2. Выполняет строку Python-кода как программу
3. Анализирует и модифицирует синтаксическое дерево Python
4. Получает исходный код функций и объектов

Подсказка:

Обратите внимание на разницу между анализом кода, шаблонами и динамическим исполнением.

- **`ast`** — Анализирует и модифицирует синтаксическое дерево Python
- **`exec()`** — Выполняет строку Python-кода как программу
- **`jinja2`** — Позволяет создавать код из шаблонов
- **`inspect`** — Получает исходный код функций и объектов

Шаг 08:

Миссия 8

Расположите шаги генерации Python-файла в правильном порядке:

1. Создать шаблон кода
2. Открыть файл для записи
3. Записать код в файл

4. Закрыть файл или выйти из блока with

 Подсказка:

Подумайте, сначала подготавливается код, потом работа с файлом, и в конце — закрытие.

Ответ:

- Создать шаблон кода
- Открыть файл для записи
- Записать код в файл
- Закрыть файл или выйти из блока with

 Шаг 09:

Миссия 9

Расположите этапы метапрограммирования для генерации функций:

1. Определение параметров для функций
2. Создание строки кода с функцией
3. Исполнение кода с помощью `exec()`
4. Вызов сгенерированной функции

 Подсказка:

Сначала задаются параметры, затем формируется код, выполняется и только потом используется.

Ответ:

- Определение параметров для функций
- Создание строки кода с функцией
- Исполнение кода с помощью `exec()`
- Вызов сгенерированной функции

 Шаг 10:

Миссия 10

Расположите шаги от генерации шаблона до документации:

1. Создание шаблона функции
2. Генерация кода на основе шаблона
3. Добавление `docstring` и комментариев
4. Сохранение и тестирование файла

 Подсказка:

Документация добавляется после генерации кода, перед сохранением и тестированием.

Ответ:

- Создание шаблона функции
- Генерация кода на основе шаблона
- Добавление `docstring` и комментариев
- Сохранение и тестирование файла

 Шаг 11:

Миссия 11

Расположите этапы динамической генерации функций с `exec()`:

1. Создать строку кода функции
2. Выполнить `exec()` для создания функции
3. Передать аргументы и вызвать функцию
4. Использовать результат в программе

 Подсказка:

Сначала формируется код, потом создается функция, затем вызывается и используется.

Ответ:

- Создать строку кода функции
- Выполнить `exec()` для создания функции
- Передать аргументы и вызвать функцию
- Использовать результат в программе

 Шаг 12:

Миссия 12

Расположите шаги от анализа AST до изменения кода:

1. Получить дерево AST с помощью `ast.parse()`
2. Проанализировать узлы дерева
3. Внести изменения в узлы
4. Сгенерировать новый Python-код с `ast.unparse()`

 Подсказка:

AST позволяет изменять код на структурном уровне: сначала анализ, потом изменение, затем генерация нового кода.

Ответ:

- Получить дерево AST с помощью `ast.parse()`
- Проанализировать узлы дерева
- Внести изменения в узлы
- Сгенерировать новый Python-код с `ast.unparse()`

 Шаг 13:

Миссия 13

Расположите шаги тестирования автоматически сгенерированного кода:

1. Создать тесты для функций
2. Выполнить сгенерированные функции
3. Сравнить результаты с ожидаемыми
4. Исправить ошибки в шаблоне или генерации

 Подсказка:

Тестирование начинается с проверки функций и заканчивается исправлением шаблонов.

Ответ:

- Создать тесты для функций
- Выполнить сгенерированные функции
- Сравнить результаты с ожидаемыми
- Исправить ошибки в шаблоне или генерации

🔮 Шаг 14:

Миссия 14

Расположите шаги организации проекта с автогенерацией кода:

1. Разделить код на модули и шаблоны
2. Создать утилиту генерации кода
3. Добавить тесты и документацию
4. Автоматизировать процесс сборки и запуска генерации

🔮 Подсказка:

Сначала проект структурируется, затем создаётся генератор, добавляются тесты и автоматизация.

Ответ:

- Разделить код на модули и шаблоны
- Создать утилиту генерации кода
- Добавить тесты и документацию
- Автоматизировать процесс сборки и запуска генерации

🔮 Шаг 15:

Миссия 15

Расположите этапы безопасного выполнения сгенерированного кода:

1. Проверить код на синтаксические ошибки
2. Ограничить доступные функции и модули
3. Выполнить код в изолированном окружении
4. Собрать результаты выполнения и обработать ошибки

🔮 Подсказка:

Безопасность важна: сначала проверка синтаксиса, потом изоляция и обработка результатов.

Ответ:

- Проверить код на синтаксические ошибки
- Ограничить доступные функции и модули
- Выполнить код в изолированном окружении
- Собрать результаты выполнения и обработать ошибки

🔮 Шаг 16:

Миссия 16

Расположите шаги интеграции автогенерации кода в CI/CD:

1. Создать скрипт генерации кода
2. Добавить генерацию в пайплайн сборки
3. Запустить автоматические тесты для сгенерированного кода
4. Развернуть и проверить корректность работы в окружении

🔮 Подсказка:

Сначала создаётся генератор, затем интеграция в сборку, тестирование и деплой.

Ответ:

- Создать скрипт генерации кода

- Добавить генерацию в пайплайн сборки
- Запустить автоматические тесты для сгенерированного кода
- Развернуть и проверить корректность работы в окружении

🔧 Шаг 17:

Миссия 17

Расположите шаги от идеи до публикации Python-скрипта, написанного Python-кодом:

1. Сформулировать задачу и определить шаблон
2. Сгенерировать Python-код автоматически
3. Протестировать и исправить ошибки
4. Опубликовать и поделиться готовым скриптом

🔧 Подсказка:

Всё начинается с идеи, затем генерация, проверка и финальная публикация.

Ответ:

- Сформулировать задачу и определить шаблон
- Сгенерировать Python-код автоматически
- Протестировать и исправить ошибки
- Опубликовать и поделиться готовым скриптом

📖 Урок 2 - AST как средство оптимизации кода

🔧 Шаг 01:

Миссия 1

Вы хотите оптимизировать Python-код с помощью AST. Что будет первым шагом?

Подсказка: Сначала нужно получить структуру кода для анализа.

Ответ:

- **Использовать `ast.parse()` для преобразования кода в дерево**
- Сразу переписать код вручную
- Добавить `print()` в каждом месте кода
- Запустить интерпретатор с флагом `-O`

🔧 Шаг 02:

Миссия 2

Как с помощью AST можно найти повторяющиеся выражения в коде?

Подсказка: Ищите узлы, которые имеют одинаковую структуру и содержимое.

Ответ:

- **Проанализировать узлы AST и сравнить их типы и значения**
- Сравнить текст кода как строки
- Использовать только линтеры без AST
- Генерировать случайные тесты

🔧 Шаг 03:

Миссия 3

Вы хотите изменить поведение функции без редактирования исходного текста напрямую. Как AST помогает это сделать?

Подсказка: Думайте о структуре кода как о дереве, а не о строках.

Ответ:

- **Изменить соответствующие узлы AST и сгенерировать новый код**
- Добавить комментарий и ожидать изменения поведения
- Менять только имена переменных вручную
- Скомпилировать код без изменений

Шаг 04:

Миссия 4

 Сопоставьте узлы AST с их назначением при оптимизации кода

Узлы:

Ответ:

- `ast.FunctionDef`
- `ast.Assign`
- `ast.If`
- `ast.Call`

Назначение:

1. Определение функции с именем, аргументами и телом
2. Присвоение значения переменной
3. Ветвление кода по условию
4. Вызов функции с аргументами

Подсказка:

AST отражает структуру Python-кода, каждый тип узла выполняет свою роль.

- **`ast.FunctionDef`** — Определение функции с именем, аргументами и телом
- **`ast.Assign`** — Присвоение значения переменной
- **`ast.If`** — Ветвление кода по условию
- **`ast.Call`** — Вызов функции с аргументами

Шаг 05:

Миссия 5

 Сопоставьте методы AST с их использованием при оптимизации кода

Методы:

Ответ:

- `ast.parse()`
- `ast.dump()`
- `ast.walk()`
- `ast.unparse()`

Использование:

1. Преобразует Python-код в дерево AST
2. Выводит структуру AST в виде строки для анализа
3. Проходит по всем узлам AST
4. Генерирует исходный код Python из дерева AST

Подсказка:

Думайте о AST как о дереве: сначала строим дерево, потом исследуем и модифицируем его, затем преобразуем обратно в код.

- **ast.parse()** — Преобразует Python-код в дерево AST
- **ast.dump()** — Выводит структуру AST в виде строки для анализа
- **ast.walk()** — Проходит по всем узлам AST
- **ast.unparse()** — Генерирует исходный код Python из дерева AST

🔧 Шаг 06:

Миссия 6

📖 Сопоставьте методы AST с их задачами при оптимизации кода

Методы:

Ответ:

- `ast.iter_child_nodes()`
- `ast.NodeTransformer`
- `ast.copy_location`
- `ast.fix_missing_locations`

Назначение:

1. Проход по дочерним узлам AST
2. Создание трансформера для изменения AST
3. Копирование позиции узла для сохранения исходной структуры
4. Корректировка информации о позициях после модификаций

🔧 Подсказка:

При оптимизации важно не потерять информацию о позициях и уметь обходить и трансформировать дерево.

- **ast.iter_child_nodes()** — Проход по дочерним узлам AST
- **ast.NodeTransformer** — Создание трансформера для изменения AST
- **ast.copy_location** — Копирование позиции узла для сохранения исходной структуры
- **ast.fix_missing_locations** — Корректировка информации о позициях после модификаций

🔧 Шаг 07:

Миссия 7

Как лучше всего использовать AST для удаления неиспользуемых переменных?

Подсказка: Сначала найдите все объявления, затем проверьте использование.

Ответ:

- **Собрать все узлы Assign и проверить, где переменные не используются, затем удалить**
- Удалять переменные вручную по имени
- Игнорировать переменные и использовать только функции
- Добавлять комментарии вместо удаления

🔧 Шаг 08:

Миссия 8

Вы хотите заменить все вызовы `print` на логгер в проекте. Как это сделать через AST?

Подсказка: Узнайте, какие узлы отвечают за вызовы функций.

Ответ:

- **Использовать NodeTransformer, изменить узлы Call с функцией print на logger**
- Заменять строки в исходном коде через replace
- Удалять все print без замены
- Добавить if перед каждым print

 Шаг 09:

Миссия 9

Расположите шаги оптимизации функции с AST:

1. Парсинг кода в AST
2. Обход узлов и анализ повторяющихся конструкций
3. Замена узлов на оптимальные конструкции
4. Генерация нового исходного кода

 Подсказка:

Сначала строим дерево, потом анализируем, трансформируем и создаём новый код.

Ответ:

- Парсинг кода в AST
- Обход узлов и анализ повторяющихся конструкций
- Замена узлов на оптимальные конструкции
- Генерация нового исходного кода

 Шаг 10:

Миссия 10

Расположите шаги применения AST для автоматической замены циклов на генераторы:

1. Определение узлов For и While
2. Анализ тела цикла и зависимостей
3. Замена на генераторные выражения или comprehensions
4. Генерация и тестирование нового кода

 Подсказка:

Сначала нужно найти циклы, затем понять, можно ли их заменить, потом провести замену и проверить результат.

Ответ:

- Определение узлов For и While
- Анализ тела цикла и зависимостей
- Замена на генераторные выражения или comprehensions
- Генерация и тестирование нового кода

 Шаг 11:

Миссия 11

Расположите шаги безопасного выполнения сгенерированного AST-кода:

1. Проверка синтаксиса AST
2. Изоляция выполнения в sandbox
3. Выполнение и сбор результата

4. Обработка возможных исключений

Подсказка:

Безопасность важна: сначала проверка структуры, потом изоляция, выполнение и обработка ошибок.

Ответ:

- Проверка синтаксиса AST
- Изоляция выполнения в sandbox
- Выполнение и сбор результата
- Обработка возможных исключений

Шаг 12:

Миссия 12

Расположите шаги профилирования кода через AST:

1. Сбор узлов, содержащих вызовы функций
2. Вставка счетчиков или таймеров через NodeTransformer
3. Генерация нового кода с измерениями
4. Сбор статистики выполнения и анализ узких мест

Подсказка:

Сначала идентифицируем вызовы, затем вставляем измерения, генерируем код и анализируем результаты.

Ответ:

- Сбор узлов, содержащих вызовы функций
- Вставка счетчиков или таймеров через NodeTransformer
- Генерация нового кода с измерениями
- Сбор статистики выполнения и анализ узких мест

Шаг 13:

Миссия 13

Расположите шаги интеграции AST в CI/CD:

1. Создание AST-трансформеров для проекта
2. Автоматическая генерация измененного кода при сборке
3. Запуск тестов на сгенерированном коде
4. Развертывание и мониторинг работы

Подсказка:

Сначала пишем трансформеры, потом интегрируем генерацию в сборку, тестируем и деплоим.

Ответ:

- Создание AST-трансформеров для проекта
- Автоматическая генерация измененного кода при сборке
- Запуск тестов на сгенерированном коде
- Развертывание и мониторинг работы

Шаг 14:

Миссия 14

Расположите шаги от идеи до публикации Python-кода, оптимизированного через AST:

1. Сформулировать задачу и определить узлы для оптимизации
2. Создать и применить AST-трансформеры
3. Генерировать и тестировать новый код
4. Опубликовать готовый код и поделиться с командой

 Подсказка:

Всё начинается с идеи, затем трансформация, тестирование и публикация.

Ответ:

- Сформулировать задачу и определить узлы для оптимизации
- Создать и применить AST-трансформеры
- Генерировать и тестировать новый код
- Опубликовать готовый код и поделиться с командой

 Шаг 15:

Миссия 15

Расположите этапы автоматической документации через AST:

1. Сбор узлов функций и классов
2. Генерация docstring на основе кода и аннотаций
3. Вставка docstring в AST
4. Генерация обновленного исходного кода с документацией

 Подсказка:

Сначала собираем структуру кода, создаем docstring, вставляем и генерируем новый код.

Ответ:

- Сбор узлов функций и классов
- Генерация docstring на основе кода и аннотаций
- Вставка docstring в AST
- Генерация обновленного исходного кода с документацией

 Шаг 16:

Миссия 16

Расположите шаги от анализа AST до публикации финальной версии:

1. Парсинг исходного кода в AST
2. Оптимизация и трансформация узлов
3. Генерация финального кода и тестирование
4. Публикация и совместное использование с командой

 Подсказка:

Сначала строим AST, затем трансформируем, тестируем и публикуем.

Ответ:

- Парсинг исходного кода в AST
- Оптимизация и трансформация узлов
- Генерация финального кода и тестирование
- Публикация и совместное использование с командой

Урок 3 - Генерация DSL на Python

Шаг 01:

Миссия 1

Что позволяет модуль ast в Python?

Подсказка: Подумайте о том, как можно представлять код как структуру.

Ответ:

- **Представлять исходный код в виде дерева синтаксических узлов**
- Компилировать код в байткод
- Запускать Python код параллельно
- Создавать GUI приложения

Шаг 02:

Миссия 2

Какая задача наиболее подходит для использования AST?

Подсказка: Подумайте, когда нужно анализировать или менять код автоматически.

Ответ:

- **Автоматическая оптимизация или трансформация кода**
- Отладка ошибок в runtime
- Создание веб-сайта
- Управление файлами операционной системы

Шаг 03:

Миссия 3

Что позволяет NodeTransformer в модуле ast?

Подсказка: Подумайте о том, как изменять узлы дерева синтаксиса.

Ответ:

- **Изменять узлы AST во время обхода**
- Выполнять Python код напрямую
- Сохранять код в файл CSV
- Создавать графический интерфейс

Шаг 04:

Миссия 4

Что нужно для безопасного выполнения сгенерированного кода через AST?

Подсказка: Безопасность важна, чтобы код не причинил вреда системе.

Ответ:

- **Использовать sandbox или ограниченное пространство выполнения**
- Выполнять код как суперпользователь
- Игнорировать ошибки безопасности
- Перезапускать компьютер после выполнения

Шаг 05:

Миссия 5

 Сопоставьте узлы AST с их назначением

Узлы:

Ответ:

- `ast.BinOp`
- `ast.Call`
- `ast.Assign`
- `ast.FunctionDef`

Назначение:

1. Операция между двумя значениями
2. Вызов функции
3. Присвоение значения переменной
4. Определение функции

 **Подсказка:**

Думайте о роли каждого узла: операции, вызовы, присвоения, определения функций.

- **`ast.BinOp`** — Операция между двумя значениями
- **`ast.Call`** — Вызов функции
- **`ast.Assign`** — Присвоение значения переменной
- **`ast.FunctionDef`** — Определение функции

 **Шаг 06:**

Миссия 6

 Сопоставьте методы AST с их задачами при оптимизации кода

Методы:

Ответ:

- `ast.iter_child_nodes()`
- `ast.NodeTransformer`
- `ast.copy_location`
- `ast.fix_missing_locations`

Назначение:

1. Проход по дочерним узлам AST
2. Создание трансформера для изменения AST
3. Копирование позиции узла для сохранения исходной структуры
4. Корректировка информации о позициях после модификаций

 **Подсказка:**

При оптимизации важно не потерять информацию о позициях и уметь обходить и трансформировать дерево.

- **`ast.iter_child_nodes()`** — Проход по дочерним узлам AST
- **`ast.NodeTransformer`** — Создание трансформера для изменения AST
- **`ast.copy_location`** — Копирование позиции узла для сохранения исходной структуры
- **`ast.fix_missing_locations`** — Корректировка информации о позициях после модификаций

 **Шаг 07:**

Миссия 7

 Сопоставьте типы узлов AST с примерами кода

Узлы:

Ответ:

- ast.List
- ast.Dict
- ast.While
- ast.BinOp

Примеры:

1. [1,2,3]
2. {"a":1}
3. while x<10: pass
4. x+y

 **Подсказка:**

Операции, коллекции и циклы соответствуют разным типам узлов.

- **ast.List** — [1,2,3]
- **ast.Dict** — {"a":1}
- **ast.While** — while x<10: pass
- **ast.BinOp** — x+y

 **Шаг 08:**

Миссия 8

Как лучше всего использовать AST для удаления неиспользуемых переменных?

Подсказка: Сначала найдите все объявления, затем проверьте использование.

Ответ:

- **Собрать все узлы Assign и проверить, где переменные не используются, затем удалить**
- Удалять переменные вручную по имени
- Игнорировать переменные и использовать только функции
- Добавлять комментарии вместо удаления

 **Шаг 09:**

Миссия 9

Что позволяет модуль ast в Python?

Подсказка: Подумайте о том, как можно представлять код как структуру.

Ответ:

- **Представлять исходный код в виде дерева синтаксических узлов**
- Компилировать код в байткод
- Запускать Python код параллельно
- Создавать GUI приложения

 **Шаг 10:**

Миссия 10

Какая задача наиболее подходит для использования AST?

Подсказка: Подумайте, когда нужно анализировать или менять код автоматически.

Ответ:

- **Автоматическая оптимизация или трансформация кода**
- Отладка ошибок в runtime
- Создание веб-сайта
- Управление файлами операционной системы

🔮 **Шаг 11:**

Миссия 11

Что позволяет NodeTransformer в модуле ast?

Подсказка: Подумайте о том, как изменять узлы дерева синтаксиса.

Ответ:

- **Изменять узлы AST во время обхода**
- Выполнять Python код напрямую
- Сохранять код в файл CSV
- Создавать графический интерфейс

🔮 **Шаг 12:**

Миссия 12

Что нужно для безопасного выполнения сгенерированного кода через AST?

Подсказка: Безопасность важна, чтобы код не причинил вреда системе.

Ответ:

- **Использовать sandbox или ограниченное пространство выполнения**
- Выполнять код как суперпользователь
- Игнорировать ошибки безопасности
- Перезапускать компьютер после выполнения

🔮 **Шаг 13:**

Миссия 13

📄 Сопоставьте узлы AST с их назначением

Узлы:

Ответ:

- ast.BinOp
- ast.Call
- ast.Assign
- ast.FunctionDef

Назначение:

1. Операция между двумя значениями
2. Вызов функции
3. Присвоение значения переменной
4. Определение функции

🔮 **Подсказка:**

Думайте о роли каждого узла: операции, вызовы, присвоения, определения функций.

- **ast.BinOp** — Операция между двумя значениями

- **ast.Call** — Вызов функции
- **ast.Assign** — Присвоение значения переменной
- **ast.FunctionDef** — Определение функции

🔗 Шаг 14:

Миссия 14

📖 Сопоставьте методы AST с их задачами при оптимизации кода

Методы:

Ответ:

- `ast.iter_child_nodes()`
- `ast.NodeTransformer`
- `ast.copy_location`
- `ast.fix_missing_locations`

Назначение:

1. Проход по дочерним узлам AST
2. Создание трансформера для изменения AST
3. Копирование позиции узла для сохранения исходной структуры
4. Корректировка информации о позициях после модификаций

🔗 Подсказка:

При оптимизации важно не потерять информацию о позициях и уметь обходить и трансформировать дерево.

- **ast.iter_child_nodes()** — Проход по дочерним узлам AST
- **ast.NodeTransformer** — Создание трансформера для изменения AST
- **ast.copy_location** — Копирование позиции узла для сохранения исходной структуры
- **ast.fix_missing_locations** — Корректировка информации о позициях после модификаций

🔗 Шаг 15:

Миссия 15

📖 Сопоставьте типы узлов AST с примерами кода

Узлы:

Ответ:

- `ast.List`
- `ast.Dict`
- `ast.While`
- `ast.BinOp`

Примеры:

1. `[1,2,3]`
2. `{"a":1}`
3. `while x<10: pass`
4. `x+y`

🔗 Подсказка:

Операции, коллекции и циклы соответствуют разным типам узлов.

- **ast.List** — `[1,2,3]`

- **ast.Dict** — {"a":1}
- **ast.While** — while x<10: pass
- **ast.BinOp** — x+y

🔧 Шаг 16:

Миссия 16

Как лучше всего использовать AST для удаления неиспользуемых переменных?

Подсказка: Сначала найдите все объявления, затем проверьте использование.

Ответ:

- **Собрать все узлы Assign и проверить, где переменные не используются, затем удалить**
- Удалять переменные вручную по имени
- Игнорировать переменные и использовать только функции
- Добавлять комментарии вместо удаления

🔧 Шаг 17:

Миссия 17

📄 Сопоставьте типы ошибок при работе с AST с их причиной

Ошибки:

Ответ:

- `SyntaxError`
- `TypeError`
- `ValueError`
- `AttributeError`

Причины:

1. Неправильный синтаксис в коде
2. Неверный тип аргумента
3. Некорректное значение при вычислении литерала
4. Отсутствие нужного атрибута у узла AST

🔧 Подсказка:

Думайте о том, что проверяет Python при разборе, вычислении и обращении к объектам AST.

- **SyntaxError** — Неправильный синтаксис в коде
- **TypeError** — Неверный тип аргумента
- **ValueError** — Некорректное значение при вычислении литерала
- **AttributeError** — Отсутствие нужного атрибута у узла AST

🔧 Шаг 18:

Миссия 18

Что необходимо для безопасного изменения AST в чужом коде?

Подсказка: Подумайте о защите от неожиданных последствий.

Ответ:

- **Работать на копии дерева AST и проверять результат тестами**
- Изменять дерево напрямую без проверки
- Удалять все узлы случайным образом

- Игнорировать предупреждения Python

Шаг 19:

Миссия 19

Как AST помогает ускорять выполнение кода?

Подсказка: Подумайте, какие операции можно вычислить заранее.

Ответ:

- **Позволяет выполнять оптимизации, вычисляя константы на этапе компиляции и упрощая выражения**
- Компилирует код в машинный язык мгновенно
- Запускает код параллельно на GPU
- Создает новые функции автоматически без анализа

Урок 4 - Практика автогенерация SQL ORM-классов

Шаг 01:

Миссия 1

Какой подход лучше использовать для генерации ORM-классов из SQL-таблицы?

Подсказка: Подумайте, как автоматизировать создание полей.

Ответ:

- **Анализ схемы таблицы и генерация классов автоматически**
- Создание классов вручную без проверки схемы
- Использование случайных имён для полей
- Игнорирование типов столбцов

Шаг 02:

Миссия 2

Что поможет при автогенерации методов CRUD для ORM-классов?

Подсказка: Подумайте, какие действия нужны для работы с базой данных.

Ответ:

- **Создание шаблонных методов: create, read, update, delete**
- Добавление случайных функций без связи с базой
- Заполнение методов вручную без проверки таблицы
- Игнорирование внешних ключей

Шаг 03:

Миссия 3

Сопоставьте типы столбцов SQL с типами полей Python ORM

Ответ:

- INTEGER
- TEXT
- BOOLEAN
- DATE

1. int

2. str
3. bool
4. datetime.date

 Подсказка:

Обратите внимание на соответствие типов данных SQL и Python.

- **INTEGER** — int
- **TEXT** — str
- **BOOLEAN** — bool
- **DATE** — datetime.date

 Шаг 04:

Миссия 4

Сопоставьте типы отношений в базе данных с действиями в ORM

Ответ:

- OneToOne
 - OneToMany
 - ManyToMany
 - ForeignKey
1. Связь один-к-одному между таблицами
 2. Связь один-ко-многим
 3. Связь многие-ко-многим
 4. Создание внешнего ключа для связи

 Подсказка:

Подумайте, как моделируются отношения между объектами и таблицами.

- **OneToOne** — Связь один-к-одному между таблицами
- **OneToMany** — Связь один-ко-многим
- **ManyToMany** — Связь многие-ко-многим
- **ForeignKey** — Создание внешнего ключа для связи

 Шаг 05:

Миссия 5

Сопоставьте этапы автогенерации ORM-классов с их действиями

Ответ:

- Анализ таблицы
 - Создание классов
 - Добавление методов CRUD
 - Тестирование сгенерированных классов
1. Чтение схемы таблицы и получение полей
 2. Определение структуры класса и атрибутов
 3. Генерация методов для работы с данными
 4. Проверка работоспособности классов

 Подсказка:

Сначала узнаём структуру, потом создаём классы, добавляем методы и тестируем.

- **Анализ таблицы** — Чтение схемы таблицы и получение полей
- **Создание классов** — Определение структуры класса и атрибутов
- **Добавление методов CRUD** — Генерация методов для работы с данными
- **Тестирование сгенерированных классов** — Проверка работоспособности классов

🔮 Шаг 06:

Миссия 6

Расположите шаги безопасного добавления новых таблиц в автогенератор ORM:

1. Добавление схемы таблицы
2. Генерация класса Python
3. Создание методов CRUD
4. Тестирование функциональности

🔮 Подсказка:

Сначала добавляем схему, потом создаём класс, затем методы и проверяем работу.

Ответ:

- Добавление схемы таблицы
- Генерация класса Python
- Создание методов CRUD
- Тестирование функциональности

🔮 Шаг 07:

Миссия 7

Сопоставьте методы ORM с их назначением

Ответ:

- `save()`
 - `delete()`
 - `update()`
 - `get()`
1. Сохраняет объект в базу
 2. Удаляет объект из базы
 3. Обновляет поля объекта
 4. Получает объект по фильтру

🔮 Подсказка:

Подумайте, что делает каждый метод в CRUD.

- **`save()`** — Сохраняет объект в базу
- **`delete()`** — Удаляет объект из базы
- **`update()`** — Обновляет поля объекта
- **`get()`** — Получает объект по фильтру

🔮 Шаг 08:

Миссия 8

Сопоставьте типы полей в ORM с их ограничениями

Ответ:

- CharField
 - IntegerField
 - BooleanField
 - DateField
1. Ограничение длины строки
 2. Только целые числа
 3. Логическое значение True/False
 4. Хранение даты

 Подсказка:

Подумайте о типичных ограничениях для каждого типа данных.

- **CharField** — Ограничение длины строки
- **IntegerField** — Только целые числа
- **BooleanField** — Логическое значение True/False
- **DateField** — Хранение даты

 Шаг 09:

Миссия 9

Сопоставьте атрибуты класса с их ролями

Ответ:

- Meta
 - __str__
 - id
 - objects
1. Настройки модели (таблица, порядок)
 2. Строковое представление объекта
 3. Уникальный идентификатор записи
 4. Менеджер объектов для запросов

 Подсказка:

Обратите внимание на стандартные атрибуты Django ORM.

- **Meta** — Настройки модели (таблица, порядок)
- **__str__** — Строковое представление объекта
- **id** — Уникальный идентификатор записи
- **objects** — Менеджер объектов для запросов

 Шаг 10:

Миссия 10

Сопоставьте типы связей ORM с их SQL аналогами

Ответ:

- OneToOneField
- ForeignKey
- ManyToManyField

- `PrimaryKey`

1. `UNIQUE + FOREIGN KEY`
2. `FOREIGN KEY`
3. Таблица связи многие-ко-многим
4. `PRIMARY KEY`

🔗 Подсказка:

Связи ORM отражают ограничения базы данных.

- **`OneToOneField`** — `UNIQUE + FOREIGN KEY`
- **`ForeignKey`** — `FOREIGN KEY`
- **`ManyToManyField`** — Таблица связи многие-ко-многим
- **`PrimaryKey`** — `PRIMARY KEY`

🔗 Шаг 11:

Миссия 11

Расположите шаги автогенерации ORM-классов в правильном порядке:

1. Подключение к базе данных
2. Получение схем таблиц
3. Генерация Python-классов
4. Добавление CRUD-методов
5. Тестирование классов

🔗 Подсказка:

Сначала подключаемся, затем получаем схему, создаём классы, добавляем методы и тестируем.

Ответ:

- Подключение к базе данных
- Получение схем таблиц
- Генерация Python-классов
- Добавление CRUD-методов
- Тестирование классов

🔗 Шаг 12:

Миссия 12

Расположите действия при обновлении схемы таблицы для ORM:

1. Анализ изменений схемы
2. Обновление классов Python
3. Обновление методов CRUD
4. Тестирование обновлённых классов

🔗 Подсказка:

Сначала анализируем изменения, потом обновляем классы и методы, в конце проверяем тестами.

Ответ:

- Анализ изменений схемы
- Обновление классов Python
- Обновление методов CRUD

- Тестирование обновлённых классов

Шаг 13:

Миссия 13

Расположите этапы тестирования сгенерированных ORM-классов:

1. Создание тестовой базы данных
2. Создание объектов через ORM
3. Чтение и проверка данных
4. Удаление тестовых данных

Подсказка:

Сначала создаём тестовую базу, потом объекты, проверяем чтение и очищаем базу.

Ответ:

- Создание тестовой базы данных
- Создание объектов через ORM
- Чтение и проверка данных
- Удаление тестовых данных

Шаг 14:

Миссия 14

Расположите шаги при добавлении нового поля в ORM-класс:

1. Изменение схемы таблицы в базе
2. Добавление атрибута в Python-класс
3. Обновление методов CRUD
4. Тестирование нового поля

Подсказка:

Сначала меняем схему, потом класс, методы и проверяем через тесты.

Ответ:

- Изменение схемы таблицы в базе
- Добавление атрибута в Python-класс
- Обновление методов CRUD
- Тестирование нового поля

Шаг 15:

Миссия 15

Расположите действия при удалении поля из ORM-класса:

1. Удаление столбца в базе данных
2. Удаление атрибута в Python-классе
3. Обновление методов CRUD
4. Проверка работы ORM после изменений

Подсказка:

Удаляем сначала в базе, потом в классе, далее методы и проверка.

Ответ:

- Удаление столбца в базе данных

- Удаление атрибута в Python-классе
- Обновление методов CRUD
- Проверка работы ORM после изменений

Шаг 16:

Миссия 16

Расположите этапы полной генерации ORM-классов для всех таблиц базы:

1. Сбор схем всех таблиц
2. Создание Python-классов для каждой таблицы
3. Добавление атрибутов и связей
4. Добавление CRUD-методов
5. Тестирование всех классов

Подсказка:

Этапы идут от сбора схем до финального тестирования всех сгенерированных классов.

Ответ:

- Сбор схем всех таблиц
- Создание Python-классов для каждой таблицы
- Добавление атрибутов и связей
- Добавление CRUD-методов
- Тестирование всех классов

Урок 5 - Лабиринт мини-компилятор в Python

Шаг 01:

Миссия 1

Вы нашли кусок Python-кода, который генерирует другой код динамически. Как лучше всего проверить его корректность?

Подсказка: Важно убедиться, что создаваемый код работает, прежде чем его использовать.

Ответ:

- **Написать тесты для сгенерированного кода**
- Прямо запускать код без проверки
- Игнорировать ошибки и надеяться на лучшее
- Сразу публиковать код в продакшн

Шаг 02:

Миссия 2

Ваша задача — сгенерировать функцию для подсчета суммы элементов списка. Какой способ генерации будет самым безопасным?

Подсказка: Лучше избегать использования eval без необходимости.

Ответ:

- **Использовать ast.parse и compile для безопасной генерации**
- Использовать eval на произвольном тексте
- Сохранять код как строку и запускать через exec
- Писать код вручную, игнорируя генерацию

Шаг 03:

Миссия 3

Как лучше отлаживать мини-компилятор, если он генерирует много кода сразу?

Подсказка: Легче анализировать маленькие фрагменты.

Ответ:

- **Генерировать и тестировать код по частям**
- Сразу тестировать весь массив кода
- Игнорировать ошибки на этапе генерации
- Удалять тесты, чтобы ускорить генерацию

Шаг 04:

Миссия 4

Вы хотите, чтобы мини-компилятор сам исправлял ошибки в коде. Что нужно учитывать в первую очередь?

Подсказка: Автоматическая коррекция может изменить логику кода.

Ответ:

- **Сохранять поведение программы без изменений**
- Менять логику для быстрого исправления
- Пропускать проверку синтаксиса
- Удалять все комментарии для уменьшения ошибок

Шаг 05:

Миссия 5

При генерации кода важно учитывать стиль и читаемость. Почему это важно?

Подсказка: Другое программисту проще исправлять и улучшать такой код.

Ответ:

- **Чтобы облегчить поддержку и понимание кода**
- Чтобы код выглядел сложнее и круче
- Чтобы увеличить размер файлов
- Чтобы ускорить компиляцию любой ценой

Шаг 06:

Миссия 6

 Задача: Сопоставьте элементы мини-компилятора с их ролью

Понятия:

Ответ:

- AST (Abstract Syntax Tree)
- Tokenization
- Code Generation
- Optimization

Определения:

1. Разбор кода на удобную для обработки структуру
2. Разбиение исходного кода на базовые элементы

3. Создание исполняемого кода из представления
4. Улучшение производительности сгенерированного кода

Подсказка: Думайте, что происходит первым, а что последним.

- **AST (Abstract Syntax Tree)** — Разбор кода на удобную для обработки структуру
- **Tokenization** — Разбиение исходного кода на базовые элементы
- **Code Generation** — Создание исполняемого кода из представления
- **Optimization** — Улучшение производительности сгенерированного кода

Шаг 07:

Миссия 7

 Задача: Сопоставьте ошибки генерации кода с последствиями

Понятия:

Ответ:

- Синтаксическая ошибка
- Логическая ошибка
- Неопределенная переменная
- Бесконечный цикл

Определения:

1. Программа не запускается из-за неправильного синтаксиса
2. Программа выполняется неправильно, но без падений
3. Использование переменной до определения
4. Программа застревает в бесконечном повторении

Подсказка: Ошибки могут проявляться сразу или во время выполнения.

- **Синтаксическая ошибка** — Программа не запускается из-за неправильного синтаксиса
- **Логическая ошибка** — Программа выполняется неправильно, но без падений
- **Неопределенная переменная** — Использование переменной до определения
- **Бесконечный цикл** — Программа застревает в бесконечном повторении

Шаг 08:

Миссия 8

 Задача: Сопоставьте инструменты Python для метапрограммирования с их назначением

Понятия:

Ответ:

- metaclass
- decorator
- exec
- compile

Определения:

1. Создание классов на лету или модификация классов
2. Изменение поведения функций без изменения их кода
3. Выполнение строкового кода как Python-кода
4. Преобразование AST или текста в исполняемый код

Подсказка: Подумайте, какие возможности дают Python-объекты для динамического изменения.

- **metaclass** — Создание классов на лету или модификация классов
- **decorator** — Изменение поведения функций без изменения их кода
- **exec** — Выполнение строкового кода как Python-кода
- **compile** — Преобразование AST или текста в исполняемый код

Шаг 09:

Миссия 9

 Задача: Сопоставьте типы генерации кода с их примером

Понятия:

Ответ:

- Генерация функций
- Генерация классов
- Генерация модулей
- Генерация выражений

Определения:

1. Функция, созданная динамически в рантайме
2. Класс, созданный с помощью метакласса
3. Модуль, создаваемый на основе шаблонного кода
4. Строковое выражение, компилируемое и выполняемое через eval

Подсказка: Разные уровни абстракции кода требуют разных подходов.

- **Генерация функций** — Функция, созданная динамически в рантайме
- **Генерация классов** — Класс, созданный с помощью метакласса
- **Генерация модулей** — Модуль, создаваемый на основе шаблонного кода
- **Генерация выражений** — Строковое выражение, компилируемое и выполняемое через eval

Шаг 10:

Миссия 10

 Сопоставьте тип ошибки генерации кода с источником проблемы

Понятия:

Ответ:

- Ошибка синтаксиса
- Ошибка выполнения
- Ошибка логики
- Ошибка безопасности

Определения:

1. Неверно сформированный Python-код
2. Возникновение исключения при запуске
3. Программа делает не то, что ожидалось
4. Использование небезопасного eval или exec

Подсказка: Ошибки могут быть видны сразу или проявляться во время выполнения.

- **Ошибка синтаксиса** — Неверно сформированный Python-код

- **Ошибка выполнения** — Возникновение исключения при запуске
- **Ошибка логики** — Программа делает не то, что ожидалось
- **Ошибка безопасности** — Использование небезопасного eval или exec

Шаг 11:

Миссия 11

Расположите этапы работы мини-компилятора в правильном порядке:

1. Генерация кода
2. Оптимизация кода
3. Токенизация исходного кода
4. Создание AST

Подсказка: Подумайте о логике обработки: сначала читаем, потом структурируем, потом создаем и оптимизируем.

Ответ:

- Токенизация исходного кода
- Создание AST
- Генерация кода
- Оптимизация кода

Шаг 12:

Миссия 12

Расположите этапы безопасного тестирования сгенерированного кода:

1. Написание тестов
2. Выполнение кода в песочнице
3. Анализ результатов
4. Исправление ошибок

Подсказка: Тестирование начинается с подготовки, затем безопасное выполнение и исправление проблем.

Ответ:

- Написание тестов
- Выполнение кода в песочнице
- Анализ результатов
- Исправление ошибок

Шаг 13:

Миссия 13

Расположите шаги создания метакласса для мини-компилятора:

1. Определение __new__ или __init__
2. Создание класса с использованием метакласса
3. Определение атрибутов и методов
4. Использование созданного класса

Подсказка: Сначала пишем метакласс, затем создаем класс, затем используем его.

Ответ:

- Определение `__new__` или `__init__`
- Определение атрибутов и методов
- Создание класса с использованием метакласса
- Использование созданного класса

 Шаг 14:

Миссия 14

 **Задача:** Сопоставьте типы данных, которые чаще всего используются в динамически генерируемом коде, с их особенностями

Понятия:

Ответ:

- Строки
- Списки
- Словари
- Функции

Определения:

1. Хранят текстовые шаблоны для генерации кода
2. Используются для хранения последовательностей элементов
3. Служат для хранения пар ключ-значение при динамическом создании структуры
4. Позволяют создавать исполняемые блоки кода на лету

Подсказка: Подумайте, какой тип данных хранит код, а какой — структуру.

- **Строки** — Хранят текстовые шаблоны для генерации кода
- **Списки** — Используются для хранения последовательностей элементов
- **Словари** — Служат для хранения пар ключ-значение при динамическом создании структуры
- **Функции** — Позволяют создавать исполняемые блоки кода на лету

 Шаг 15:

Миссия 15

 **Задача:** Сопоставьте методы тестирования с их особенностями при работе с мини-компилятором

Понятия:

Ответ:

- Unit-тесты
- Интеграционные тесты
- Песочница (sandbox)
- Логирование

Определения:

1. Тестируют отдельные функции или классы
2. Проверяют совместную работу нескольких модулей
3. Безопасное выполнение сгенерированного кода, чтобы избежать ошибок
4. Фиксирует ошибки и шаги выполнения для анализа

Подсказка: Подумайте, какой метод безопасно проверяет код, а какой фиксирует проблемы.

- **Unit-тесты** — Тестируют отдельные функции или классы
- **Интеграционные тесты** — Проверяют совместную работу нескольких модулей
- **Песочница (sandbox)** — Безопасное выполнение сгенерированного кода, чтобы избежать ошибок
- **Логирование** — Фиксирует ошибки и шаги выполнения для анализа

🔧 Шаг 16:

Миссия 16

📖 Задача: Сопоставьте типы оптимизаций мини-компилятора с их эффектом

Понятия:

Ответ:

- Удаление дублирующегося кода
- Инлайнинг функций
- Оптимизация циклов
- Минимизация использования памяти

Определения:

1. Сокращает повторяющиеся участки кода для уменьшения объема
2. Встраивание кода функции прямо в место вызова для ускорения
3. Перестройка циклов для повышения производительности
4. Снижение потребления оперативной памяти с помощью структур данных

Подсказка: Оптимизации могут улучшать скорость, читаемость или расход ресурсов.

- **Удаление дублирующегося кода** — Сокращает повторяющиеся участки кода для уменьшения объема
- **Инлайнинг функций** — Встраивание кода функции прямо в место вызова для ускорения
- **Оптимизация циклов** — Перестройка циклов для повышения производительности
- **Минимизация использования памяти** — Снижение потребления оперативной памяти с помощью структур данных

🔧 Шаг 17:

Миссия 17

Вы работаете с мини-компилятором Python, который генерирует код на лету. Как безопаснее всего проверить его работу?

Подсказка: Тестирование позволяет найти ошибки до запуска в продакшн.

Ответ:

- **Написать тесты для сгенерированного кода**
- Запускать код напрямую в продакшн
- Игнорировать предупреждения компилятора
- Сразу исправлять ошибки вручную без проверки

Модуль 6 - Храм Архимага

Урок 1 - Реализация мини-интерпретатора Python

Шаг 01:

Миссия 1

Вы создаёте мини-интерпретатор Python. Как лучше всего проверить базовую арифметику?
Подсказка: Сначала проверяются самые простые функции.

Ответ:

- **Написать тесты с простыми выражениями**
- Запустить сложные скрипты сразу
- Игнорировать ошибки при выполнении
- Использовать только print для проверки

Шаг 02:

Миссия 2

Вы добавляете поддержку переменных. Какой шаг наиболее важен при интерпретации кода?
Подсказка: Переменные нужно хранить в отдельной структуре.

Ответ:

- **Создать таблицу символов для хранения значений**
- Игнорировать переменные
- Использовать глобальные переменные без контроля
- Записывать переменные в текстовый файл

Шаг 03:

Миссия 3

 Задача: Сопоставьте компоненты мини-интерпретатора с их назначением

Понятия:

Ответ:

- Лексер
- Парсер
- Виртуальная машина
- Таблица символов

Определения:

1. Разбивает код на токены
2. Проверяет синтаксис и строит дерево разбора
3. Выполняет скомпилированный код
4. Хранит значения переменных и функций

Подсказка: Подумайте, какая часть отвечает за разбор, а какая за исполнение.

- **Лексер** — Разбивает код на токены
- **Парсер** — Проверяет синтаксис и строит дерево разбора
- **Виртуальная машина** — Выполняет скомпилированный код
- **Таблица символов** — Хранит значения переменных и функций

Шаг 04:

Миссия 4

 Задача: Сопоставьте типы ошибок интерпретатора с их описанием

Понятия:

Ответ:

- Синтаксическая ошибка
- Ошибка выполнения
- Ошибка имени переменной
- Ошибка типов

Определения:

1. Неправильный синтаксис кода
2. Ошибка во время выполнения программы
3. Обращение к несуществующей переменной
4. Операции с несовместимыми типами

Подсказка: Сначала проверяется синтаксис, потом логика работы кода.

- **Синтаксическая ошибка** — Неправильный синтаксис кода
- **Ошибка выполнения** — Ошибка во время выполнения программы
- **Ошибка имени переменной** — Обращение к несуществующей переменной
- **Ошибка типов** — Операции с несовместимыми типами

Шаг 05:

Миссия 5

Расположите шаги обработки выражения интерпретатором в правильном порядке:

1. Лексический анализ (токенизация)
2. Синтаксический анализ (построение дерева разбора)
3. Генерация байткода
4. Выполнение на виртуальной машине

Подсказка: Подумайте, какие шаги должны идти друг за другом от анализа к исполнению.

Ответ:

- Лексический анализ (токенизация)
- Синтаксический анализ (построение дерева разбора)
- Генерация байткода
- Выполнение на виртуальной машине

Шаг 06:

Миссия 6

Вы реализуете поддержку условных операторов. Как правильно интерпретировать условие?

Подсказка: Логические выражения должны быть проверены до выполнения веток.

Ответ:

- **Вычислить выражение и перейти по нужной ветке**
- Выполнить обе ветки подряд
- Игнорировать условие и выбрать первую ветку

- Запрашивать ввод пользователя вместо вычисления

Шаг 07:

Миссия 7

Вы добавляете поддержку циклов. Что важно учитывать при выполнении кода?

Подсказка: Циклы должны завершаться, иначе интерпретатор зависнет.

Ответ:

- **Следить за условием выхода из цикла**
- Всегда выполнять бесконечно
- Игнорировать тело цикла
- Выходить из цикла через raise Exception

Шаг 08:

Миссия 8

 Задача: Сопоставьте конструкции языка Python с их интерпретацией

Понятия:

Ответ:

- if
- for
- while
- def

Определения:

1. Условный оператор, выполняет блок при истинном условии
2. Цикл с известным количеством итераций
3. Цикл с условием выхода
4. Объявление функции

Подсказка: Вспомните стандартные конструкции Python и их назначение.

- **if** — Условный оператор, выполняет блок при истинном условии
- **for** — Цикл с известным количеством итераций
- **while** — Цикл с условием выхода
- **def** — Объявление функции

Шаг 09:

Миссия 9

 Задача: Сопоставьте типы токенов интерпретатора с примером

Понятия:

Ответ:

- Ключевое слово
- Идентификатор
- Литерал
- Оператор

Определения:

1. for, if, def

2. имя переменной или функции
3. числа, строки, True, False
4. +, -, *, /

Подсказка: Обратите внимание, что ключевые слова нельзя использовать как имена переменных.

- **Ключевое слово** — for, if, def
- **Идентификатор** — имя переменной или функции
- **Литерал** — числа, строки, True, False
- **Оператор** — +, -, *, /

Шаг 10:

Миссия 10

Вы добавляете обработку функций с аргументами. Как лучше всего проверять аргументы?

Подсказка: Проверка типов и количества аргументов предотвращает ошибки во время выполнения.

Ответ:

- **Сравнивать количество и типы перед вызовом**
- Игнорировать и надеяться на правильный ввод
- Всегда приводить к строкам
- Пропускать проверку и ловить ошибки после

Шаг 11:

Миссия 11

Вы хотите реализовать возврат значения из функции. Что нужно учитывать?

Подсказка: Каждая функция должна возвращать правильный объект.

Ответ:

- **Использовать return для передачи результата**
- Использовать print вместо return
- Ничего не возвращать, полагаясь на глобальные переменные
- Возвращать всегда None

Шаг 12:

Миссия 12

 Задача: Сопоставьте типы циклов с примером использования

Понятия:

Ответ:

- for
- while

Определения:

1. Перебор элементов списка
2. Повторение до выполнения условия

Подсказка: for подходит для известных коллекций, while — для условий.

- **for** — Перебор элементов списка

- **while** — Повторение до выполнения условия

🔮 Шаг 13:

Миссия 13

📄 Задача: Сопоставьте типы ошибок функции с примером

Понятия:

Ответ:

- `TypeError`
- `ValueError`
- `ZeroDivisionError`
- `IndexError`

Определения:

1. Неверный тип аргумента
2. Недопустимое значение
3. Деление на ноль
4. Выход за пределы списка

Подсказка: Ошибки делятся на тип, значение и границы.

- **`TypeError`** — Неверный тип аргумента
- **`ValueError`** — Недопустимое значение
- **`ZeroDivisionError`** — Деление на ноль
- **`IndexError`** — Выход за пределы списка

🔮 Шаг 14:

Миссия 14

Расположите шаги обработки функции интерпретатором:

1. Проверка аргументов
2. Выполнение тела функции
3. Возврат значения

Подсказка: Сначала проверка, затем выполнение, потом результат.

Ответ:

- Проверка аргументов
- Выполнение тела функции
- Возврат значения

🔮 Шаг 15:

Миссия 15

Расположите порядок обработки условного оператора:

1. Вычисление условия
2. Выбор ветки `if` или `else`
3. Выполнение выбранной ветки

Подсказка: Логическое выражение решает, какая ветка выполняется.

Ответ:

- Вычисление условия
- Выбор ветки if или else
- Выполнение выбранной ветки

Урок 2 - Система плагинов через AST

Шаг 01:

Миссия 1

Вы разрабатываете мини-плагин для интерпретатора. Как лучше всего подключить новый модуль через AST?

Подсказка: Подумайте, как сохранить совместимость с существующими структурами кода.

Ответ:

- **Использовать AST-узлы для вставки нового функционала**
- Прямо менять исходный код интерпретатора
- Игнорировать AST и добавлять код вручную
- Выполнять код через eval без изменений

Шаг 02:

Миссия 2

Вы хотите добавить поддержку пользовательских функций через AST. Что важно проверить перед вставкой?

Подсказка: Проверяйте синтаксис и уникальность имени функции.

Ответ:

- **Убедиться, что AST узлы корректны и имена функций уникальны**
- Добавлять функцию без проверки синтаксиса
- Использовать только eval для функций
- Игнорировать имена функций

Шаг 03:

Миссия 3

 Сопоставьте элементы AST с их назначением

Ответ:

- Module
 - FunctionDef
 - Assign
 - Expr
1. Корневой узел для всего файла
 2. Определение функции
 3. Присваивание значения переменной
 4. Вычисляемое выражение или вызов функции

Подсказка: AST строится из узлов, каждый имеет строгое предназначение.

- **Module** — Корневой узел для всего файла
- **FunctionDef** — Определение функции

- **Assign** — Присваивание значения переменной
- **Expr** — Вычисляемое выражение или вызов функции

🔧 Шаг 04:

Миссия 4

📖 Сопоставьте методы AST с их функционалом

Ответ:

- parse
 - compile
 - NodeVisitor
 - NodeTransformer
1. Разбор исходного кода в AST
 2. Компиляция AST в исполняемый код
 3. Посещение всех узлов AST без изменения
 4. Изменение узлов AST

Подсказка: Разделяйте чтение, обход и модификацию AST.

- **parse** — Разбор исходного кода в AST
- **compile** — Компиляция AST в исполняемый код
- **NodeVisitor** — Посещение всех узлов AST без изменения
- **NodeTransformer** — Изменение узлов AST

🔧 Шаг 05:

Миссия 5

Вы создаёте плагин для логирования вызовов функций. Как лучше всего внедрить функционал через AST?

Подсказка: Логика должна внедряться до и после выполнения функции.

Ответ:

- **Добавить вызовы логирования в теле функции через NodeTransformer**
- Изменять исходный код строками
- Добавлять логирование через комментарии
- Игнорировать AST и модифицировать только байткод

🔧 Шаг 06:

Миссия 6

Вы хотите динамически подключать плагины. Как лучше всего реализовать подключение?

Подсказка: Подумайте о безопасности и согласованности AST.

Ответ:

- **Разбирать код плагина через ast.parse и применять NodeTransformer**
- Вставлять код плагина через exes
- Игнорировать проверку AST и импортировать напрямую
- Подключать плагин только вручную в коде

🔧 Шаг 07:

Миссия 7

 Сопоставьте виды AST узлов с примерами кода

Ответ:

- Call
- BinOp
- Return
- Import

1. Вызов функции: `print("Hello")`
2. Операция сложения: `a + b`
3. Возврат значения из функции: `return x`
4. Импорт модуля: `import math`

Подсказка: Каждый узел соответствует конкретной конструкции Python.

- **Call** — Вызов функции: `print("Hello")`
- **BinOp** — Операция сложения: `a + b`
- **Return** — Возврат значения из функции: `return x`
- **Import** — Импорт модуля: `import math`

Шаг 08:

Миссия 8

 Сопоставьте методы NodeTransformer с их назначением

Ответ:

- visit_FunctionDef
- visit_Assign
- generic_visit

1. Обработка и изменение всех узлов функции
2. Обработка присваивания
3. Обход всех дочерних узлов без изменений

Подсказка: Специальные методы обрабатывают конкретные типы узлов.

- **visit_FunctionDef** — Обработка и изменение всех узлов функции
- **visit_Assign** — Обработка присваивания
- **generic_visit** — Обход всех дочерних узлов без изменений

Шаг 09:

Миссия 9

Вы реализуете плагин для проверки типов аргументов функций. Как лучше всего это сделать через AST?

Подсказка: Внедряйте проверки до вызова функции.

Ответ:

- **Добавить узлы проверки типа в начале тела функции через NodeTransformer**
- Игнорировать типы и проверять после вызова
- Использовать eval для проверки аргументов
- Добавлять проверку через print

Шаг 10:

Миссия 10

Расположите шаги добавления плагина через AST:

1. Разбор исходного кода плагина в AST
2. Применение NodeTransformer для модификации узлов
3. Компиляция AST в исполняемый код
4. Вызов или регистрация плагина

Подсказка: Сначала анализ, затем изменение, потом компиляция и запуск.

Ответ:

- Разбор исходного кода плагина в AST
- Применение NodeTransformer для модификации узлов
- Компиляция AST в исполняемый код
- Вызов или регистрация плагина

Шаг 11:

Миссия 11

Расположите этапы внедрения логирования вызовов функций через AST:

1. Определение функций для логирования
2. Вставка вызовов логирования перед и после основной логики функции
3. Компиляция и регистрация модифицированного AST

Подсказка: Сначала определяем, потом вставляем, потом применяем изменения.

Ответ:

- Определение функций для логирования
- Вставка вызовов логирования перед и после основной логики функции
- Компиляция и регистрация модифицированного AST

Шаг 12:

Миссия 12

Вы хотите добавить поддержку условных вставок в AST. Как правильно проверять условие?

Подсказка: Логическое выражение должно быть вычислено до вставки кода.

Ответ:

- **Вычислять условие и применять вставку только если оно истинно**
- Вставлять код всегда независимо от условия
- Игнорировать условие и выполнять ручную
- Добавлять проверку через print

Шаг 13:

Миссия 13

 Сопоставьте этапы обработки AST при плагинах с их назначением:

Ответ:

- parse
- NodeTransformer

- `compile`
- `exec`

1. Создание AST из исходного кода
2. Изменение узлов AST
3. Компиляция AST в объект кода
4. Выполнение скомпилированного кода

Подсказка: Помните цепочку: разбор, изменение, компиляция, запуск.

- **`parse`** — Создание AST из исходного кода
- **`NodeTransformer`** — Изменение узлов AST
- **`compile`** — Компиляция AST в объект кода
- **`exec`** — Выполнение скомпилированного кода

Шаг 14:

Миссия 14

Вы разрабатываете мини-плагин для интерпретатора. Как лучше всего подключить новый модуль через AST?

Подсказка: Подумайте, как сохранить совместимость с существующими структурами кода.

Ответ:

- **Использовать AST-узлы для вставки нового функционала**
- Прямо менять исходный код интерпретатора
- Игнорировать AST и добавлять код вручную
- Выполнять код через `eval` без изменений

Шаг 15:

Миссия 15

Вы хотите, чтобы генератор тестов проверял корректность поведения функции при граничных значениях. Как действовать?

Подсказка: Подумайте о крайних случаях, которые чаще всего вызывают ошибки.

Ответ:

- **Создать тестовые случаи с минимальными, максимальными и нулевыми значениями**
- Тестировать только стандартные случаи
- Игнорировать граничные значения
- Случайным образом подставлять числа без проверки

Шаг 16:

Миссия 16

 Сопоставьте методы анализа кода с их функционалом:

Ответ:

- `NodeVisitor`
- `NodeTransformer`
- `compile`
- `exec`

1. Проход по AST без изменения структуры

2. Модификация AST с возможностью изменения кода
3. Компиляция кода в байткод
4. Выполнение скомпилированного кода

Подсказка: Разделяйте анализ, трансформацию и выполнение кода.

- **NodeVisitor** — Проход по AST без изменения структуры
- **NodeTransformer** — Модификация AST с возможностью изменения кода
- **compile** — Компиляция кода в байткод
- **exec** — Выполнение скомпилированного кода

Шаг 17:

Миссия 17

Расположите шаги интеграции нового плагина через AST в правильном порядке:

1. Проход по AST и выявление точек вставки
2. Создание AST-узлов нового функционала
3. Вставка новых узлов в AST
4. Компиляция и тестирование изменённого кода

Подсказка: Сначала анализируем, затем создаём узлы, вставляем и тестируем.

Ответ:

- Проход по AST и выявление точек вставки
- Создание AST-узлов нового функционала
- Вставка новых узлов в AST
- Компиляция и тестирование изменённого кода

Урок 3 - Генератор тестов на основе кода

Шаг 01:

Миссия 1

Вы хотите автоматически генерировать тесты для функций. С чего начать?

Подсказка: Сначала подумайте, что можно анализировать в коде.

Ответ:

- **Проанализировать функции через AST для понимания их параметров**
- Просто записать ручные тесты в файл
- Игнорировать код и генерировать случайные данные
- Использовать eval для всех функций без проверки

Шаг 02:

Миссия 2

Вы хотите, чтобы тесты автоматически проверяли значения аргументов. Как лучше это реализовать?

Подсказка: Обратите внимание на сигнатуры функций.

Ответ:

- **Извлекать аргументы функций через AST и формировать тестовые случаи**
- Присваивать случайные значения аргументам без анализа
- Писать тесты вручную без использования кода функции

- Пропускать проверку аргументов и тестировать только результат

Шаг 03:

Миссия 3

 Сопоставьте элементы генератора тестов с их назначением

Ответ:

- AST
 - TestCase
 - assert
 - Mock
1. Разбор кода и извлечение функций
 2. Класс для хранения тестов и их запуска
 3. Проверка корректности результата
 4. Замена внешних зависимостей во время тестирования

Подсказка: Каждый элемент выполняет отдельную задачу — думайте о роли.

- **AST** — Разбор кода и извлечение функций
- **TestCase** — Класс для хранения тестов и их запуска
- **assert** — Проверка корректности результата
- **Mock** — Замена внешних зависимостей во время тестирования

Шаг 04:

Миссия 4

 Сопоставьте типы тестов с их задачами

Ответ:

- Unit Test
 - Integration Test
 - Regression Test
 - Smoke Test
1. Проверка отдельной функции или метода
 2. Проверка работы нескольких компонентов вместе
 3. Проверка, что новые изменения не сломали старый функционал
 4. Быстрая проверка базовой работоспособности приложения

Подсказка: Разделяйте по уровню проверки и цели тестирования.

- **Unit Test** — Проверка отдельной функции или метода
- **Integration Test** — Проверка работы нескольких компонентов вместе
- **Regression Test** — Проверка, что новые изменения не сломали старый функционал
- **Smoke Test** — Быстрая проверка базовой работоспособности приложения

Шаг 05:

Миссия 5

Вы хотите автоматически проверять возвращаемые значения функций. Что нужно сделать первым делом?

Подсказка: Обратите внимание на то, что функция возвращает.

Ответ:

- **Извлечь сигнатуру функции и определить тип возвращаемого значения**
- Сразу генерировать случайные результаты
- Игнорировать тип результата
- Только логировать функцию без проверки результата

Шаг 06:

Миссия 6

Вы хотите, чтобы генератор тестов создавал проверки на исключения. Как поступить?

Подсказка: Подумайте, как отследить ошибки без остановки программы.

Ответ:

- **Оборачивать вызовы в try/except и проверять ожидаемые исключения**
- Игнорировать исключения и продолжать генерацию
- Только печатать ошибки без проверки
- Прерывать генерацию тестов при первой ошибке

Шаг 07:

Миссия 7

 Сопоставьте подходы генерации тестов с их описанием

Ответ:

- Black-box
 - White-box
 - Property-based
 - Parameterized
1. Тестирование без знания внутренней структуры кода
 2. Тестирование с использованием знаний внутренней структуры кода
 3. Автоматическая проверка свойств функции на множестве данных
 4. Создание тестов с различными комбинациями входных значений

Подсказка: Смотрите на доступ к внутренним деталям функции.

- **Black-box** — Тестирование без знания внутренней структуры кода
- **White-box** — Тестирование с использованием знаний внутренней структуры кода
- **Property-based** — Автоматическая проверка свойств функции на множестве данных
- **Parameterized** — Создание тестов с различными комбинациями входных значений

Шаг 08:

Миссия 8

 Сопоставьте методы проверки тестов с их назначением

Ответ:

- assertEquals
 - assertTrue
 - assertRaises
 - assertIn
1. Проверка на равенство ожидаемого и фактического значения

2. Проверка, что выражение истинно
3. Проверка, что функция вызывает ожидаемое исключение
4. Проверка, что элемент присутствует в коллекции

Подсказка: Подумайте о типах проверки и о том, что именно вы хотите тестировать.

- **assertEqual** — Проверка на равенство ожидаемого и фактического значения
- **assertTrue** — Проверка, что выражение истинно
- **assertRaises** — Проверка, что функция вызывает ожидаемое исключение
- **assertIn** — Проверка, что элемент присутствует в коллекции

Шаг 09:

Миссия 9

Вы хотите проверять функции с множеством аргументов. Как автоматизировать генерацию входных данных?

Подсказка: Рассмотрите комбинации аргументов для разных сценариев.

Ответ:

- **Использовать `parameterized` или `product` для генерации комбинаций аргументов**
- Вводить данные вручную
- Игнорировать аргументы и проверять только один вариант
- Генерировать случайные строки без связи с аргументами

Шаг 10:

Миссия 10

Вы хотите, чтобы генератор тестов проверял корректность документации функций. Что делать?

Подсказка: Смотрите на `docstring` функций.

Ответ:

- **Анализировать `docstring` и генерировать тесты на основе описанных аргументов и результата**
- Игнорировать документацию
- Проверять только название функции
- Сравнивать `docstring` с комментарием вручную

Шаг 11:

Миссия 11

Расположите этапы генерации тестов автоматически через код:

1. Разбор функций через AST
2. Генерация входных данных для тестов
3. Создание тестовых функций и проверок
4. Запуск и сбор результатов тестирования

Подсказка: Сначала анализ кода, потом генерация данных, формирование тестов, запуск.

Ответ:

- Разбор функций через AST
- Генерация входных данных для тестов
- Создание тестовых функций и проверок
- Запуск и сбор результатов тестирования

Шаг 12:

Миссия 12

Расположите шаги проверки исключений в автоматических тестах:

1. Определение ожидаемого исключения
2. Оборачивание вызова функции в try/except
3. Проверка типа пойманного исключения
4. Логирование результата проверки

Подсказка: Сначала ожидаем, потом проверяем, затем фиксируем результат.

Ответ:

- Определение ожидаемого исключения
- Оборачивание вызова функции в try/except
- Проверка типа пойманного исключения
- Логирование результата проверки

Шаг 13:

Миссия 13

Вы хотите, чтобы тесты проверяли функции с вложенными вызовами. Как действовать?

Подсказка: Обратите внимание на порядок вызовов функций.

Ответ:

- **Использовать NodeVisitor или NodeTransformer для анализа вложенных вызовов**
- Игнорировать вложенные вызовы
- Ручная проверка каждого вызова
- Только логировать первый вызов функции

Шаг 14:

Миссия 14

 Сопоставьте виды генерации тестов с их преимуществами

Ответ:

- Unit tests
 - Property-based
 - Parameterized
 - Integration tests
1. Легко локализовать ошибку в отдельной функции
 2. Проверяет свойства функции на множестве данных
 3. Автоматически проверяет несколько комбинаций аргументов
 4. Проверяет взаимодействие между компонентами

Подсказка: Смотрите на цель каждого типа тестов.

- **Unit tests** — Легко локализовать ошибку в отдельной функции
- **Property-based** — Проверяет свойства функции на множестве данных
- **Parameterized** — Автоматически проверяет несколько комбинаций аргументов
- **Integration tests** — Проверяет взаимодействие между компонентами

Шаг 15:

Миссия 15

Вы хотите автоматически генерировать тесты для функций с разными аргументами. Какой подход будет наиболее эффективным?

Подсказка: Подумайте о переборе вариантов и покрытии всех сценариев.

Ответ:

- **Использовать генерацию тестовых случаев на основе AST-анализа аргументов функции**
- Только тестировать стандартные значения
- Игнорировать аргументы по умолчанию
- Создавать случайные числа без проверки сценариев

Шаг 16:

Миссия 16

 Сопоставьте инструменты Python с их ролью при генерации тестов:

Ответ:

- AST
 - unittest
 - pytest
 - inspect
1. Разбор и анализ структуры кода
 2. Стандартная библиотека для тестирования
 3. Популярная библиотека с расширенными возможностями тестирования
 4. Получение информации о сигнатурах функций

Подсказка: Внимательно различайте анализ кода и запуск тестов.

- **AST** — Разбор и анализ структуры кода
- **unittest** — Стандартная библиотека для тестирования
- **pytest** — Популярная библиотека с расширенными возможностями тестирования
- **inspect** — Получение информации о сигнатурах функций

Шаг 17:

Миссия 17

Расположите этапы генерации автоматических тестов в правильном порядке:

1. Сбор информации о функциях (аргументы, сигнатуры)
2. Анализ кода через AST
3. Создание тестовых функций
4. Запуск тестов и проверка результатов

Подсказка: Сначала изучаем функцию, затем создаём тесты и проверяем их выполнение.

Ответ:

- Сбор информации о функциях (аргументы, сигнатуры)
- Анализ кода через AST
- Создание тестовых функций
- Запуск тестов и проверка результатов

Урок 4 - Практика самописный transpiler

Шаг 01:

Миссия 1

Вы начинаете создавать свой transpiler, который преобразует Python в упрощённый Python. С чего стоит начать?

Подсказка: Подумайте о структуре программы и её элементах.

Ответ:

- **Создание парсера для синтаксического дерева (AST)**
- Написание генератора документации
- Оптимизация стандартной библиотеки
- Изучение алгоритмов сортировки

Шаг 02:

Миссия 2

Какой инструмент Python поможет вам анализировать структуру кода для transpiler?

Подсказка: Это встроенный модуль для разбора кода на компоненты.

Ответ:

- **ast**
- os
- sys
- random

Шаг 03:

Миссия 3

Вы хотите поддерживать возможность расширений через плагины. Как это проще реализовать?

Подсказка: Подумайте о динамическом импорте модулей.

Ответ:

- **Использовать механизм импорта модулей по имени**
- Жёстко прописывать все плагины в коде
- Не использовать плагины вообще
- Использовать только глобальные переменные

Шаг 04:

Миссия 4

 Сопоставьте элементы transpiler с их назначением:

Ответ:

- Lexer
 - Parser
 - AST Transformer
 - Code Generator
1. Разбивает исходный код на токены
 2. Проверяет синтаксис и строит дерево разбора
 3. Преобразует дерево в новый вид для генерации кода

4. Создаёт конечный код на целевом языке

Подсказка: Внимательно следите за последовательностью обработки кода.

- **Lexer** — Разбивает исходный код на токены
- **Parser** — Проверяет синтаксис и строит дерево разбора
- **AST Transformer** — Преобразует дерево в новый вид для генерации кода
- **Code Generator** — Создаёт конечный код на целевом языке

Шаг 05:

Миссия 5

 Сопоставьте типы узлов AST с их ролью:

Ответ:

- FunctionDef
 - Assign
 - Call
 - If
1. Определение функции
 2. Присваивание значения переменной
 3. Вызов функции
 4. Условная конструкция

Подсказка: Подумайте о назначении каждого узла при разборе кода.

- **FunctionDef** — Определение функции
- **Assign** — Присваивание значения переменной
- **Call** — Вызов функции
- **If** — Условная конструкция

Шаг 06:

Миссия 6

 Сопоставьте фазы работы transpiler с их действием:

Ответ:

- Tokenization
 - Parsing
 - Transformation
 - Code Generation
1. Преобразование исходного кода в токены
 2. Создание структуры кода в виде AST
 3. Изменение AST под требования целевого языка
 4. Вывод готового кода

Подсказка: Подумайте о порядке обработки кода.

- **Tokenization** — Преобразование исходного кода в токены
- **Parsing** — Создание структуры кода в виде AST
- **Transformation** — Изменение AST под требования целевого языка
- **Code Generation** — Вывод готового кода

🔮 Шаг 07:

Миссия 7

Расположите этапы работы собственного transpiler в правильном порядке:

1. Разбиение кода на токены
2. Построение AST
3. Трансформация AST
4. Генерация целевого кода

Подсказка: Логика работы идет от исходного кода к готовому результату.

Ответ:

- Разбиение кода на токены
- Построение AST
- Трансформация AST
- Генерация целевого кода

🔮 Шаг 08:

Миссия 8

Расположите шаги от написания функции до её вызова через transpiler:

1. Определение функции
2. Анализ AST функции
3. Генерация кода для вызова
4. Выполнение сгенерированного кода

Подсказка: Сначала создаём, потом анализируем и только потом вызываем.

Ответ:

- Определение функции
- Анализ AST функции
- Генерация кода для вызова
- Выполнение сгенерированного кода

🔮 Шаг 09:

Миссия 9

Вы хотите, чтобы ваш transpiler поддерживал несколько целевых языков. Какой подход предпочтителен?

Подсказка: Подумайте о разделении логики генерации кода.

Ответ:

- **Создать отдельные генераторы для каждого языка**
- Использовать один генератор без изменений
- Генерировать код вручную
- Игнорировать разные языки

🔮 Шаг 10:

Миссия 10

Что следует делать перед трансформацией AST?

Подсказка: Думайте о корректности и целостности структуры дерева.

Ответ:

- Проверить корректность синтаксиса
- Генерировать конечный код сразу
- Запустить программу без AST
- Удалить все комментарии

Шаг 11:

Миссия 11

 Сопоставьте элементы transpiler с их задачами:

Ответ:

- AST Node
 - Token
 - Visitor
 - Target Code
1. Представляет отдельный элемент кода в дереве
 2. Минимальная единица исходного текста
 3. Обходит дерево и применяет преобразования
 4. Результат работы transpiler

Подсказка: Вспомните роль каждого компонента в процессе трансформации.

- **AST Node** — Представляет отдельный элемент кода в дереве
- **Token** — Минимальная единица исходного текста
- **Visitor** — Обходит дерево и применяет преобразования
- **Target Code** — Результат работы transpiler

Шаг 12:

Миссия 12

 Сопоставьте типы ошибок transpiler с их примером:

Ответ:

- Syntax Error
 - Transformation Error
 - Runtime Error
 - Code Generation Error
1. Неправильное расположение скобок
 2. Ошибка при изменении AST узла
 3. Ошибка выполнения сгенерированного кода
 4. Ошибка вывода кода на целевой язык

Подсказка: Ошибки могут возникать на разных этапах обработки.

- **Syntax Error** — Неправильное расположение скобок
- **Transformation Error** — Ошибка при изменении AST узла
- **Runtime Error** — Ошибка выполнения сгенерированного кода
- **Code Generation Error** — Ошибка вывода кода на целевой язык

🔮 Шаг 13:

Миссия 13

Как проверить корректность работы transpiler на маленьких примерах?

Подсказка: Начните с простых программ.

Ответ:

- **Сравнить вывод с ожиданием на тестовых примерах**
- Запускать только большие проекты
- Игнорировать тесты
- Редактировать вручную каждый результат

🔮 Шаг 14:

Миссия 14

Вы хотите логировать процесс работы transpiler. Что эффективнее всего записывать?

Подсказка: Подумайте о критических этапах обработки.

Ответ:

- **AST до и после трансформации**
- Только исходный код
- Только конечный код
- Только имена файлов

🔮 Шаг 15:

Миссия 15

Расположите этапы обработки программы через transpiler с учётом тестирования:

1. Разбор кода
2. Трансформация AST
3. Генерация кода
4. Сравнение результата с тестами

Подсказка: После генерации кода следует проверка его правильности.

Ответ:

- Разбор кода
- Трансформация AST
- Генерация кода
- Сравнение результата с тестами

🔮 Шаг 16:

Миссия 16

Расположите шаги для добавления нового синтаксического правила в transpiler:

1. Изменение парсера
2. Обновление AST Transformer
3. Добавление генерации кода
4. Тестирование нового правила

Подсказка: Сначала нужно научить parser понимать новый синтаксис.

Ответ:

- Изменение парсера
- Обновление AST Transformer
- Добавление генерации кода
- Тестирование нового правила

 Шаг 17:

Миссия 17

 Сопоставьте типы AST узлов с примерами кода:

Ответ:

- BinOp
- Compare
- Return
- While

1. $x + y$
2. $x > 10$
3. `return x`
4. `while x < 5:`

Подсказка: Обратите внимание на оператор и контекст выражения.

- **BinOp** — $x + y$
- **Compare** — $x > 10$
- **Return** — `return x`
- **While** — `while x < 5:`

 Шаг 18:

Миссия 18

Вы хотите оптимизировать transpiler для больших проектов. С чего начать?

Подсказка: Сосредоточьтесь на повторяющихся процессах.

Ответ:

- **Кэширование промежуточных результатов AST**
- Игнорирование ошибок во время трансформации
- Удаление всех комментариев
- Генерация кода без AST

 Шаг 19:

Миссия 19

 Сопоставьте элементы transpiler с их назначением:

Ответ:

- Parser
- Lexer
- Code Emitter
- Optimizer

1. Разбивает исходный код на токены

2. Создает AST из токенов
3. Генерирует код для целевого языка
4. Улучшает производительность кода

Подсказка: Каждый элемент выполняет определённую задачу в цепочке.

- **Parser** — Создает AST из токенов
- **Lexer** — Разбивает исходный код на токены
- **Code Emitter** — Генерирует код для целевого языка
- **Optimizer** — Улучшает производительность кода

Урок 5 - Финал Python, пишущий Python

Шаг 01:

Миссия 1

Что является первым шагом при написании Python-кода, который генерирует другой Python-код?

Подсказка: Подумайте о структуре данных для представления кода.

Ответ:

- **Создать AST для исходного кода**
- Непосредственно записывать строки кода в файл
- Запустить интерпретатор без анализа кода
- Игнорировать структуру кода

Шаг 02:

Миссия 2

Какой инструмент Python помогает безопасно модифицировать и генерировать код?

Подсказка: Он позволяет работать с синтаксическим деревом программы.

Ответ:

- **Модуль ast**
- Модуль os
- Модуль math
- Модуль random

Шаг 03:

Миссия 3

 Сопоставьте элементы Python-транспайлера с их назначением:

Ответ:

- AST Parser
- Code Generator
- Node Transformer
- Code Executor

1. Преобразует AST в выполняемый Python-код
2. Разбирает код в дерево синтаксиса
3. Выполняет модификации узлов AST
4. Запускает сгенерированный код

Подсказка: Обдумайте цепочку от анализа к выполнению.

- **AST Parser** — Разбирает код в дерево синтаксиса
- **Code Generator** — Преобразует AST в выполняемый Python-код
- **Node Transformer** — Выполняет модификации узлов AST
- **Code Executor** — Запускает сгенерированный код

💡 Шаг 04:

Миссия 4

Что важно проверять при автоматической генерации кода?

Подсказка: Код может быть корректным синтаксически, но неправильным логически.

Ответ:

- **Синтаксис и логическую корректность кода**
- Только стиль оформления
- Ничего не проверять
- Только наличие комментариев

💡 Шаг 05:

Миссия 5

📖 Сопоставьте термины с определениями при генерации кода:

Ответ:

- Transpiler
 - Macro
 - Decorator
 - AST Node
1. Элемент синтаксического дерева
 2. Функция, изменяющая поведение другой функции
 3. Инструмент для преобразования кода в код на том же или другом языке
 4. Механизм генерации повторяющихся фрагментов кода

Подсказка: Подумайте о разных уровнях кода — от структуры до поведения.

- **AST Node** — Элемент синтаксического дерева
- **Decorator** — Функция, изменяющая поведение другой функции
- **Transpiler** — Инструмент для преобразования кода в код на том же или другом языке
- **Macro** — Механизм генерации повторяющихся фрагментов кода

💡 Шаг 06:

Миссия 6

Расположите этапы создания Python-кода, который пишет Python, в правильном порядке:

1. Анализ исходного кода (Parsing)
2. Модификация AST
3. Генерация нового кода
4. Запуск и тестирование сгенерированного кода

Подсказка: Думайте как транслятор: сначала читаем, потом меняем, потом создаём, потом проверяем.

Ответ:

- Анализ исходного кода (Parsing)
- Модификация AST
- Генерация нового кода
- Запуск и тестирование сгенерированного кода

 Шаг 07:

Миссия 7

Какая функция Python позволяет динамически выполнять сгенерированный код?

Подсказка: Подумайте о встроенных функциях для исполнения строк кода.

Ответ:

- **exec()**
- eval_ast()
- run()
- compile_code()

 Шаг 08:

Миссия 8

Что необходимо учитывать при генерации тестов для кода, который сам себя модифицирует?

Подсказка: Подумайте о повторяемости и предсказуемости тестов.

Ответ:

- **Стабильность и повторяемость тестов**
- Количество комментариев
- Использовать random без контроля
- Игнорировать ошибки

 Шаг 09:

Миссия 9

 Сопоставьте элементы транспайлера с их ролью:

Ответ:

- Parser
- Transformer
- CodeEmitter
- Runtime Checker

1. Проверяет поведение сгенерированного кода во время выполнения
2. Преобразует исходный код в AST
3. Применяет изменения к узлам AST
4. Создаёт финальный исполняемый Python-код

Подсказка: Следуйте логике анализа → изменения → генерация → проверка.

- **Parser** — Преобразует исходный код в AST
- **Transformer** — Применяет изменения к узлам AST
- **CodeEmitter** — Создаёт финальный исполняемый Python-код
- **Runtime Checker** — Проверяет поведение сгенерированного кода во время выполнения

🔮 Шаг 10:

Миссия 10

Что ускоряет разработку Python-транспайлера для повторяющихся шаблонов кода?

Подсказка: Подумайте о способе повторного использования кода.

Ответ:

- **Использование макросов и шаблонов**
- Писать все вручную
- Игнорировать повторения
- Только комментарии

🔮 Шаг 11:

Миссия 11

📄 Сопоставьте элементы системы с их функцией:

Ответ:

- Unit Test
 - Integration Test
 - AST Validator
 - Code Sandbox
1. Тестирует отдельные функции кода
 2. Проверяет корректность AST
 3. Используется для безопасного запуска сгенерированного кода
 4. Проверяет взаимодействие модулей

Подсказка: Тестирование может быть на уровне узлов, функций или всей системы.

- **Unit Test** — Тестирует отдельные функции кода
- **Integration Test** — Проверяет взаимодействие модулей
- **AST Validator** — Проверяет корректность AST
- **Code Sandbox** — Используется для безопасного запуска сгенерированного кода

🔮 Шаг 12:

Миссия 12

Какой подход помогает отлаживать динамически генерируемый код?

Подсказка: Подумайте о способе выявления ошибок на раннем этапе.

Ответ:

- **Локальная проверка и поэтапное тестирование**
- Игнорировать ошибки
- Вставлять print в каждую строку
- Оставлять код без проверки

🔮 Шаг 13:

Миссия 13

Расположите шаги транспилиции Python-кода по порядку:

1. Парсинг исходного кода
2. Преобразование AST

3. Генерация кода из AST
4. Выполнение и тестирование

Подсказка: Логика: сначала читаем → меняем → создаём → проверяем.

Ответ:

- Парсинг исходного кода
- Преобразование AST
- Генерация кода из AST
- Выполнение и тестирование

Шаг 14:

Миссия 14

Что делает Node Transformer в транспайлере?

Подсказка: Подумайте о роли изменений внутри AST.

Ответ:

- **Модифицирует узлы AST перед генерацией кода**
- Создаёт текстовый файл с кодом
- Запускает код
- Только читает комментарии

Шаг 15:

Миссия 15

 Сопоставьте типы ошибок с примерами при генерации кода:

Ответ:

- Синтаксическая ошибка
 - Логическая ошибка
 - Runtime ошибка
 - Ошибки безопасности
1. Использование переменной до её определения
 2. Пропущенные двоеточия или неправильные отступы
 3. Неправильная реализация алгоритма
 4. Выполнение опасного кода, например открытие файлов без проверки

Подсказка: Подумайте, что возникает во время компиляции, выполнения и что связано с безопасностью.

- **Синтаксическая ошибка** — Пропущенные двоеточия или неправильные отступы
- **Логическая ошибка** — Неправильная реализация алгоритма
- **Runtime ошибка** — Использование переменной до её определения
- **Ошибки безопасности** — Выполнение опасного кода, например открытие файлов без проверки

Шаг 16:

Миссия 16

Расположите этапы отладки Python-кода, который генерирует другой Python-код:

1. Написание генератора кода
2. Проверка сгенерированного кода на ошибки

3. Использование тестов для подтверждения корректности

4. Запуск сгенерированного кода в безопасной среде

Подсказка: Логика: сначала создаём → проверяем → тестируем → безопасно запускаем.

Ответ:

- Написание генератора кода
- Проверка сгенерированного кода на ошибки
- Использование тестов для подтверждения корректности
- Запуск сгенерированного кода в безопасной среде

 **Шаг 17:**

Миссия 17

Какой подход наиболее безопасен при генерации кода, который будет выполняться на сервере?

Подсказка: Подумайте о разделении доверенного и недоверенного кода.

Ответ:

- **Запуск в изолированном sandbox с проверкой входных данных**
- Выполнение без ограничений
- Игнорирование ошибок безопасности
- Ручная проверка каждого сгенерированного файла